

IT PROJECT PORTAL

IT Project Portal 문서 모음

루트 README와 README가 직접 연결한 로컬 Markdown 문서 17개

포함 문서

1. IT Project Portal
README.md
2. IT Portal 프론트엔드
it_frontend/README.md
3. IT Portal 프론트엔드 작업 계약
it_frontend/CLAUDE.md
4. 프론트엔드 개발 가이드
it_frontend/docs/guides/README.md
5. API 클라이언트 가이드
it_frontend/docs/guides/architecture/api-client.md
6. 공통 컴포넌트 가이드
it_frontend/docs/guides/components/common-components.md
7. 디자인 토큰 가이드
it_frontend/docs/guides/styling/design-tokens.md
8. IT Portal 백엔드
it_backend/README.md
9. IT Portal 백엔드 작업 계약
it_backend/CLAUDE.md
10. 백엔드 개발 가이드
it_backend/docs/guides/README.md
11. 데이터 모델 인덱스
it_backend/docs/guides/persistence/data-model.md
12. 인증과 인가 가이드
it_backend/docs/guides/security/authentication-authorization.md
13. Flyway 운영 가이드
it_backend/docs/guides/operations/flyway.md
14. IT Portal 데이터베이스
it_database/README.MD
15. IT Portal 데이터베이스 작업 계약
it_database/CLAUDE.md
16. 데이터베이스 개발 가이드
it_database/docs/guides/README.md
17. Flyway 마이그레이션 작성과 검증
it_database/docs/guides/migrations.md
18. 1. 프로젝트와 저장소
CLAUDE.md

IT Project Portal

SOURCE: README.md

IT Project Portal

정보화 예산, 사업, 인력, 문서 검토, 결재와 관리자 로그를 제공하는 사내 포털입니다.

저장소 구성

```
C:\it\  
├─ it_frontend/   Nuxt 4 CSR 프론트엔드  
├─ it_backend/    Spring Boot API  
├─ it_database/   DDL·시드·Flyway 마이그레이션  
├─ docs/          공통 설계·계획·검증 문서와 Superpowers 산출물  
├─ CLAUDE.md      공통 필수 규칙  
└─ TASK.md        미구현·기술부채
```

각 하위 저장소는 독립 Git 저장소입니다. 코드 변경과 테스트는 해당 저장소에서 수행합니다.

빠른 시작

1. Oracle 확인

- 접속 계정: ITPAPP
- 객체 스키마: ITPOWN
- 주소: 127.0.0.1:11521/XEPDB1

```
sqlplus ITPAPP@127.0.0.1:11521/XEPDB1
```

비밀번호는 sqlplus 콘솔 프롬프트에만 입력합니다. CI·무인 실행은 명령줄 또는 환경변수 비밀번호 대신 Oracle Secure External Password Store의 Wallet 별칭(/@별칭)을 사용합니다.

2. 백엔드

배포파일 빌드

```
cd C:\it\it_backend; ./gradlew bootWar -x test
```

```
cd C:\it\it_backend
$env:SPRING_PROFILES_ACTIVE = "local-ext"
./gradlew bootRun
```

- API: <http://localhost:28080>
- Swagger: <http://localhost:28080/swagger-ui/index.html>
- Health: <http://localhost:28080/actuator/health>

외부망 로컬 개발은 모의 SSO와 Flyway 자동 적용을 사용하는 `local-ext`, 내부 ESSO 연동 환경은 `local-int` 프로파일을 사용합니다. `dev` 와 `prod` 에서는 Flyway를 자동 실행하지 않습니다.

3. 프론트엔드

배포파일 빌드

```
cd C:\it\it_frontend; npm run generate:dev
```

```
cd C:\it\it_frontend
npm ci
npm run dev
```

- 화면: <http://localhost:3000>
- 기본 API 주소: <http://localhost:28080>

최초 체크아웃과 CI는 `package-lock.json` 을 그대로 재현하는 `npm ci` 를 사용합니다. 패키지를 추가하거나 버전을 바꿀 때만 `npm install` 로 lockfile을 함께 갱신합니다.

4. 정적 배포 빌드

프론트 정적 생성은 백엔드와 같은 `SPRING_PROFILES_ACTIVE` 값을 읽어 환경별 스크립트를 선택합니다. 코마로 여러 프로파일을 지정하면 첫 번째 값만 판정하며, `local*` 은 `.env.local`, `dev` 는 `.env.development`, `prod` 는 `.env.production` 을 사용합니다.

```
cd C:\it\it_frontend
$env:SPRING_PROFILES_ACTIVE = "prod"
npm run generate
```

모든 정적 생성 스크립트는 API 기준 주소를 빈 값으로 강제해 브라우저가 same-origin `/api/` 와 `/sso/` 를 호출하게 합니다. 따라서 nginx-WebTobe는 두 경로를 백엔드로 프록시해야 합니다. API 호출에는 CORS가 발생하지 않지만 SSO 완료 후 복귀 주소는 백엔드 Origin 허용 목록으로 검증하므로, 실제 프론트 Origin을 `APP_FRONTEND_URL` 또는 `CORS_ALLOWED_ORIGINS` 에 `scheme://host[:port]` 형식으로 등록합니다.

nginx.conf

```
server {
    listen      80;
    server_name localhost;
    root C:/it/it_frontend/.output/public; # 빌드된 public 폴더 경로

    location / {
        index index.html index.htm;
        try_files $uri $uri/ /index.html; # SPA
    }

    location /sso/ {
        proxy_pass http://localhost:28080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # 캐시 설정
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
        add_header Cache-Control "public, no-transform";
    }

    location /api/ {
        proxy_pass http://localhost:28080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # redirect server error pages to the static page /50x.html
    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root html;
    }
}
```

IDE 설정(VS Code)

`.vscode/settings.json` · `.vscode/launch.json` 은 **의도적으로 추적합니다**. 프로젝트 공통 함정의 회피책이 들어 있어 새 개발자가 같은 문제를 다시 겪지 않게 하는 것이 목적입니다. 대신 **머신 종속 값은 이 파일에 두지 않습니다** — 워크스페이스 설정이 사용자 설정을 덮으므로, 절대경로를 박아 두면 저장소를 다른 경로나 다른 PC에 클론한 사람의 Java 확장 임포트가 실패합니다(REPO-01).

각자의 사용자 설정에 넣을 값

`Ctrl+Shift+P` → `Preferences: Open User Settings (JSON)` 에 아래 두 값을 자기 PC 경로로 넣습니다.

```
{
  // Lombok javaagent 경로 - 클론 위치에 맞게 고칩니다.
  // 이 설정은 키 단위로 덮이므로 gc.힙 옵션까지 함께 적어야 합니다.
  "java.jdt.ls.vmargs": "-XX:+UseParallelGC -XX:GCTimeRatio=4 -XX:AdaptiveSizePolicyWeight=90 -Dsun.zip.disableMemoryMapping=true -Xmx2G -Xms100m -Xlog:disable -javaagent:C:/it/it_backend/lib/lombok.jar",
  // 직접 풀어 둔 Gradle 설치본 경로 (버전은 gradle/wrapper/gradle-wrapper.properties와 맞춥니다)
  "java.import.gradle.home": "C:/gradle/gradle-9.2.1"
}
```

`java.import.gradle.home` 이 필요한 이유는 워크스페이스 설정의 `java.import.gradle.wrapper.enabled: false` 와 짝입니다. 어떤 환경에서는 `.gradle\wrapper\dists` 의 zip rename이 보안 프로그램에 막혀 Gradle 확장(`vscjava.vscode-gradle`)이 배포판을 매번 삭제·재설치하다 실패합니다. 터미널의 `./gradlew` 는 wrapper를 그대로 쓰므로 이 설정과 무관하게 동작합니다.

standalone Gradle을 직접 실행하지 않습니다

빌드·테스트는 항상 저장소의 Wrapper(`./gradlew`)로 실행합니다. PATH나 IDE에 다른 버전의 Gradle 설치본이 있어도 `it_backend` 에서 `gradle ...` 을 직접 실행하지 않습니다.

- standalone Gradle로 `wrapper` 태스크가 돌면(직접 입력했든 IDE·도구가 대신 실행했든) `gradle/wrapper/gradle-wrapper.properties` 가 **그 설치본 버전으로 덮어써집니다**. `distributionSha256Sum` 은 이전 값이 그대로 남아 URL과 짝이 어긋나고, 폐쇄망용 `file:///c:/maven-repo/...` 주석 블록도 함께 지워집니다.
- 덮어쓴 버전이 구버전이면 Java 확장 임포트가 `Can't use Java 25.0.2 and Gradle 8.9 to import Gradle project it_backend` 로 실패합니다. Gradle 8.9의 실행 JVM 상한은 Java 22이고 Java 25 실행 지원은 Gradle 9.1부터라, JDK 25 위에서는 동작하지 않습니다. 이 프로젝트 기준은 **Gradle 9.2.1 + JDK 25**입니다.
- 같은 증상이 특정 PC에서만 난다면 그 PC의 사용자 설정 `java.import.gradle.home` · `java.import.gradle.version` 이 옛 버전을 가리키는지도 함께 확인합니다.

이미 덮어썼다면 이렇게 되돌립니다.

```
git -C C:\it\it_backend checkout -- gradle/wrapper/gradle-wrapper.properties
Remove-Item -Recurse -Force C:\it\it_backend\gradle\8.9 # 덮어쓴 버전의 캐시 폴더
```

이후 VS Code에서 `Java: Clean Java Language Server Workspace` 로 재임포트합니다. Wrapper 버전을 실제로 올릴 때는 `./gradlew wrapper --gradle-version <버전>` 으로 갱신하고, `distributionUrl` · `distributionSha256Sum` · 폐쇄망 주석이 모두 맞는지 확인한 뒤 커밋합니다.

지우면 기동이 깨지는 설정

`java.import.exclusions` 의 두 항목은 취향이 아니라 **장애 회피책**입니다.

제외 경로	지우면 생기는 일
<code>**/bin/</code> <code>**</code>	<code>bin</code> 아래에 <code>.project</code> · <code>build.gradle</code> 이 생기면 JDt가 이를 별도 프로젝트로 임포트해 같은 클래스가 두 출력 루트(<code>bin/main</code> , <code>build/classes/java/main</code>)로 올라갑니다. Spring 기동이 <code>BeanDefinitionOverrideException (already defined)</code> 으로 실패합니다.
<code>**/.worktrees/*</code> <code>*</code>	<code>.worktrees</code> 하위에 같은 이름의 <code>it_backend</code> 체크아웃이 있어 JDt가 프로젝트명 <code>it-it_backend</code> 를 중복 등록하고 <code>Duplicate root element</code> 로 임포트가 실패합니다. 워크트리는 각자의 창에서 엽니다.

`launch.json` 의 `classPaths` 에 `$Runtime` 과 출력 폴더 제외를 명시한 이유도 같은 계열입니다. 기본값(`$Auto`)은 테스트 출력 폴더까지 런타임 클래스패스에 올려, 테스트의 중첩 `@SpringBootTestConfiguration` 이 컴포넌트 스캔에 걸리고 JPA 리포지토리 빈이 두 번 등록되어 기동이 실패합니다.

품질 확인

프론트엔드:

```
cd C:\it\it_frontend
npm run format:check
npm run check
npm test
npm run test:e2e
```

백엔드:

```
cd C:\it\it_backend
./gradlew spotlessCheck
./gradlew test
./gradlew integrationTest
./gradlew jacocoTestCoverageVerification
./gradlew check
```

백엔드 기본 테스트는 로컬 Oracle 의존 통합 테스트를 제외하며, 실제 Oracle 매핑과 QueryDSL은 `integrationTest` 로 분리합니다. 브라우저 핵심 흐름은 두 서버를 실행한 뒤 `npm run test:e2e` 또는 Playwright MCP로 확인합니다.

전체 요청 흐름

Nuxt 페이지·컴포넌트

- composable / Pinia store
- useApiFetch(GET) 또는 \$apiFetch(명령형 요청)
- Spring Security·JWT 쿠키 필터
- Spring Controller(DTO·Bean Validation)
- Service(권한·트랜잭션·업무 규칙)
- Repository(JPA·QueryDSL)
- Oracle ITPOWN 스키마

- 프론트가 사용하는 응답 타입은 백엔드 OpenAPI 스펙(/v3/api-docs)에서 생성합니다(`npm run codegen` → `it_frontend/app/types/api.d.ts`). 백엔드 응답 DTO의 `@Schema` 제약이 프론트 타입의 단 일 출처이며, 스펙 변경 후 재생성 누락은 `npm run codegen:check` 가 잡습니다.
- 프론트는 Nuxt `ssr:false` 정적 CSR이므로 인증 진입과 세션 복원은 클라이언트 `auth.global.ts` 가 담당합니다. 핵심 E2E는 API를 모킹하고 Nuxt 개발 서버도 Playwright가 자동 기동하므로, 실제 백엔 드 연동 검증과 구분해 실행합니다.
- 프론트는 `runtimeConfig.public.apiBase` 를 접두사로 구성한 API URL과 인증 쿠키를 사용합니다. BioAgent 설치 안내는 `NUXT_PUBLIC_ONEPASS_INSTALL_URL` , OpenAPI 타입 생성은 `OPENAPI_BACKEND_URL` 을 별도로 사용합니다.
- 개발 실행은 절대 API URL을 사용하고, 정적 프록시 배포는 빈 API 접두사와 same-origin `/api/` · `/sso/` 경로를 사용합니다.
- 백엔드는 Controller에서 입력을 받고, Service에서 JWT 사용자 기준 부서·소유권·상태 전이를 검증합 니다.
- 목록·검색은 필요한 경우 QueryDSL 프로젝션을 사용하고, 물리 DB 변경은 애플리케이션 코드와 분리 된 Flyway 스크립트로 관리합니다.
- 결재 상태처럼 원 트랜잭션과 함께 성공해야 하는 처리는 동기 이벤트로 연결하고, 알림·메일처럼 원 업무를 롤백하면 안 되는 부수효과는 커밋 이후 별도 트랜잭션으로 처리합니다.

- 업무·검증 예외는 백엔드 전역 처리기가 일관된 JSON 오류 응답으로 변환하고, 프론트 페이지·다이얼 로그가 사용자 동작 맥락에 맞는 toast와 이동 여부를 결정합니다.

새 API나 조회 흐름을 추가할 때는 백엔드 DTO·OpenAPI 계약과 계약 테스트를 먼저 확정합니다. 목록은 DB 프로젝트·안정 정렬·페이지 상한을 사용하고, 여러 ID를 조립하는 응답은 IN 배치 조회를 사용합니다. 프론트는 계약 확정 후 `npm run codegen` 과 `npm run codegen:check` 를 실행하며, 정상 빈 결과·초기 조회 실패·재조회 실패를 서로 다른 UI 상태로 유지합니다.

주요 업무 모듈

흐름	프론트 진입점	백엔드 책임
정보화사업·예산	<code>app/pages/info/projects</code> , <code>app/pages/info/cost</code> , <code>app/pages/budget</code>	정보화사업·경상사업·전산업무비, SNO 기반 재상신 이력과 결재 연계
사업계획	<code>app/pages/project/bizplan</code>	계획 대상 확인, 보고서·일정·품목·계약 병합, 작성 상태 관리
사업 집행	<code>app/pages/project</code>	소요예산 산정 → 과업심의 → 입찰계약 → 대금지급 상태 전이
문서·사전 협의	<code>app/pages/info/documents</code>	문서 버전, 검토 의견, 첨부파일과 결재 연결
홈 대시보드	<code>app/pages/info/index.vue</code>	연도별 사업·예산 KPI, 예산 일정, 공지·일정 피드, 홈 배너
수기 엑셀 이관	<code>app/pages/admin/migration</code>	편성요청서 분석·반입, 원본 파일 보관과 결재 연계
전자결재	<code>app/pages/approval</code>	결재 대시보드·목록·상세와 승인·반려·회수 명령
공통 게시판	<code>app/pages/board</code>	게시글·댓글·첨부파일·멘션 관리. 본문은 Tiptap HTML로 저장하고 서버 정화 후 표시
공통 안내 팝업	<code>app/pages/admin/common-popup.vue</code> , 공통 <code>AppShell</code>	TPRMPP_BGDOCM 의 <code>common.popup</code> 게시·중지와 콘텐츠 버전별 안내 표시
스피드다이얼	공통 <code>AppShell</code> , <code>app/pages/admin/contact-information.vue</code>	전역 FAQ 조회·Q&A 등록·담당자 정보 열람. FAQ·Q&A는 공통 게시판 유형(004 · 005)을 재사용
사용자 가이드	헤더 버튼, <code>app/pages/admin/user-guides.vue</code>	전사 공개 가이드 파일 업로드·내려받기·이력 되돌리기. 현재 가이드는 항상 0건 또는 1건
공통 데이터 이관	<code>app/pages/admin/migration/common-data.vue</code>	메뉴·메뉴권한·경로·공통코드·다국어를 개발→운영으로 내보내기·dry-run·확정 반영

흐름	프론트 진입점	백엔드 책임
사전진단·가이드	<code>app/pages/diagnosis</code> , <code>app/pages/guide</code>	클라이언트 설문 결과와 서버 문서·첨부·변수 카탈로그 조회
공통 기능	공통 레이아웃·메뉴·알림·관리자 화면	인증, MFA, IAM, 서버 권한 메뉴, 게시판, 알림, 다국어, 배너, 입력 길라잡이, 감사·실시간 로그와 WAS 로그

메뉴는 백엔드가 사용자 권한으로 필터링한 `/api/menus` 트리를 프론트 헤더·사이드바·Breadcrumb·상단 탭이 함께 사용합니다. 탭 제목도 화면에 하드코딩하지 않고 이 트리의 메뉴명을 따릅니다. 프론트 메뉴 숨김은 화면 편의를 위한 것이며 API 접근 권한을 대신하지 않습니다.

주요 설계 원칙

- JWT는 백엔드가 httpOnly 쿠키로 발급하고 프론트가 직접 저장하거나 읽지 않습니다.
- 프론트 라우트 가드는 UX 보호이며 백엔드 권한 검증이 최종 보안 경계입니다.
- 백엔드 접속 계정과 객체 소유 스키마를 분리하고 세션 `CURRENT_SCHEMA=ITPOWN` 을 사용합니다.
- 업무 엔티티는 Soft Delete와 감사 로그 패턴을 사용합니다.
- Oracle 문자셋은 `AL32UTF8` 이고 `VARCHAR2` 는 BYTE semantics를 사용합니다. 저장 한도는 UTF-8 실제 바이트 길이로 판정하며, 신규·변경 입력은 프론트와 백엔드의 공통 바이트 유틸로 검증하고 신규 DDL은 `VARCHAR2(n BYTE)` 를 명시합니다.
- 결재완료된 정보화사업·경상사업·전산업무비의 재상신은 같은 관리번호에서 다음 SNO의 비최종 초안을 만들고, 승인 완료 시 해당 순번을 최종본으로 전환합니다. 후속 업무 목록·집계는 `LST_YN='Y'` 만 사용하되 미상신 작성 목록에는 재상신 초안도 표시합니다.
- 공통코드·메뉴 권한·알림 미읽음 수·Tiptap 메타데이터는 Caffeine 캐시를 사용하며, 원본 변경 시 서비스 캐시를 무효화합니다.
- 데이터베이스 변경은 `it_database/migrations` 의 새 Flyway 스크립트로 관리합니다.
- 운영 정적 생성은 백엔드와 공유하는 `SPRING_PROFILES_ACTIVE` 를 사용하며, 미설정·미지원 프로파일은 fail-fast로 중단합니다.
- 신규 주석은 한글로 작성하고 코드만 읽어도 명확한 설명은 생략합니다.

세부 규칙은 각 저장소의 `CLAUDE.md` 와 가이드를 확인합니다.

문서 역할

문서	역할
<code>README.md</code>	설치, 실행, 저장소 탐색
<code>CLAUDE.md</code>	반드시 지킬 공통 규칙
<code>it_frontend/CLAUDE.md</code>	프론트 필수 규칙

문서	역할
it_backend/CLAUDE.md	백엔드 필수 규칙과 보안 경계
it_database/CLAUDE.md	DB 변경·마이그레이션 안전 규칙
docs/guides	주제별 상세 설명과 예제
docs/operations	배포·복구·인계 기록
TASK.md	미구현, 기술부채, 후속 검증
it_database/migrations	물리 DB 변경 이력

상세 문서

프론트엔드

- [프론트 빠른 시작](#)
- [프론트 필수 규칙](#)
- [프론트 가이드 인덱스](#)
- [API 클라이언트](#)
- [공통 컴포넌트](#)
- [디자인 토큰](#)

백엔드

- [백엔드 빠른 시작](#)
- [백엔드 필수 규칙](#)
- [백엔드 가이드 인덱스](#)
- [데이터 모델 인덱스](#)
- [인증과 인가](#)
- [Flyway 운영](#)

데이터베이스

- [데이터베이스 빠른 시작](#)
- [데이터베이스 필수 규칙](#)
- [데이터베이스 가이드 인덱스](#)
- [마이그레이션 작성과 검증](#)

AI 작업 흐름

에이전트의 필수 작업 순서, 스킬 우선순위와 검증 진입점은 [CLAUDE.md](#) §4~5를 따릅니다. 사람은 위 「품질 확인」 명령과 각 저장소 README의 개발·배포 절차로 동일한 검증을 수행할 수 있습니다.

변경 이력

새 개발자가 최근 흐름을 파악할 수 있도록 주요 변경을 최신순으로 기록합니다. 시점성 정보(파일 수·테스트 건수 등)는 여기 또는 [TASK.md](#)에만 남기고 [CLAUDE.md](#)에는 남기지 않습니다.

- 2026-09-03: 공통 안내 팝업 — 관리자가 `TPRMPP_BGDOCM`의 `common.popup` 문서를 게시·중지하고, 인증 화면의 공통 AppShell이 콘텐츠 버전별 안내 다이얼로그를 표시. "다시 보지 않기"는 1년 쿠키로 관리하며 내용 변경 시 다시 표시
- 2026-09-02: 게시글 본문 리치에디터 전환과 멘션 — 게시글 본문을 `textarea`에서 Tiptap으로 바꾸고, 저장된 멘션 식별자를 보존해 상세에서 직원 정보 버튼으로 표시
- 2026-09-01: 결재선 편집 — 결재중 신청서의 미결 결재선을 다이얼로그에서 원자적으로 교체. 완료된 결재자의 순서는 보존하고, 상태 변경 명령은 신청서 행을 잠근 뒤 수행
- 2026-09-01: DB 문자열 기준 확정 — ITPOWN의 `VARCHAR2`를 `AL32UTF8 BYTE semantics`로 통일하고, 공통 유틸과 주요 사업·이관 입력을 UTF-8 실제 바이트 길이 기준으로 정렬
- 2026-09-01: 예산 재상신 — 정보화사업·경상사업은 `BPROJM.SNO`, 전산업무비는 `BCOSTM.BG_SNO`로 결재완료본의 다음 순번 초안을 만들고 승인 시 최종본을 전환하도록 변경
- 2026-08-31: 사용자가이드 — 전용 테이블 없이 공통첨부파일(`TPRMPP_CFILEM`)을 재사용해 전사 공개 가이드를 관리. 업로드·되돌리기가 같은 트랜잭션에서 기존 활성 건을 내려 현재 가이드를 항상 0건 또는 1건으로 유지
- 2026-08-30: 스피드다이얼 — 관리자 화면을 제외한 전 화면에 FAQ·Q&A·담당자 정보 전역 버튼 신설. FAQ·Q&A는 공통 게시판 유형을 재사용하고, Q&A 등록 시 현재 화면 이름·경로를 함께 남김
- 2026-08-29: 공통 데이터 이관 — 메뉴·메뉴권한·경로·공통코드·다국어 5개 테이블을 `xlsx`로 내보내 운영에 dry-run 후 확정 반영. 메뉴 반입 뒤 시퀀스를 파일 최대 번호 이상으로 전진
- 2026-08-29: 금융정보단말기 일괄 반입 — 단말기 업로드 행을 연도별 전산업무비 원장으로 검증·반영
- 2026-08-28: 정적 분석 취약점 조치 — Refresh Token·MFA 거래의 저장 지문을 무키 SHA-256에서 서버 비밀키와 용도 구분자를 쓰는 HMAC-SHA256으로 전환하고, 프론트 식별자 난수의 `Math.random()` 폴백을 제거해 미지원 환경에서는 명시적으로 실패하도록 변경
- 2026-08-28: 결재선 관리 — 결재중 신청서에 미결재 결재자를 추가·삭제하고 순서를 조정. 결재자 구분 플래그는 도입 당일 철회해 모든 결재자를 동일하게 취급
- 2026-08-27: 예산작성 화면 카드 메모 — 정보화사업·전산업무비·경상사업 카드의 안내 메모를 문서(`BGDOCM`)로 보관하고 관리자만 편집
- 2026-08-25: 사업 입력 길라잡이 — 서버 고정 카탈로그가 정한 입력 필드에 관리자가 안내 본문을 등록하고(`/admin/form-guides`), 사업·전산업무비 입력 폼이 활성 필드의 안내를 옆 패널로 표시. 본문은 저장 시 서버 정화, 표시 시 클라이언트 정화를 모두 거침
- 2026-08-24: 정보화사업 금액 의미 정리 — 당해금액과 총소요금액을 구분하고 품목 예정금액을 독립 금액으로 저장. 외화 예정금액의 원화 환산과 저장 단위 반올림·범위 검증을 한 곳으로 모으고, 계속사업 금액의 자동 이월을 제거

- 2026-08-24: 공통첨부파일 부모 키 컬럼 개명·폭 축소 — 실제 쓰임이 "첨부파일 종류와 연결 콘텐츠"였던 두 컬럼을 이름과 폭에 맞게 정정(BE-51 후속)
- 2026-08-22: 준비중 메뉴 — 관리자가 메뉴를 준비중으로 표시하면 서버가 `/preparing/{mnuId}` 경로를 채번해 라우트 카탈로그에 함께 등록하고, 준비중 해제·삭제 시 자동 생성한 경로만 회수를 채번해 라우트 카탈로그에 함께 등록하고, 준비중 해제·삭제 시 자동 생성한 경로만 회수
- 2026-08-22: 게시판 첨부 일괄 내려받기(ZIP)와 일정 게시판 도입 — 일정 게시판은 시작·종료일자를 공개기간이 아니라 일정으로 다뤄 공개기간 필터에서 제외
- 2026-08-20: 편성요청서 반입 원본 파일 트리과 진행률 표시 — 부서 폴더째 드롭 업로드, 분석·반입 배치 진행률 다이얼로그, 부서별 원본 파일·진단 다이얼로그, 원본 ZIP 내려받기
- 2026-08-20: SEC-16 MFA 다중 인스턴스 지원 — MFA·로그인대기 거래를 Oracle 공유 테이블 (`TPRMPP_CMFATM`/`TPRMPP_CMFADM`)로 옮기고 서비스·공급자의 인스턴스 로컬 상태를 제거, 만료 행 정리 배치 추가
- 2026-08-20: 실시간 WAS 로그 뷰어(`/admin/was-logs`) — logback 링버퍼 적재, 관리자 전용 조회·다운로드, TTL이 붙은 런타임 로그레벨 변경, 다중 인스턴스 피어 팜아웃과 관리자 행위 감사
- 2026-08-19: `/info` 홈 개편 — 올해·내년 KPI 카드(건수형·금액형)와 예산 일정 카드, 부서·전체 조회 범위 전환, 홈 배너 캐러셀과 배너 관리 화면(`/admin/banners`) 신설
- 2026-08-19: 편성요청서 반입 판정 개선 — 대상 아닌 파일 SKIPPED 분리, 일반관리비 통화 공통코드 확정(미해석 행 차단·후보 제시), 전결권자 미매칭 진단에 공통코드 후보 제공
- 2026-08-18: 편성요청서 반입 원본 파일 보관·열람 — 반입 원본을 공통첨부파일로 보관(부서 교차 연결 차단)하고 결재현황 뷰어에서 원본 목록 표시, 파일 읽기/쓰기 authorizer 등록
- 2026-08-18: FE-37 사이트 전체 고정 문구 i18n 이관 완료(고정 문구 감사 기준선 0건 달성)
- 2026-08-18: 다국어 번역 변경로그(`TPRMPP_CLANGL`) 추가와 관리자 다국어 관리 화면(`/admin/translations`) 신설

IT Portal 프론트엔드

SOURCE: `it_frontend/README.md`

IT Portal 프론트엔드

정보화 예산, 사업, 문서 검토, 전자결재와 관리자 기능을 제공하는 Nuxt 4 기반 사내 포털 프론트엔드입니다. CSR 방식으로 정적 생성하여 배포합니다.

기술 스택

- Nuxt 4 / Vue 3 / TypeScript
- PrimeVue / Tailwind CSS / `@nuxtjs/i18n`
- Pinia
- Vitest / Playwright
- Tiptap / Excalidraw / ExcelJS / pdfmake / JSZip

정확한 버전은 `package.json` 과 `lockfile`을 기준으로 확인합니다.

Nuxt는 파일 기반 라우팅과 자동 임포트를 담당하고, Pinia는 인증·문서 검토처럼 여러 화면에서 이어지는 상태에만 사용합니다. PrimeVue 컴포넌트 위에는 Tailwind와 `assets/css`의 디자인 토큰·보정 레이어를 적용합니다. Tiptap·Excalidraw는 문서 편집, ExcelJS·pdfmake·JSZip은 Excel·PDF·HWPX 산출물 생성에 사용합니다.

빠른 시작

```
cd C:\it\it_frontend
npm ci
npm run dev
```

- 프론트엔드: <http://localhost:3000>
- 백엔드 기본 주소: <http://localhost:28080>

백엔드를 먼저 실행하고 로그인·결재·관리자 기능을 확인합니다. `package-lock.json`을 기준으로 동일한 의존성을 설치하려면 최초 체크아웃과 CI에서는 `npm ci`를 사용합니다. 패키지를 추가·변경할 때만 `npm install`로 `lockfile`을 함께 갱신합니다.

openapi-typescript (FE-15 codegen)는 typescript@^5.x 를 peer로 요구하지만 이 프로젝트는 typescript@^6.0.3 을 고정해서 씁니다(하향 금지). 두 버전이 충돌하는 것을 막기 위해 package.json 의 overrides 에 "openapi-typescript": { "typescript": "\$typescript" } 를 추가해 해당 peer 해석을 루트 typescript 버전으로 강제했습니다. 이 override 덕분에 npm ci / npm install 은 별도 플래그 없이 그대로 동작합니다 — --legacy-peer-deps 나 --force 가 필요 없습니다.

환경변수

공개 런타임 설정만 env 파일에 둡니다. 파일별 용도는 다음과 같습니다.

파일	용도	로드 명령	NUXT_PUBLIC_API_BASE 값
.env	로컬 개발	npm run dev (자동 로드)	http://localhost:28080
.env.local	로컬 nginx 정적 배포	npm run generate:local	빈 값 (same-origin 프록시)
.env.development	개발서버 정적 배포	npm run generate:dev	빈 값 (same-origin 프록시)
.env.production	운영서버 정적 배포	npm run generate:prod	빈 값 (same-origin 프록시)

npm run generate 는 백엔드와 공유하는 SPRING_PROFILES_ACTIVE 환경변수 하나로 배포 프로파일을 자동 선택합니다(scripts/generate.mjs). 콤마로 여러 프로파일이 지정되면 공백을 제거한 첫 번째 프로파일만 판정하며, local* (local-ext, local-int) → generate:local, dev → generate:dev, prod → generate:prod 로 매핑합니다. 미설정이거나 매핑 불가 값이면 안내와 함께 실패합니다. 프로파일별 스크립트는 --dotenv 로 해당 파일을 직접 지정하므로 .env 는 정적 생성에 로드되지 않습니다. 앱이 소비하는 환경변수는 NUXT_PUBLIC_API_BASE 하나이며, 값의 종류에 따라 의미가 다릅니다.

값	의미
미설정(undefined)	로컬 기본값 http://localhost:28080 사용
절대 URL	해당 주소로 교차 출처 직접 호출 (개발 서버, 내부망 IP 접속)
빈 문자열	same-origin 상대경로 호출 — 웹 서버가 /api/ · /sso/ 를 백엔드로 프록시하는 배포 전제

개발(npm run dev)에서는 .env 나 OS 환경변수로 덮어씁니다. 정적 생성 값은 빌드 시 번들에 포함되므로 빌드 전에 결정되어야 합니다. 비밀값은 NUXT_PUBLIC_* 에 넣지 않습니다. BioAgent 설치 안내에는 NUXT_PUBLIC_ONEPASS_INSTALL_URL 을 추가로 사용합니다.

OpenAPI 타입 생성은 런타임 API 주소와 분리합니다. OPENAPI_BACKEND_URL 은 scripts/codegen.mjs 가 /v3/api-docs 를 조회할 때만 사용하는 codegen 전용 변수입니다.

초기 조회는 정상 빈 결과와 실패를 구분해 표시하고, 같은 화면의 중복 인스턴스가 만드는 동일한 초기 요청은 in-flight 요청을 공유할 수 있습니다. 사용자가 누르는 재시도·새로고침은 독립적인 \$apiFetch 요청으로 실행하며, 실패한 재조회에서 이전 성공 데이터를 성공처럼 유지하지 않습니다.

내부망 IP로 접속할 때

`http://<내부IP>:3000` 으로 접속하면 API 주소도 같은 호스트로 맞춰야 합니다. 쿠키는 포트가 아니라 호스트 기준이라, 페이지는 IP-API는 `localhost` 인 조합에서는 인증 쿠키가 실리지 않습니다.

```
setx NUXT_PUBLIC_API_BASE "http://10.9.16.109:28080" # 새 터미널부터 적용
npm run dev
```

백엔드에도 같은 Origin을 허용해야 합니다(`APP_FRONTEND_URL`, `CORS_ALLOWED_ORIGINS`). 상세는 [백엔드 README](#)의 환경 설정을 따릅니다.

`setx` 로 영구 등록한 값은 `npm run dev`에만 영향을 줍니다. generate 스크립트는 `NUXT_PUBLIC_API_BASE`를 빈 값(same-origin)으로 강제 주입하므로 OS 환경변수 `.env` 값이 배포 빌드에 섞이지 않습니다(아래 [운영 빌드](#) 참조).

주요 명령어

명령	용도
<code>npm run dev</code>	개발 서버 실행
<code>npm run build</code>	Nuxt 애플리케이션 빌드
<code>npm run generate</code>	<code>SPRING_PROFILES_ACTIVE</code> 기준 프로파일 자동 선택 정적 생성
<code>npm run generate:local</code>	로컬 nginx용 CSR 정적 파일 생성
<code>npm run generate:dev</code>	개발서버용 CSR 정적 파일 생성
<code>npm run generate:prod</code>	운영서버용 CSR 정적 파일 생성
<code>npm run preview</code>	빌드 결과 미리보기
<code>npm run typecheck</code>	Nuxt-Nitro 타입 검사
<code>npm run lint</code>	ESLint 검사
<code>npm run lint:css</code>	Stylelint 검사
<code>npm run format:check</code>	Prettier 준수 확인
<code>npm run check</code>	타입 검사와 ESLint 일괄 실행
<code>npm run check:copy</code>	사용자 노출 고정 리터럴 ratchet 단독 실행
<code>npm run codegen</code>	백엔드 OpenAPI 스펙으로 <code>app/types/api.d.ts</code> 재생성

명령	용도
<code>npm run codegen:check</code>	스펙과 커밋된 생성 타입의 드리프트 검사 (파일 미기록)
<code>npm test</code>	Vitest 단위 테스트
<code>npm run test:watch</code>	Vitest 감시 모드
<code>npm run test:coverage</code>	단위 테스트 커버리지 생성
<code>npm run test:e2e</code>	Playwright E2E 테스트
<code>npm run test:e2e:core</code>	핵심 3시나리오 E2E (아래 참조)
<code>npm run test:e2e:static</code>	정적 생성물 대상 Playwright E2E
<code>npm run test:e2e:ui</code>	Playwright UI 모드 E2E
<code>npm run test:ui</code>	Vitest UI 모드
<code>npm run test:auth</code>	인증 관련 단위 테스트
<code>npm run generate-report</code>	E2E 결과 리포트 생성

핵심 E2E 정기 실행

배포 전(운영 `npm run generate` 직전)과 주요 화면 리팩터링 후에는 핵심 3시나리오 E2E를 실행한다.

```
npm run test:e2e:core
```

- 대상: 로그인(`auth.spec.ts`), 프로젝트 조회/생성(`projects.spec.ts`), 결재 처리(`approval.spec.ts`)
- 사전조건 없음: API는 `tests/e2e/helpers/mockApi.ts` 로 전면 모킹되고 인증은 테스트용 쿠키 주입 방식이라 백엔드.로컬 Oracle 기동이 필요 없다. dev 서버(3002)는 Playwright가 자동 기동한다.
- 전체 E2E는 `npm run test:e2e`, 결과 리포트는 `npm run generate-report`.

프로젝트 구조

```

it_frontend/
├─ app/
│   ├─ assets/      전역 CSS와 이미지
│   ├─ components/  공통·도메인 UI
│   ├─ composables/ 재사용 상태와 API 흐름
│   ├─ features/    도메인 기능 모듈 (전자결재 신청서 등)
│   ├─ layouts/     화면 레이아웃
│   ├─ middleware/  인증·권한 UX 가드
│   ├─ pages/       파일 기반 라우트
│   ├─ plugins/     Nuxt 플러그인
│   ├─ stores/      Pinia 상태
│   ├─ types/       TypeScript 타입
│   └─ utils/       순수 유틸리티
├─ i18n/           언어 설정과 도메인별 번역 메시지(`messages/`)
├─ shared/         앱·Nitro 양쪽에서 쓰는 순수 유틸
├─ server/         Nitro 미들웨어와 플러그인
├─ scripts/        정적 생성 프로파일 디스패처
├─ public/         정적 파일
├─ tests/unit/     단위 테스트
├─ tests/e2e/      브라우저 시나리오
├─ nuxt.config.ts  CSR·런타임 설정·모듈·전역 스타일
└─ package.json   실행·검사·테스트·정적 생성 명령

```

Nuxt 4 소스 루트는 `app/` 입니다.

새 기능을 추적할 때는 `pages` 에서 라우트와 화면 조합을 찾고, 그 페이지가 가져오는 도메인 `components` 와 `composables` 를 따라갑니다. 대부분의 페이지는 템플릿만 갖고 조회·상태·업무 흐름은 같은 이름의 화면 전용 `composable`(`composables/use{화면}Page.ts`)에 있으므로, 실제 로직은 페이지가 아니라 그 파일에서 읽습니다. 조회를 먼저 끝내야 하는 화면은 `fetch{화면}PageData()` 로 비동기 조회를 분리해 두었습니다 — 상태 구성을 동기로 남겨 컴포넌트의 lifecycle과 effect scope를 보존하기 위한 것입니다. 서버 요청·타입 변환은 `composable`, 여러 화면에서 유지되는 상태는 `stores`, 외부 상태가 필요 없는 포맷·계산은 `utils` 에서 확인합니다. `features` 는 전자결재 양식처럼 공통 화면에 플러그인 방식으로 연결되는 응집된 기능 단위입니다.

애플리케이션 구조와 부팅 흐름

`nuxt.config.ts` 의 `ssr: false` 설정으로 모든 화면을 브라우저에서 렌더링합니다. `app/app.vue` 는 현재 파일 기반 라우트에 레이아웃을 씌우고, 최근 페이지를 keep-alive로 유지하며, 전역 확인 다이얼로그와 Toast를 한 번만 배치합니다. 기본 화면은 `layouts/default.vue`, 로그인 화면은 `layouts/login.vue` 를 사용합니다. 관리자 화면도 `layouts/default.vue` 를 그대로 사용합니다 — 레이아웃 이름이 바뀌면 Nuxt가 `NuxtLayout` 서브트리를 새 key로 다시 마운트해 `app.vue` 의 keep-alive 캐시가 전부 비워지므로, 화면 골격이 같은 화면에는 별도 레이아웃을 만들지 않습니다.

초기 진입과 인증 복원은 다음 순서로 동작합니다.

1. HTML head의 인라인 스크립트가 테마 쿠키를 먼저 적용하고, 사용자 쿠키가 없는 보호 화면은 SSO 이동이 끝날 때까지 숨깁니다.
2. 개발 서버나 `nuxt preview`의 문서 요청은 `server/middleware/sso-auth-redirect.ts`가 먼저 검사하고, 미인증 요청을 백엔드 `/sso/business`로 보냅니다.
3. CSR 라우팅은 `middleware/auth.global.ts`가 `it-portal-user` 쿠키를 복원한 뒤 미인증 사용자를 SSO로 이동시킵니다. 원래 내부 경로는 검증된 `next` 값으로 보존합니다.
4. `stores/auth.ts`는 화면 복원에 필요한 사용자 정보만 Pinia와 `it-portal-user` 쿠키로 관리합니다. Access/Refresh Token은 백엔드가 발급한 httpOnly 쿠키에만 존재합니다.
5. 관리자 경로는 전역 가드와 페이지 미들웨어가 UX 수준에서 차단하며, 실제 권한과 데이터 범위는 백엔드가 다시 검증합니다.

라우트별 추가 가드는 페이지의 `definePageMeta`에서 선언합니다.

미들웨어	적용 목적	실패 시 동작
<code>admin</code>	시스템관리자 화면과 예산 편성·현황 보호	홈으로 이동
<code>plan</code>	정보기술부문 계획 화면 보호	홈으로 이동
<code>council-manager</code>	시스템관리자 또는 정보보호관리자의 협의회 개최준비 진입 확인	홈으로 이동
<code>budget-period</code>	사업·전산업무비 작성 화면의 신청 기간 확인	예산 메인으로 이동

이 미들웨어들은 화면 진입을 보조하는 클라이언트 UX 가드입니다. 협의회 심의유형 범위, 작성자·부서 범위, 관리자 권한의 최종 판정은 대응 백엔드 API가 수행합니다.

정적 배포에서는 `npm run generate`가 만든 `dist/`를 웹 서버가 제공하므로, 모든 SPA 경로를 `index.html`로 연결해야 합니다. 클라이언트 전역 미들웨어가 정적 배포 환경의 인증 진입도 담당합니다.

API, 인증, 메뉴의 연결

화면에서 백엔드 주소를 조합할 때는 항상 `runtimeConfig.public.apiBase`를 접두사로 사용합니다. 개발에서는 절대 URL, same-origin 정적 배포에서는 빈 접두사와 `/api/.../sso/...` 경로가 결합됩니다.

계층	역할	선택 기준
<code>composables/useApiFetch.ts</code>	<code>useFetch</code> 기반 반응형 조회, 쿠키 전송, 상태 코드별 사용자 알림, 토큰 갱신 후 조회 재실행	페이지 생명주기와 쿼리 Ref에 따라 갱신되는 GET
<code>plugins/auth.ts</code> 의 <code>\$apiFetch</code>	명령형 요청, 쿠키 전송, 동시 401의 단일 refresh 공유, 갱신 후 원 요청 재시도	POST/PUT/PATCH/DELETE와 저장 후 단건 재조회

계층	역할	선택 기준
stores/auth.ts 의 \$fetch	로그인, 로그아웃, 토큰 갱신과 개발용 사용자 전환	인증 플러그인과의 순환 참조를 피해야 하는 인증 액션

응답 타입은 손으로 쓰지 않고 백엔드에서 생성합니다. `npm run codegen` 이 기동 중인 백엔드의 `/v3/api-docs` 를 받아 `openapi-typescript` 로 `app/types/api.d.ts` 를 만들고, `app/types/*.ts` 의 도메인 타입은 그 스키마(`components['schemas']['X']`)를 alias해 재노출합니다. 따라서 백엔드 DTO의 `@Schema (required·nullable·enum)`가 프론트 타입의 단일 출처이며, 스펙을 바꾸고 재생성을 잊으면 `npm run codegen:check` 가 드리프트로 실패합니다(CI가 없으므로 이 확인이 유일한 게이트입니다). 생성물은 직접 수정하지 않습니다. 편집 중 신규 행처럼 서버 응답에 없는 필드가 필요한 화면 모델은 생성 타입을 넓히지 않고 별도 UI 타입(`types/cost-editor.ts`)으로 둡니다.

일반적인 요청 흐름은 `page → domain component/composable → useApiFetch` 또는 `$apiFetch → Spring Boot API` 입니다. 조회 `composable`은 `data`, `pending`, `error`, `refresh` 같은 반응형 상태를 페이지에 돌려주고, 저장 `composable`은 성공 결과를 반환하거나 예외를 호출자에게 전파합니다. 따라서 페이지·다이얼로그가 성공 toast, 실패 메시지, 이동 여부를 사용자 동작의 맥락에 맞게 결정합니다.

상단 헤더와 사이드바는 정적 배열이 아니라 `useMenu()` 가 `/api/menus` 에서 받은 트리를 함께 사용합니다. 백엔드는 역할에 맞는 메뉴만 반환하고, 프론트는 현재 경로와 가장 길게 일치하는 메뉴 경로를 찾아 활성 헤더와 사이드바 자식을 결정합니다. 같은 트리의 ID·경로 인덱스는 Breadcrumb에도 사용되며, 아이콘·관리자 표시·배지 연결처럼 화면 표현에만 필요한 규칙은 `utils/menuPresentation.ts` 에 둡니다.

헤더 아래 탭 목록도 같은 트리를 소비합니다. `AppHeader.vue` 가 라우트와 경로 역인덱스(`nodeByPath`)를 함께 감시하다가 현재 경로의 메뉴명을 `useTabs.addTab` 에 넘기므로, 탭 이름은 화면에 하드코딩한 문자열이 아니라 DB 메뉴명을 따릅니다. 메뉴에 없거나 메뉴명과 다르게 보여야 하는 화면만 `definePageMeta({ tabTitle })` 로 예외를 선언하고, 메뉴 조회가 탭 생성보다 늦게 끝나면 도착 시점에 기존 탭 제목을 다시 맞춥니다. 탭을 클릭하면 경로 이동만 일어나고 전역 데이터 재조회는 하지 않으며, 재방문 화면의 최신화는 각 페이지의 `onActivated` 재조회가 담당합니다.

메뉴 경로는 `PGE` 내부 경로, `LNK` 안전성 검사를 거친 외부 링크, 서버가 메뉴 ID로 채번한 `/preparing/{mnuId}` 준비중 경로로 구분합니다. 준비중 화면은 `pages/preparing/[[slug]].vue` 하나가 처리하므로 관리 화면에 사람이 임의의 경로를 입력하지 않습니다.

추가 인증(MFA) 연결

수동 로그인과 사용자 전자결재 명령은 백엔드가 MFA를 강제합니다. 화면은 `useMfa()` 상태 기계와 `MfaDialog` 하나를 공유합니다.

구성	역할
types/mfa.ts	인증수단·목적·표준 오류 코드. 응답 타입은 생성물에서 파생

구성	역할
<code>composables/useMfa.ts</code>	challenge 시작·검증·취소 상태 기계. <code>openMfa()</code> 가 성공/취소로 결정된다
<code>composables/useMfa.ts</code> 의 <code>useSharedMfa()</code>	앱 전역 공유 인스턴스. <code>AppShell</code> 이 대화상자를 한 번만 마운트
<code>composables/useMfaProtectedAction.ts</code>	결재 명령 실행 래퍼(<code>runWithApprovalMfa</code>)
<code>components/mfa/MfaDialog.vue</code>	인증수단 선택·QR·OTP·에이전트 안내·카운트다운

- **로그인**은 `startLogin()` → `openMfa({ purpose: 'LOGIN' })` → `completeLogin()` 순서입니다. 자격증명만으로는 토큰이 발급되지 않고, 대기 거래나 증표가 만료되면 비밀번호부터 다시 입력받습니다. 로그인 화면은 `AppShell` 밖이라 자체 `useMfa()` 인스턴스를 씁니다.
- **전자결재 명령**은 반드시 `runWithApprovalMfa(() => ...)` 로 감쌉니다. 인증 상태를 재사용하지 않으므로 신청·승인·반려·회수마다 새 MFA가 필요합니다. 어떤 명령이 래퍼를 거치는지는 `tests/unit/composables/approval-mfa-coverage.test.ts` 가 고정하므로 명령을 추가하면 그 목록도 함께 갱신합니다.
- 조회·임시저장·상태 전이(PATCH)는 백엔드에서도 MFA 대상이 아니므로 감싸지 않습니다.

운영 전제:

- **BioAgent** — 지정맥을 쓰려면 사용자 PC에서 BioAgent가 실행 중이어야 합니다. 화면이 `ws://127.0.0.1:8089/bio` 로 연결하고, 실패하면 실행 안내와 `BioAgent://` 실행 버튼을 표시합니다.
- **지정맥 프로토콜** — `mfa.md` 규격을 따릅니다. 소켓을 열면 `{type:'identify', key, sysGubun:'ITP', id, fingertype}` 을 보냅니다. `key` 는 서버가 **challenge** 응답으로 내려준 6자리 랜덤키이고, `id` 는 사번, `fingertype` 은 등록한 손가락 종류입니다. 클라이언트가 키를 만들면 서버가 해시를 대조할 수 없으므로 반드시 서버 값을 그대로 씁니다.
- **해시 제출** — `identify` 응답은 본인확인 결과의 해시입니다. 이 값을 보관했다가 `errCode` 응답이 `FE00` 이면 해시를 `verificationValue` 로 서버에 제출합니다. 서버가 같은 해시를 만들어 비교하므로 결과 코드만으로는 통과하지 않습니다.
- **sysGubun** — API 사양과 `Fv_identify_proc` 정의가 모두 `ITP` 를 명시해 이를 따릅니다. 연계 예시 주석의 `ITS`, 데모 페이지의 `WEB` 과는 표기가 다릅니다.
- **응답 처리** — 응답이 `FE` 로 시작하면 결과 코드로 보고 바로 판정하고, 그렇지 않으면 해시로 보관한 뒤 `{type:'errCode', key, sysGubun}` 를 한 번 더 보냅니다. `FE00` 이면서 해시를 받은 경우에만 성공이며 판정 후 소켓을 닫습니다.
- **손가락 식별자** — 현재 화면은 손가락 종류 선택 UI를 제공하지 않으며, BioAgent 요청에는 오른손 검지 기본값('10')을 전달합니다. 등록 정보와 다른 값으로 요청하면 인증이 실패할 수 있습니다.
- **결과 코드 안내** — `FE00` ~ `FE24` 의 안내 문구는 `types/mfa.ts` 의 `BIO_AGENT_MESSAGE` 에 둡니다. 에이전트가 보낸 문자열을 그대로 렌더링하지 않고 코드로만 조회합니다.
- **local-ext** — 모의 공급자를 쓰지만 화면 흐름은 동일합니다. 대화상자를 여는 것만으로는 통과하지 않고 [확인]을 눌러 검증을 요청해야 성공합니다.
- 마지막 성공 인증수단은 `mfa-last-method` 쿠키에 코드만 저장하며, 값이 없거나 허용 목록 밖이면 지정맥으로 폴백합니다.

프로파일별 동작과 지정맥 클라이언트 증명 한계는 [백엔드 README의 내부망 MFA](#)를 기준으로 합니다.

주요 업무 흐름

영역	시작 화면	구현 흐름
홈 대시 보드	<code>pages/info/index.vue</code>	올해·내년 KPI 카드(건수형·금액형), 예산 일정, 공지·일정 피드, 즐겨찾기와 배너 캐러셀을 한 화면에 조합합니다. 연도 별 사업·전산업무비 집계는 <code>useInfoDashboardYear</code> 가 조회하고 집계 규칙은 <code>utils/infoDashboardSummary.ts</code> 가 소유하며, 부서·전체 조회 범위 전환은 <code>utils/infoDashboardScope.ts</code> 가 담당합니다.
정보 화사 업· 계획	<code>pages/info/projects/</code> , <code>pages/info/plan/</code>	정보화사업과 경상사업을 같은 원장에서 조회·작성하고, 결재완료 최종본은 <code>SNO</code> 가 증가한 초안으로 수정 후 재상신합니다. 상세·수정·결재 경로는 개정 순번을 보존합니다. 입력 폼은 관리자가 등록한 입력 길라잡이를 옆 패널(<code>FormGuidePanel</code>)로 함께 보여 주며, 사업 첨부과 편성요청서 반입 원본은 <code>useProjectAttachments</code> 가 한 목록으로 묶습니다.
사업 계획	<code>pages/project/bizplan/</code>	목록에서 사업을 선택하면 상세 진입 API가 사업계획을 조회하거나 최초 생성합니다. 상세 화면은 보고서, 일정, 품목, 계약을 한 번에 편집·저장하고 작성중에서 작성완료로 상태를 전이합니다.
전산 업무 비	<code>pages/info/cost/</code>	목록 인라인 편집, 셀 붙여넣기, Excel 입출력, 단건·복수 품, 단말기 관리, 전년도 계속계약 불러오기와 <code>BG_SNO</code> 기반 이력·수정 후 재상신을 <code>useCostListPage</code> 와 비용 컴포넌트가 조합합니다.
예산 신청 ·결 재	<code>pages/budget/index.vue</code> , <code>list.vue</code> , <code>approval.vue</code> , <code>report.vue</code>	정보화사업·전산업무비·경상사업을 통합 조회하고 선택 항목을 PDF 신청서로 만든 뒤 결재선을 지정해 상신합니다. 재상신 초안은 관리번호와 <code>sno</code> 를 함께 전달해 최종본과 구분합니다.
예산 편성 ·조 회	<code>pages/budget/work.vue</code> , <code>status.vue</code> , <code>summary.vue</code> , <code>comparison.vue</code>	결재 완료 대상을 기준으로 편성률을 적용하고, 사업·비목별 결과와 전년 비교를 조회합니다. 편성·현황 화면은 관리자 UX 가드를 적용하고 표 결과를 Excel로 내보냅니다.

영역	시작 화면	구현 흐름
요구 사항 문서 사전협의	<code>pages/info/documents/</code>	Tiptap HTML 문서의 CRUD와 버전 이력, 첨부파일, 인라인·일반 검토 의견, 검토자 상태를 문서 composable과 review store로 관리합니다.
협의회	<code>pages/info/council-1-request/</code>	목록·상세·개최준비·결과 화면이 <code>useCouncil</code> 과 협의회 컴포넌트를 공유하고, 개최준비에는 <code>council-manager</code> UX 가드를 적용합니다.
사업 후속 문서	<code>pages/project/estimate/</code> , <code>deliberation/</code> , <code>contract/</code> , <code>payment/</code>	견적, 심의, 계약, 지급 문서를 각각의 타입과 composable로 조회·처리하며 공통 인증 API 계층을 사용합니다.
전자 결재	<code>pages/approval/</code>	결재 대시보드·목록·상세에서 결재선과 신청서를 조회하고 승인·반려·회수 명령을 수행합니다.
공통 게시판	<code>pages/board/</code>	게시판·게시글 CRUD와 댓글·첨부파일·멘션 기능을 공통 composable로 연결합니다. 게시글 본문은 <code>TiptapEditor</code> 로 입력받고, 상세는 저장된 멘션 span을 <code>utils/boardContent.ts</code> 가 직원 정보 버튼으로 바꾼 뒤 <code>DOMPurify</code> 로 정화해 표시합니다.
스피드다이얼	공통 <code>AppShell</code> 의 <code>SpeedDial.vue</code>	관리자 화면을 제외한 모든 화면 우측에 떠 있는 전역 버튼으로 FAQ 조회, Q&A 등록, 담당자 정보 열람을 제공합니다. FAQ·Q&A는 전용 화면 없이 공통 게시판의 FAQ·Q&A 유형 게시판을 재사용하고, Q&A 등록 시 현재 화면 이름·경로를 메뉴 트리에서 스냅샷해 함께 보냅니다. 감춤 여부만 <code>localStorage</code> UI 선호값으로 유지합니다.
사용자 가이드	헤더 <code>UserGuideButton.vue</code> , <code>pages/admin/user-guides.vue</code>	헤더 [통합검색] 좌측 버튼이 현재 가이드를 내려받고(없으면 버튼 자체를 숨김), 관리 화면에서 업로드·이력 되돌리기를 수행합니다. 현재 가이드는 항상 0건 또는 1건입니다.

영역	시작 화면	구현 흐름
공통 안내 팝업	공통 AppShell 의 CommonPo pupDialog .vue , pages/ad min/commo n- popup.vu e	인증 화면에 진입하면 어느 경로로 들어오든 활성 팝업을 다이얼로그로 표시합니다. [다시 보지 않기] 는 콘텐츠 버전을 1년 쿠키에 담아 같은 버전만 숨기므로 관리자가 내용을 바꾸면 다시 나타납니다. 조회 실패는 화면 진입을 막지 않습니다.
사전 진단 ·가 이드	pages/di agnosis/ , pages/gu ide/	진단은 클라이언트 설문 결과를 계산하고, 가이드는 서버 문서·첨부·Tiptap 변수 카탈로그를 조회합니 다.
시스 템 관리 ·실 시간 로그	pages/ad min/	사용자·권한·메뉴·코드·로그 관리 화면은 admin 레이아웃과 가드를 공유합니다. 실시간 로그 화면은 useRealtimeLogs 의 폴링 결과와 useRealtimeLogPreferences 의 UI 선호값을 결합합니다. 다국 어 관리 화면(pages/admin/translations.vue)은 메뉴명·공통코드 표시명 번역(TPRMPP_CLANGM) 을 등록·수정하며, 빈 문자열 저장은 논리 삭제로 처리됩니다. WAS 로그 화면(pages/admin/was- logs.vue)은 useWasLogFeed 가 3초 주기로 인스턴스별 로그를 폴링해 표에 이어 붙이고, 도구모음 에서 로그레벨을 TTL과 함께 임시 변경합니다. 배너 관리 화면(pages/admin/banners.vue)은 홈 캐 러셀 이미지를 업로드하고 활성 상태를 전환합니다. 입력 길라잡이 관리 화면(pages/admin/form- guides/index.vue)은 서버 고정 카탈로그가 돌려준 사업 유형별 입력 필드 목록을 표로 보여 주고, 필드별 안내 본문을 Tiptap 편집기로 등록·삭제합니다.
공통 데이 터 이관	pages/ad min/migra tion/comm on- data.vue	메뉴·메뉴권한·경로·공통코드·다국어 5개 테이블을 개발 환경에서 xlsx로 내보내고 운영 환경에서 올려 dry-run으로 추가·갱신·부활 건수를 확인한 뒤 확정 반영합니다. 직렬화·파싱은 utils/commonDataMigration.ts 가 담당합니다.
편성 요청 서 반입	pages/ad min/migra tion/	수기 Excel 제출본을 부서 폴더째 드롭존(FileDropzonePicker)으로 올려 분석하고, 미리보기에서 담당자·비목·통화 같은 미해석 항목을 보정한 뒤 반입하며, 결과 표와 진단 줄로 건별 판정을 확인합니 다. 분석·반입은 배치 단위로 진행률 다이얼로그(RequestFormCommitProgressDialog)에 표시하고, 부서별 원본 파일 트리와 진단은 RequestFormAnalysisFilesDialog 에서 엽니다. 반입으로 만들어 진 신청서는 결재현황 뷰어에서 원본 파일 목록(RequestFormSourceFiles)을 보여 줍니다.

공통 UI와 문서 생성 모듈

- 데이터 표는 components/common/StyledDataTable.vue 를 중심으로 구성하고, 검색·인라인 편집·직원 선택·버전 이력 같은 반복 UI는 components/common/ 에서 재사용합니다. 파일 선택·드래그앤드롭은 FileDropzonePicker.vue 가 파일과 폴더 반입 화면에 같은 UI를 제공하고 업로드·업무 검증은 호출 화

면이 맞습니다. 첨부 목록의 확장자 아이콘은 `FileFormatIcon.vue`, 게시물 첨부 열기·내려받기 동작은 `BoardAttachmentActions.vue` 를 사용합니다. 비용·예산·계획처럼 화면 조합이 큰 영역은 도메인 컴포넌트가 공통 UI 위에 업무 규칙을 더합니다.

- 첨부파일은 상세 화면의 목록 표시를 `AttachmentListSection.vue`, 등록·수정 품의 파일 선택을 `AttachmentUploadField.vue` 가 담당합니다. 선택 컴포넌트는 스스로 업로드하지 않습니다 — 신규 등록은 저장 전에 부모 식별자가 없으므로 선택 결과만 `v-model` 로 올려 주고, 실제 업로드·삭제는 부모 식별자가 확정된 뒤 화면의 저장 흐름이 수행합니다. 인증 쿠키가 필요한 내려받기는 `useAttachmentDownload()` 가 처리하고, 출처가 다른 첨부을 한 화면에 함께 실을 때는 그룹 목록으로 넘겨 그룹별 재조회 실패를 따로 안내합니다. 선택 기준은 [공통 컴포넌트 가이드](#)를 따릅니다.
- 사용자 입력 안내는 화면마다 문구를 심지 않고 관리자가 등록한 입력 길라잡이를 서버에서 받아 표시합니다. 폼 컨테이너가 `capture` 단계에서 입력·클릭을 위임받아 `data-guide-id` 를 가진 가장 가까운 필드를 활성 항목으로 잡고, `FormGuidePanel.vue` 가 본문과 등록 순번을 보여 줍니다. 조회 실패는 패널을 숨기고 `FormGuideLoadError.vue` 로 재시도를 안내합니다.
- 긴 입력 필드는 `useDatabaseByteLimit()` 이 UTF-8 실제 바이트 상한을 넘는 값을 모델 단계에서 되돌리고 공통 `Toast`로 안내하며, 남은 용량은 `TextLengthIndicator.vue` 가 표시합니다. 대상 컬럼은 `AL32UTF8` 의 `VARCHAR2(n BYTE)` 이므로 화면과 DB의 저장 한도는 같은 `n` 바이트입니다. 새 필드를 추가할 때는 물리 컬럼의 `DATA_LENGTH` 를 확인하고 별도 글자 수 업무 제한이 있으면 `maxCharacters` 를 함께 지정합니다.
- `TiptapEditor.vue` 는 확장 등록과 편집기 본체를 담당하고, 툴바·표 도구·이미지 삽입·변수 해석은 별도 컴포넌트와 `composable`로 분리합니다. 커스텀 확장 자체는 `components/editor/extensions/` 아래 `content` (첨부·수식·변수·글자크기·제목), `media` (이미지·Excalidraw), `table` (노드·행 리사이즈·컬럼 폭)로 나누어 두고, 소비처는 하위 파일을 직접 가리키지 않고 배럴 `tiptap-extensions.ts` (전체) 또는 `tiptap-content-extensions.ts` (본문 계열)로만 가져옵니다. 변수는 서버 카탈로그를 조회해 HTML의 토큰을 표시값으로 해석하며, 비동기 응답의 최신성은 편집기 계층에서 관리합니다.
- HWPX 내보내기는 `useHwpExport` 가 첨부된 Excalidraw 장면을 SVG로 전처리한 뒤 `utils/hwp.ts` 에 변환을 위임합니다. 이미지 처리는 `utils/hwp-images.ts`, 패키지 XML은 `utils/hwp-package-xml.ts` 가 담당하고, 최종 결과는 JSZip 기반 HWPX로 내려받습니다.
- 전자결재 신청서는 공통 작업영역(`ApprovalFormWorkspace`) + 양식 등록부(`approvalFormRegistry`) + 양식별 PDF 렌더러 구조를 사용합니다. 양식은 화면 제목이 아니라 변경되지 않는 `form.id` 로 식별하고, 양식별 코드(정의·스키마·PDF 렌더러)는 `app/features/approval/forms/{양식명}` 에 모읍니다. 전산예산 신청서 PDF는 `features/approval/forms/itBudget/useItBudgetApprovalFormPdf` 가 프로젝트·전산업무비와 결재선을 pdfmake 문서로 구성합니다. 새 신청서 추가 절차는 [전자결재 신청서 확장 가이드](#)를 따릅니다. 일반 표 내보내기는 `utils/excel.ts` 의 ExcelJS 스타일·다운로드 헬퍼를 우선 사용합니다.

핵심 설계 결정

- 사내 포털은 SEO가 필요하지 않고 정적 배포에서 인증 상태가 빌드 페이로드로 덮이는 문제를 피해야 하므로 완전한 CSR을 사용합니다.
- JWT는 JavaScript에서 읽지 않고 httpOnly 쿠키로만 전달하며, 프론트 사용자 쿠키와 라우트 가드는 빠른 화면 복원과 UX 보호에만 사용합니다.
- 조회와 명령을 서로 다른 API 래퍼로 나눠 반응형 재조회와 401 원 요청 재시도의 책임을 명확히 합니다.

- 메뉴와 데이터 권한의 최종 필터링은 서버가 담당하고, 프론트는 서버 메뉴 트리를 탐색 UI의 단일 소스로 사용합니다.
- 페이지는 라우팅과 화면 조합에 집중하고, 재사용 상태·업무 흐름은 composable, 장기 공유 상태는 Pinia store, 순수 변환은 utility에 둡니다. 화면 하나에만 쓰이는 상태도 페이지에 남기지 않고 `use{화면}Page` composable로 내려 단위 테스트가 가능하게 하며, 그 경계는 `tests/unit/pages/*PageBoundary.test.ts` 가 고정합니다.
- 서버 응답 타입은 수기 미러가 아니라 백엔드 OpenAPI 스펙에서 생성합니다. 백엔드 DTO의 계약이 곧 프론트 타입이 되어 두 저장소가 따로 표류하지 않습니다.
- PrimeVue 위에 디자인 토큰과 공통 컴포넌트 계층을 두어 표 밀도, 색상, 오류 피드백과 내보내기 동작을 화면마다 다시 구현하지 않습니다.
- 입력 안내 문구는 코드가 아니라 데이터로 관리합니다. 안내 대상 필드는 서버 고정 카탈로그가 정하고 본문은 관리자가 등록하므로, 문구가 바뀔 때 화면을 고쳐 배포하지 않습니다.
- 세션 만료 후 어디로 보낼지는 호출한 화면이 아니라 한 곳에서 결정합니다. 보호 경로의 인증 진입은 SSO가 기본이라는 계약을 만료 처리에도 그대로 적용하되, 왕복이 유효한 세션을 만들지 못하는 상황을 간격으로 감지해 수동 로그인으로 빠져나옵니다.

일반적인 개발 흐름

1. 기존 공통 컴포넌트·composable·utility를 검색합니다.
2. 페이지나 기능을 `app/` 아래에 구현합니다.
3. API URL은 `runtimeConfig.public.apiBase` 를 사용합니다.
4. 로직 단위 테스트와 필요한 E2E 시나리오를 추가합니다.
5. `npm run format:check`, `npm run check`, 관련 테스트를 실행합니다.

Vitest는 `happy-dom` 과 `tests/setup.ts` 의 Nuxt 전역 스텝을 사용하며 `utils`, `stores`, `composables`, `middleware` 를 단위 커버리지 대상으로 삼습니다. Playwright는 `tests/e2e` 를 실행하면서 `http://localhost:3002` 에 Nuxt 개발 서버를 자동으로 기동하고, 각 시나리오가 필요한 인증 쿠키와 API 응답을 직접 준비합니다. 변경 범위별 명령과 Mock 원칙은 [프론트 테스트 가이드](#)를 따릅니다.

필수 코딩 규칙은 [CLAUDE.md](#), 상세 예제는 [가이드 인덱스](#)를 확인합니다.

운영 빌드

백엔드와 공유하는 `SPRING_PROFILES_ACTIVE` 환경변수 하나로 배포 프로파일이 결정됩니다. 서버에 백엔드용으로 이미 설정된 값을 그대로 사용하므로 프론트용 환경변수를 따로 관리할 필요가 없습니다.

```
npm run generate # SPRING_PROFILES_ACTIVE 값에 따라 generate:local / :dev / :prod 자동 실행
```

SPRING_PROFILES_ACTIVE	실행 스크립트	로드 프로파일
local-ext, local-int 등 local*	npm run generate:local	.env.local
dev	npm run generate:dev	.env.development
prod	npm run generate:prod	.env.production

미설정이거나 매핑할 수 없는 값이면 안내 메시지와 함께 실패합니다(fail-fast). 특정 대상을 강제하려면 프로파일별 스크립트를 직접 실행합니다.

실제 빌드 전에 디스패처의 선택 결과만 확인하려면 `--dry-run` 을 전달합니다. 이 모드는 대상 스크립트 이름을 출력하고 정적 생성은 실행하지 않습니다.

```
$env:SPRING_PROFILES_ACTIVE = 'local-int,dev'
npm run generate -- --dry-run
# [generate] SPRING_PROFILES_ACTIVE="local-int,dev" → npm run generate:local
```

각 프로파일 스크립트는 `--dotenv` 로 해당 파일을 로드하고, `cross-env` 로 `NUXT_PUBLIC_API_BASE=` (빈 값)을 한 번 더 강제 주입합니다. OS 환경변수(`setx` 등록 값 포함)는 `dotenv` 파일보다 항상 우선하기 때문에, 스크립트에서 강제하지 않으면 내부망 IP 테스트용으로 등록한 절대 URL이 배포 번들에 박히는 사고가 납니다. 결과적으로 정적 빌드는 항상 **same-origin 상대경로**(`/api/...`, `/sso/...`)로 호출하며, 웹 서버(로컬 nginx, 개발·운영 WebTobe)가 백엔드로 프록시하는 구조를 전제합니다.

생성 결과는 `.output/public/` 에 위치하며, 로컬의 `dist/` 는 이 디렉토리를 가리키는 junction입니다. 정적 배포에서는 Nitro `server/` 미들웨어가 실행되지 않으므로 인증 진입은 `auth.global.ts` 가 담당합니다.

웹 서버 요구사항 (nginx 예시)

웹 서버는 다음 세 가지를 반드시 처리해야 합니다.

1. 모든 SPA 경로를 `index.html` 로 연결 (직접 진입·새로고침 대응)
2. `/api/` 를 백엔드로 프록시 (XHR API 호출)
3. `/sso/` 를 백엔드로 프록시 (SSO 전체 페이지 리다이렉트) — 누락하면 `/sso/business` 가 SPA fallback 으로 되돌아와 **무한 리다이렉트 루프**가 됩니다

```
server {
    listen 80;
    root C:/it/it_frontend/.output/public;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://localhost:28080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /sso/ {
        proxy_pass http://localhost:28080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

백엔드 환경변수 (프록시 배포 시)

same-origin 프록시 구조에서는 API 호출에 CORS가 발생하지 않지만, **SSO 완료 후 복귀 origin 검증은 백엔드** `cors.allowed-origins` 목록을 그대로 사용합니다. 프론트 접속 origin(예: 로컬 nginx `http://localhost`, 운영 `https://itp.kdb.co.kr:20443`)이 목록에 없으면 SSO가 성공해도 `app.frontend-url` 기본값으로 되돌아갑니다.

- 로컬 nginx 검증: `local-ext / local-int` 프로파일 기본값에 `http://localhost` 가 포함되어 있으므로 프로파일을 지정해 기동하면 됩니다. `APP_FRONTEND_URL · CORS_ALLOWED_ORIGINS` 환경변수가 등록되어 있으면 기본 목록을 덮어쓰므로 `http://localhost` 를 직접 포함시켜야 합니다.
- 운영: `APP_FRONTEND_URL` 에 실제 프론트 접속 URL을 지정합니다. 상세는 [백엔드 README](#)의 환경 설정을 따릅니다.

절대 URL이 필요한 특수 배포

프록시 없이 백엔드를 다른 origin으로 직접 호출해야 하는 배포라면 스크립트를 거치지 않고 값을 직접 지정합니다. 이 경우 백엔드 `cors.allowed-origins` 에 프론트 origin 허용이 필요합니다.

```
npx cross-env NUXT_PUBLIC_API_BASE=https://backend.example.internal nuxt generate
```

문제 해결

Nuxt 타입이 생성되지 않을 때

```
npm run postinstall
npm run typecheck
```

`npm run generate`가 프로파일 오류로 종료될 때

`SPRING_PROFILES_ACTIVE`가 비어 있거나 첫 번째 프로파일이 `local*`, `dev`, `prod` 중 하나가 아니면 디스패처가 종료 코드 1로 실패합니다. 현재 세션 값을 확인한 뒤 `--dry-run`으로 매핑을 검증합니다.

```
$env:SPRING_PROFILES_ACTIVE
npm run generate -- --dry-run
```

API가 JSON 대신 HTML을 반환할 때

`npm run dev` 중에는 브라우저 Network 탭에서 요청이 `localhost:3000/api/...`로 향하는지 확인합니다. 개발 기본값이라면 `localhost:28080/api/...`로 호출되어야 하므로, API 경로를 직접 넘기지 말고 `runtimeConfig.public.apiBase`를 접두사로 사용했는지와 `NUXT_PUBLIC_API_BASE` 값을 점검합니다. 정적 프로덕트 배포에서 같은 origin의 `/api/...`로 향하는 것은 정상입니다.

CORS 403 — 화면은 뜨는데 모든 API가 실패하고 SSO도 시도하지 않을 때

콘솔에 `No 'Access-Control-Allow-Origin' header is present`와 `403`이 함께 보이면 인증 실패가 아니라 백엔드 CORS 차단입니다. 접속 Origin이 백엔드 `cors.allowed-origins`에 없으면 인증 판단 전에 걸려집니다.

- 백엔드에 현재 Origin을 허용시킵니다(위 [내부망 IP로 접속할 때](#) 참조).
- 이때 SSO로 넘어가지 않는 것은 정상 동작입니다. `it-portal-user` 쿠키가 남아 있으면 `auth.global.ts`가 인증 상태로 판단하고, CORS 차단 응답은 상태 코드를 읽을 수 없어 401 재인증 경로가 동작하지 않습니다. 설정을 고친 뒤 해당 사이트 쿠키를 지우고 다시 진입합니다.

정적 배포에서 모든 API가 403 (Invalid CORS request)

nginx 등으로 정적 배포했는데 요청이 `localhost:28080` 같은 백엔드 주소로 직접 나가면서 응답 본문이 `Invalid CORS request (20바이트)`라면, 빌드에 절대 URL이 박힌 구버전 산출물입니다. `npm run generate`로 재빌드하면 same-origin 상대경로로 호출합니다. 재빌드 후에도 절대 URL이 보이면 `.output/public/index.html`의 `window.__NUXT__.config`에서 `apiBase` 값이 `""`인지 확인합니다.

SSO 무한 리다이렉트 (`/sso/business?next=...` 가 계속 중첩)

정적 배포에서 로그인 시도 시 URL의 `next` 파라미터가 계속 중첩되며 반복된다면, 웹 서버가 `/sso/` 를 백엔드로 프록시하지 않아 `/sso/business` 가 SPA fallback(`index.html`)으로 되돌아오는 것입니다. [웹 서버 요구사항](#)의 `/sso/` 프록시 블록을 추가하고, 중첩된 URL이 아닌 사이트 루트로 새로 진입해 확인합니다.

401 응답

- 백엔드 `http://localhost:28080` 실행 여부를 확인합니다.
- 브라우저 쿠키와 `NEXT_PUBLIC_API_BASE` 를 확인합니다.
- 로그인 후에도 반복되면 백엔드 인증 로그를 확인합니다.
- 401 뒤 토큰 갱신까지 실패하면 세션 만료가 확정된 것으로 보고 로컬 인증 상태를 정리한 뒤 SSO로 재진입합니다. 화면이 SSO와 로그인 사이를 계속 오간다면 SSO 왕복이 유효한 세션을 만들지 못하는 상태입니다 — 이때는 재진입 간격 판정이 수동 로그인(`/login?error=sso`)으로 폴백시키므로, 브라우저가 아니라 백엔드 세션 발급 쪽을 확인합니다.

E2E 불안정

고정 지연을 추가하기 전에 API mock, URL 전환, 화면 준비 조건을 명시적으로 기다리는지 확인합니다.

문서

- [필수 프론트 규칙](#)
- [상세 개발 가이드](#)
- [공통 컴포넌트 인덱스](#)
- `docs/preview/`: 디자인 시안·컴포넌트 정적 미리보기
- [루트 개발 안내](#)
- [기술부채와 후속 과제](#)

IT Portal 프론트엔드 작업 계약

SOURCE: it_frontend/CLAUDE.md

IT Portal 프론트엔드 작업 계약

이 파일은 프론트엔드 변경 시 반드시 지킬 규칙만 정의합니다. 설치·실행·배포·문제 해결은 [README](#), 배경과 예제는 [가이드 인덱스](#)를 따릅니다. 상위 공통 규칙은 [루트 CLAUDE](#)를 함께 적용합니다.

1. 기술과 구조

- Nuxt 4, Vue 3 Composition API, TypeScript, Pinia, PrimeVue를 사용하며 정적 CSR 배포를 전제로 합니다.
- 앱 코드는 `app/`, 번역은 `i18n/messages/`, 단위 테스트는 `tests/unit/`, E2E는 `tests/e2e/`에 둡니다.
- 페이지는 라우팅·템플릿·화면 조합만 담당합니다. 화면 상태와 업무 흐름은 `use{화면}Page`, 도메인 재사용 로직은 `composables/{도메인}/`, 순수 변환은 `utils/`에 둡니다.
- Pinia는 장기 공유 상태만 담당하며 신규 store가 composable을 호출하는 구조를 만들지 않습니다.
- 신규 운영 `.ts`·`.vue` 파일은 `scripts/max-lines-baselines.mjs`의 800줄 상한을 넘기지 말고 책임을 분리합니다.

2. API와 오류 처리

- 조회·반응형 재조회는 `useApiFetch`, 사용자 명령과 일반 요청은 `$apiFetch`를 사용합니다. 선택 기준과 상태 계약은 [API 클라이언트 가이드](#)가 SoT입니다.
- 인증 Store와 Route Middleware처럼 composable/plugin 컨텍스트 제약이 있는 코드는 예외적으로 native `$fetch`를 사용할 수 있지만, 반드시 `credentials: 'include'`를 지정합니다.
- 반응형 `useApiFetch` / `useFetch`는 setup 동기 구간에서 생성하고, `await` 이후의 일회성 조회나 생명주기 콜백에서는 `$apiFetch`를 사용합니다.
- API 경로는 `runtimeConfig.public.apiBase`를 사용하고 주소를 화면에 하드코딩하지 않습니다.
- 서버 DTO 계약이 바뀌면 백엔드를 기동해 `npm run codegen`으로 `app/types/api.d.ts`를 재생성하고 `npm run codegen:check`을 통과시킵니다.
- Nuxt `useFetch` 어댑터와 관리자 행 정제 경계는 생성 타입·제네릭으로 표현하고 `any`와 `as unknown as T`를 사용하지 않습니다. 화면 전용 타입이 필요하다면 generated API 타입과 명시적으로 분리합니다.
- DB 문자열 컬럼의 입력 한도는 화면마다 다시 세지 말고 `utils/textLength.ts`와 `useDatabaseByteLimit()`으로 판정하고, 남은 용량 표시는 `TextLengthIndicator`를 재사용합니다. 대상 컬럼은 AL32UTF8의 VARCHAR2(n BYTE)이므로 DB가 강제하는 제약은 UTF-8 실제 바이트 수 `n`입니다.

새 필드에 한도를 걸 때는 물리 컬럼의 `DATA_LENGTH`를 확인하고, 별도 글자 수 업무 제한이 있으면 `maxCharacters`로 함께 검증합니다.

- 변경 요청 오류는 화면별로 서버 문자열을 직접 렌더링하지 말고 공통 오류 매핑을 사용합니다.
- 목록 전체 건수만 필요한 화면은 목록을 통째로 받아 세지 않고 `page=0&size=1`로 한 건만 요청한 뒤 응답 헤더 `X-Total-Count`를 읽습니다. 헤더가 없으면 0으로 감추지 말고 예외로 실패시킵니다.
- `suppressErrorToast`는 자체 오류 UI(인라인 경고·배너·전용 Toast)가 있는 화면에서만 켵니다. 켜 두고 오류 UI가 없으면 조회 실패가 빈 화면으로만 보입니다.
- 재조회 실패는 이전 성공 데이터를 성공처럼 유지하지 말고 `useRefreshGuard`로 명시합니다. 같은 조회에는 가드 하나를 공유하고 서로 다른 조회는 상태와 안내를 분리합니다.
- 같은 화면의 여러 인스턴스가 첫 진입 데이터를 읽는 경우 요청 키별 in-flight 공유를 사용하되, 명시적 새로고침·재시도는 `$apiFetch`로 독립 실행합니다. 정상 빈 결과와 초기 조회 실패는 별도 상태로 렌더링합니다.
- Nuxt `refresh()`는 실패해도 Reject하지 않을 수 있으므로 `try/catch`만으로 판정하지 말고 `useRefreshGuard` 또는 `refreshOrThrow` 계약을 사용합니다.
- 버튼 라벨은 최초 실행은 다시 시도, 데이터 새로고침은 새로고침, 실패한 자동 재로드는 다시 시도 로 구분합니다.

3. 인증·인가·보안

- JWT를 JavaScript 저장소에 저장하거나 직접 읽지 않습니다. Access/Refresh Token은 백엔드 `httpOnly` 쿠키로만 전달됩니다.
- `it-portal-user` 쿠키와 라우트 가드는 화면 복원과 UX 보호용이며 서버 권한 판단을 대신하지 않습니다.
- 인증 진입과 세션 복원은 `auth.global.ts`가 담당합니다. 정적 생성에서는 Nitro `server/` 미들웨어에 의존하지 않습니다.
- 401 이후 토큰 갱신까지 실패해 세션 만료가 확정되면 화면별로 이동을 결정하지 말고 `utils/sessionExpiryRedirect.ts`의 재진입 결정을 사용합니다. 보호 경로의 기본 진입은 SSO이며, 최소 간격 안에 만료가 다시 확정되면 무한 왕복을 끊기 위해 수동 로그인으로 폴백합니다.
- 안전한 `next`는 단일 `/`로 시작하는 내부 경로만 허용하고 `//`와 `/login`을 거부합니다.
- 역할 문자열은 `types/auth.ts`의 `ROLE` 상수를 사용합니다.
- 인증·권한·개인정보를 `localStorage`에 추가하지 않습니다. UI 선호값만 허용합니다.
- 전자결재 상태 변경 명령은 `useMfaProtectedAction().runWithApprovalMfa()`로 감싸고 `coverage` 계약 테스트를 함께 갱신합니다. 조회·임시저장·일반 PATCH는 대상이 아닙니다.
- MFA 대화상자는 `AppShell`의 공유 인스턴스 하나만 사용하고, 서버·에이전트 오류 문자열을 직접 노출하지 않습니다.
- 사용자 또는 서버 HTML을 `v-html`로 표시할 때는 `isomorphic-dompurify`로 정화합니다. 서버도 저장 시 정화하지만 렌더 직전 정화를 생략하지 않습니다.
- HTML 문자열 가공(개행 복원, 맨션·변수 치환 등)은 `DOMPurify` 정화 이전에 끝내고, 정화 결과 문자열을 다시 조립하지 않습니다. 정화가 파이프라인의 마지막 단계여야 합니다.
- 식별자용 난수는 `utils/secureId.ts`의 Web Crypto 경로만 사용합니다. `Math.random()` 폴백을 되살리지 말고, 미지원 환경에서는 명시적으로 실패시킵니다.

4. 상태·메뉴·라우팅

- 메뉴 트리는 서버가 역할별로 필터링한 `/api/menus` 응답이 SoT입니다. 화면에 메뉴 구조·이름·아이콘을 중복 하드코딩하지 않습니다.
- 게시판 메뉴 경로는 `types/menu.ts`, 외부 링크·준비중 경로 판별은 `utils/menuPath.ts`, 아이콘과 표시 메타데이터는 `utils/menuPresentation.ts`의 헬퍼와 상수를 사용합니다.
- 준비중 메뉴의 화면 경로는 서버가 `/preparing/{menuId}`로 채번하므로 관리 화면에서 사람이 입력받지 않습니다. 준비중 경로는 `pages/preparing/[[slug]].vue` 하나가 처리하며, 준비중을 해제할 때는 직전 자동 경로를 그대로 두지 말고 비워 새 경로를 고르게 합니다.
- 준비중 자동 경로의 메뉴 ID는 소문자로 정규화하며, 대소문자 변형을 별도 경로로 취급하지 않습니다.
- KeepAlive 재방문 시 최신성이 필요한 화면은 `onActivated`에서 재조회하되 첫 활성화의 중복 요청을 막습니다.
- 정보화사업·경상사업과 전산업무비의 재상신 초안은 관리번호만으로 식별하지 않습니다. 상세·수정·삭제·결재 신청 URL과 API에 `sno`를 끝까지 보존합니다. 순번이 없는 단건·후속업무 조회는 `LST_YN='Y'` 최종본을 의미하며, 미상신 작성 목록에서 받은 재상신 초안은 응답의 `sno`를 다음 화면까지 전달합니다.
- 여러 composable이 같은 접근 판정을 사용하면 공통 access context를 공유하고, 화면별 업무 규칙만 각 composable에 남깁니다.
- 전역 이벤트 리스너·polling timer·Observer는 등록된 모듈이 `onUnmounted` 또는 `onDeactivated`에서 해제·중지하며, 재활성화 시 중복 등록하지 않습니다.
- AppShell을 공유하는 화면에 별도 layout을 추가하지 않습니다. layout 전환은 보존된 페이지 캐시와 메뉴 상태를 함께 폐기합니다.
- 날짜·금액·API 오류는 `utils/common.ts`, Toast 시간은 `~/utils/toast`, 공통 선택·표 동작은 기존 composable을 재사용합니다.

5. UI와 스타일

- 신규 색상·간격·타이포그래피는 [디자인 토큰](#)을 따릅니다.
- 일반 표는 `StyledDataTable.vue`를 사용하고 화면별로 공통 밀도·편집 CSS를 재구현하지 않습니다.
- 공통 컴포넌트 선택과 계약은 [공통 컴포넌트](#)와 [StyledDataTable](#)을 따릅니다.
- 첨부파일 UI는 화면마다 다시 만들지 않고 목록은 `AttachmentListSection`, 폼 선택은 `AttachmentUploadField`를 사용합니다. `AttachmentUploadField`는 업로드하지 않고 선택 결과만 올려 주며, 실제 업로드·삭제는 부모 식별자가 확정된 뒤 화면의 저장 흐름이 수행합니다. 인증이 필요한 내려받기는 `<a href>`가 아니라 `useAttachmentDownload()`를 사용합니다.
- 입력 길라잡이는 필드마다 핸들러를 붙이지 않고 폼 컨테이너가 `useFormGuideActivation`으로 capture 단계에서 위임합니다. 대상 필드는 `data-guide-id`로 표시하고 활성 강조는 `base.css`의 `[data-guide-active]` 규칙에 맡깁니다.
- 페이지네이터 표시 문구는 `t()`로 직접 만들지 말고 `resolvePaginatorReportTemplate`을 사용합니다. PrimeVue `currentPageReportTemplate`과 `vue-i18n`이 같은 중괄호 문법을 쓰므로, 인자 없이 번역하면 `{totalRecords}` 같은 자리표시자가 빈 문자열로 치환되어 숫자가 사라집니다.

- 앱 전역에 하나만 존재해야 하는 오버레이(MFA 대화상자, 공통 안내 팝업, 스피드다이얼)는 `AppShell` 이 한 번만 마운트하고 화면은 그 인스턴스를 공유합니다. 화면마다 새로 배치하지 않습니다.
- `components/common` 컴포넌트는 명시적으로 import합니다.
- CSS 클래스는 kebab-case 또는 BEM을 사용합니다. DB camelCase 컬럼명을 클래스명에 그대로 넣지 않습니다.
- 전역 `user-select: none` 정책을 제거하지 말고 선택 가능한 요소만 `base.css` 허용 목록에 추가합니다.

6. 다국어

- 사용자 노출 고정 문구는 `i18n/messages/{도메인}.ts` 에 두고 `npm run check:copy` 기준선을 늘리지 않습니다.
- 번역 키는 한국어 문장이 아니라 의미가 안정적인 도메인 키를 사용하며 한국어·영어 트리를 함께 완성합니다.
- 용어는 [다국어 용어집](#), 언어 추가는 [언어 추가 가이드](#)를 따릅니다.
- 지원 언어는 `types/i18n.ts` 의 `APP_LOCALES`, 전환은 `useAppLocale().setAppLocale` 이 SoT입니다.
- 날짜·숫자·금액 포맷터에는 현재 `localeTag` 를 전달합니다.
- 메뉴명과 공통코드 표시명은 DB 번역이 SoT이며 업무 분기·저장에는 번역명이 아닌 코드값을 사용합니다.

7. 에디터와 문서 생성

- Tiptap nullable model은 `?? ''` 로 문자열 기본값을 지정하고 비동기 변수 해석의 stale 응답을 무시합니다.
- 커스텀 확장은 `components/editor/extensions/{content,media,table}/` 에 두고 정식 배럴로만 import합니다.
- 표 도구는 [Tiptap 표](#), 변수는 [Tiptap 변수](#)를 따릅니다.
- HWPX 변경은 `utils/hwp*.ts` 와 관련 테스트, Excel 내보내기는 `utils/excel.ts` 를 우선 확인합니다.
- 전자결재 신청서는 안정된 `form.id` 로 식별하고 양식별 코드를 `app/features/approval/forms/{양식명}/` 에 모읍니다. 추가 절차는 [신청서 확장 가이드](#)를 따릅니다.

8. 테스트와 검증

- 순수 유틸·store·composable 로직에는 Vitest 단위 테스트를 추가합니다.
- 신규 페이지나 핵심 사용자 흐름에는 Playwright 시나리오를 추가합니다.
- Mock과 변경 범위별 명령은 [프론트 테스트 가이드](#)를 따릅니다.

```
npm run format:check
npm run check
npm test
```

CSS 변경은 `npm run lint:css`, 사용자 흐름 변경은 `npm run test:e2e`, API 계약 변경은 `npm run codegen:check`, 정적 생성 변경은 `npm run generate -- --dry-run` 을 추가로 실행합니다.

9. 문서 위치

- 설치·실행·환경·배포·문제 해결: [README](#)
- 상세 개발 가이드: [docs/guides/README.md](#)
- 공통 컴포넌트 인덱스: [app/components/common/README.md](#)
- 백엔드 인증·인가 정책: `../it_backend/CLAUDE.md`
- 미구현·기술부채: `../TASK.md`

프론트엔드 개발 가이드

SOURCE: [it_frontend/docs/guides/README.md](#)

프론트엔드 개발 가이드

필수 작업 계약은 [프론트엔드 CLAUDE](#), 설치·실행·배포·문제 해결은 [프론트엔드 README](#)가 담당합니다. 이 디렉터리는 반복해서 참고하는 구현 방법과 설계 배경만 보관합니다.

아키텍처

- [API 클라이언트](#)
- [전자결재 신청서 확장](#)
- [알림 상태와 폴링](#)
- [공용 준비중 페이지](#)

컴포넌트

- [공통 컴포넌트](#)
- [StyledDataTable](#)
- 자동 등록 범위와 컴포넌트 목록은 [공통 컴포넌트 인덱스](#)를 함께 확인합니다.

에디터와 문서 생성

- [Tiptap 표 도구](#)
- [Tiptap 변수](#)

다국어

- [다국어 용어집](#)
- [언어 추가](#)

스타일과 테스트

- [디자인 토큰](#)

- [프론트엔드 테스트](#)

작성 원칙

- 디자인 시안·컴포넌트 미리보기 HTML은 `docs/preview/` 에 두고 앱 코드에서 import하지 않습니다.
- 가이드에는 재사용 가능한 규칙·예제·체크리스트만 기록합니다.
- 날짜, 완료 상태, 파일·테스트 개수처럼 자주 변하는 정보는 기록하지 않습니다.
- CLAUDE와 README의 본문을 복제하지 말고 정식 문서로 연결합니다.

API 클라이언트 가이드

SOURCE: [it_frontend/docs/guides/architecture/api-client.md](#)

API 클라이언트 가이드

선택 기준

상황	사용 도구	이유
페이지 진입 시 반응형 GET 조회	<code>useApiFetch<T></code>	Nuxt <code>useFetch</code> 반환값과 <code>watch-refresh</code> 사용
사용자 이벤트의 일회성 GET	<code>\$apiFetch</code>	명령형 호출에 적합
POST·PUT·PATCH·DELETE	<code>\$apiFetch</code>	변경 시점과 오류 처리를 호출자가 제어

두 래퍼 모두 인증 쿠키를 포함하고 401 갱신 흐름에 참여합니다.

OpenAPI 타입 codegen (FE-15)

백엔드 `GET /v3/api-docs` 스펙에서 요청·응답 타입만 생성합니다(`openapi-typescript`, 조건부 채택 — 루트 저장소의 `docs/superpowers/reports/2026-07-fe12-openapi-codegen-spike.md` 참고. `it_frontend` 와 별도 git 저장소라 상대 링크로 연결하지 않습니다). `fetch` 호출 함수는 생성하지 않으므로 `$apiFetch/useApiFetch` 호출부는 그대로 두고, 응답 타입만 `app/types/api.d.ts` 의 `paths / components["schemas"]` 에서 가져다 씁니다.

명령	용도
<code>npm run codegen</code>	백엔드가 기동된 상태에서 <code>app/types/api.d.ts</code> 를 재생성합니다.
<code>npm run codegen:check</code>	재생성 결과를 커밋된 파일과 비교만 합니다(파일을 쓰지 않음). 다르면 드리프트로 실패합니다.

- **언제 돌리나:** 백엔드 DTO·엔드포인트를 변경한 뒤, 또는 병합 전 스펙과 커밋된 타입이 어긋나지 않았는지 확인할 때 `npm run codegen:check` 를 돌립니다. 이 저장소에는 CI가 없으므로(`.github/workflows` 부재) 이 확인은 로컬에서 수행합니다.
- **백엔드 URL:** 기본값 `http://localhost:28080`. 다른 포트나 환경을 쓰려면 `OPENAPI_BACKEND_URL` 환경 변수로 재정의합니다.

- `app/types/api.d.ts` 는 생성물입니다. 직접 수정하지 마세요. 스펙이 바뀌면 `npm run codegen` 으로 재 생성하고 결과를 커밋합니다(백엔드 없이도 타입 검사가 되도록 생성물 자체를 커밋 대상으로 관리합니다).
- **적용 범위:** 현재 `app/composables/useCost.ts` 한 곳에만 생성 타입(`components["schemas"]` [`"CostDto.Response"`])을 연결했습니다(컴파일타임 호환성 검증, `ItCostMatchesApiSpec`). 전면 마이그레이션은 아직 하지 않았습니다 — 다른 composable은 여전히 수기 인터페이스를 사용합니다. 새 도메인에 적용할 때는 이 파일의 패턴 (스칼라 필드만 `extends` 로 비교하고, 화면 편의상 스펙보다 넓은 필드·nullable 필드는 명시적으로 제외)을 참고하세요.
- **가드의 한계:** `A extends B ? true : never` 비교는 구조적 타이핑이라 좌측(로컬 타입)의 초과 속성을 허용합니다 — 양쪽에 공통으로 있는 필드의 타입 불일치(예: `costTotXpAmt: number → string`)는 잡지만, 백엔드가 필드를 완전히 삭제·개명해 로컬 타입에만 남는 경우는 잡지 못합니다. 이 패턴만으로 필드 삭제성 드리프트까지 잡았다고 과신하지 마세요.

절대 URL

`$apiFetch` 는 `baseUrl` 을 자동으로 주입하지 않습니다. 상대 경로를 사용하면 Nuxt 개발 서버가 `index.html` 을 200으로 반환하여 JSON 파싱 오류처럼 보일 수 있습니다.

```
const config = useRuntimeConfig();
const { $apiFetch } = useNuxtApp();
const apiUrl = `${config.public.apiBase}/api/projects`;

const projects = await $apiFetch<Project[]>(apiUrl);
```

```
// 금지: 현재 Nuxt origin으로 전송될 수 있습니다.
await $apiFetch('/api/projects');
```

변경 요청 오류 처리

```
const toast = useToast();

try {
  await $apiFetch(apiUrl, { method: 'POST', body: payload });
  toast.add({ severity: 'success', summary: '완료', detail: '저장되었습니다.' });
} catch (error) {
  toast.add({ severity: 'error', summary: '오류', detail: formatApiError(error) });
}
```

- store 액션은 오류를 삼키거나 toast를 표시하지 않고 호출자에게 전파합니다.
- 서버 오류에 내부 정보가 포함될 수 있으면 일반 사용자 메시지로 치환합니다.
- `stores/auth.ts` 는 인증 플러그인과의 순환 참조 때문에 `$fetch` 를 직접 사용합니다.

재조회(refresh) 실패 판정

`useApiFetch` 가 돌려주는 `refresh()` 는 Nuxt `useAsyncData` 계약을 따르므로 실패해도 **reject**하지 않습니다. 그리고 결과는 성공·실패 두 가지가 아니라 **세 가지**입니다.

abort가 일어난 시점의 내부 상태는 취소 경로에 따라 다릅니다(Nuxt 4.4.8 `asyncData.js` 기준). `dedupe·타임아웃`으로 거부되는 `DOMException abort(:443-445)`는 `data·error`를 건드리지 않지만, `clear()`가 부르는 `clearNuxtDataByKey(:325-330)`는 `data`를 `_default()`로, `error`를 `undefined`로 덮은 뒤 `status`를 `'idle'`로 세웁니다 — `abort` 거부 핸들러는 `:437`의 조기 반환(승자 판별 실패)에 걸려 애초에 `:443-445`에 도달하지 않으므로, 위 두 경로는 서로 다른 코드가 만드는 결과입니다:

경로	Nuxt가 하는 일
성공	<code>data = 결과</code> , <code>error = undefined</code> , <code>status = 'success'</code>
실패	<code>error = createError(원인)</code> , <code>data = 기본값</code> , <code>status = 'error'</code>
<code>dedupe·타임아웃 abort(DOMException, :443-445)</code>	<code>error·data</code> 를 건드리지 않고 <code>status</code> 만 <code>'idle'</code> 이 됩니다(완료 상태가 아닙니다)
<code>clear()</code> (<code>clearNuxtDataByKey</code> , <code>:325-330</code>)	<code>data</code> 를 <code>_default()</code> 로, <code>error</code> 를 <code>undefined</code> 로 덮은 뒤 <code>status = 'idle'</code>

주의: 이 표는 abort가 일어난 그 순간의 내부 상태일 뿐입니다. `dedupe`로 취소된 요청은 Nuxt 내부에서 이전 `promise`가 승자(새 요청) `promise`를 그대로 채택(`adopt`)하므로, `await refresh()`가 실제로 `resolve`하는 시점에는 위 표의 `abort` 행과 다른 상태를 관측하게 됩니다. `await refresh()`가 **resolve**한 뒤 호출자가 실제로 보는 결과는 취소 원인별로 다릅니다:

취소 원인	호출자가 보는 결과
<code>dedupe</code> (같은 키에 새 요청 — 이 코드베이스에서는 <code>onActivated</code> 재조회, <code>tokenRefreshSignal</code> 갱신)	승자 요청의 결과: <code>'success'</code> 또는 승자의 오류 throw
<code>clear()</code> 중 취소	<code>'aborted'</code>
새 요청이 아직 진행 중이어서 <code>status</code> 가 <code>'pending'</code> 으로 잔류	<code>'aborted'</code>

즉 이 코드베이스가 실제로 겪는 `dedupe-abort(onActivated 재조회, tokenRefreshSignal 갱신)`에서 `refreshOrThrow`는 `'aborted'`를 반환하지 않습니다. 승자 요청의 `'success'`를 반환하거나 승자의 오류를 throw합니다. `'aborted'`가 실제로 나오는 경로는 `clear()`와 `'pending'` 잔류뿐입니다.

'aborted' 는 실패도 성공도 아닙니다. 오류가 없으므로 오류 안내를 띄우면 안 되고, 조회가 완료되지 않았으므로 "최신 데이터를 받았다"는 전제의 후속 동작(성공 토스트, 편집 종료, 화면 전환)을 해서도 안 됩니다. 따라서 `error.value` 의 `truthy` 여부만으로 판정하면 안 됩니다. 직전 실패의 잔여 `error` 가 남은 채 abort되면 멀쩡한 호출을 실패로 오판하고(거짓 실패), 잔여 `error` 가 없는 채 abort되면 미완료를 성공으로 오판합니다(거짓 성공).

- `try { await refresh() } catch { ... } / refresh().catch(...)` 는 도달하지 않는 죽은 분기입니다. 쓰지 마세요.
- 상태로 다룰 때: `await refresh()` 뒤에 `status.value` 를 먼저 보고 필요하면 `error.value` 를 확인합니다.
- 예외로 다룰 때: `refreshOrThrow` 를 사용합니다. 호출 형태는 두 가지이며 기준은 다음과 같습니다.
 - `useApiFetch / useAsyncData` 의 반환 객체를 그대로 들고 있으면 객체 형태
`refreshOrThrow(result)` 를 씁니다. `result` 가 `{ refresh, error, status }` 를 가진 객체 하나면 됩니다. 반환 객체를 그대로 넘길 때만 세 값이 같은 요청에서 나온 것이 보장됩니다. 이 보장은 타입 시스템이 아니라 "재조립하지 않는다"는 호출 습관에서 나옵니다 — 서로 다른 요청의 필드를 섞어 만든 객체 리터럴(`{ refresh: a.refresh, error: b.error, status: b.status }`)도 `{ refresh, error, status }` 구조만 맞으면 컴파일이 통과합니다(`npx tsc --noEmit --strict` 로 실측 확인, 아래 3-인자 형태와 동일한 제약). 그래도 반환 객체를 그대로 넘길 수 있는 호출부에서는 호출부가 짧아지고 재조립할 필요가 없어 실수 여지가 줄어드므로 이 형태를 기본으로 씁니다.
 - 파사드가 개별 ref만 넘기는 경우 3-인자 형태 `refreshOrThrow(refresh, error, status)` 를 씁니다. `status` 는 선택 인자가 아니라 판정 기준이므로 반드시 함께 넘깁니다. 인자 순서는 타입으로 검증되지만 서로 다른 요청의 ref를 섞어 넘기는 것은 타입으로 막히지 않으므로 반드시 같은 요청에서 나온 값을 넘깁니다. 현재 운영 코드에는 이 형태의 호출부가 없습니다 — 마지막 사례였던 `useCostEditingState` 가 통합 리뷰 ②에서 화면 공유 가드로 합쳐졌습니다. 새 지점을 만들 때는 `useRefreshGuard` 를 쓸 수 있는지 먼저 확인하세요.

```
// 객체 형태 – 반환 객체를 직접 들고 있을 때 (권장 기본형)
const costs = useApiFetch<Cost[]>(url);

try {
  const outcome = await refreshOrThrow(costs);
  // 'success' = 완료(승자 요청 포함), 'aborted' = clear()로 취소되었거나 다른 요청이 아직 진행 중
  if (outcome === 'success')
    toast.add({ severity: 'success', summary: '최신 목록으로 갱신했습니다.' });
} catch (e) {
  // status === 'error'인 경우에만 여기로 온다(dedupe-abort라면 승자의 오류)
  toast.add({ severity: 'error', summary: '목록 갱신 실패', detail: formatApiError(e) });
}

// 3-인자 형태 – 파사드가 개별 ref만 넘길 때 (현재 운영 코드에는 호출부가 없습니다)
await refreshOrThrow(refreshList, listError, listStatus);
```

- 반환값을 무시해도 됩니다. 무시하면 "실패는 예외로 받고 abort는 조용히 넘긴다"가 되어 대부분의 재조회에 적절합니다. **성공에만 실행해야 하는 후속 동작이 있을 때만** `=== 'success'` 를 확인하세요.
- `status` 는 `refreshOrThrow` 가 abort를 판별하는 유일한 근거이자, 같은 `useApiFetch` 인스턴스의 모든 요청이 공유하는 상태입니다. `await refresh()` 직후의 한 마이크로태스크 사이에 같은 인스턴스의 다른 요청이 끼어들면(성공 직후 새 `execute`가 시작되거나, 다른 요청이 먼저 실패하는 등) `=== 'success'` 판정이 그 요청의 상태를 대신 관측할 수 있습니다. 반환값을 무시하는 기본 사용에서는 무해하지만, 성공 가드를 쓰는 호출부에서는 이 한계를 감안하세요.
- 쓰기 성공 직후의 재조회는 실패를 반드시 판정하세요. 판정하지 않으면 사용자는 성공 안내와 함께 낡은 목록을 최신으로 오인합니다.
- 테스트에서 재조회 실패를 모사할 때는 **reject가 아니라** "resolve + `data` 초기화 + `error` 세팅 + `status = 'error'`"로 모킹해야 실제 계약과 일치합니다. `clear()` 취소나 pending 잔류로 인한 `'aborted'` 는 "resolve + `error` 미변경 + `status = 'idle' / 'pending'`"으로 모킹합니다. `dedupe-abort(onActivated, tokenRefreshSignal)`는 `'aborted'` 가 아니라 승자 요청의 결과이므로 위 두 경로 중 하나로 모킹하는 것이 맞고, 별도의 `"dedupe-abort → 'aborted'"` 모킹을 만들지 마세요.

주의: 같은 이름이라도 자체 `$apiFetch` 기반으로 정의한 `refresh` 함수(예: `useNotifications`)는 정상적으로 reject하므로 기존 try/catch가 맞습니다. `refreshOrThrow` 로 감싸면 안 됩니다.

`useRefreshGuard` — 재조회 실패를 화면 상태로 남기는 기본 처방

`refreshOrThrow` 는 판정만 합니다. **사용자에게 무엇을 보여줄지까지 포함한 표준 처방은**

`app/composables/useRefreshGuard.ts` 를 씁니다. 재조회 지점마다 try/catch·플래그·Toast를 손으로 다시 짜지 마세요.

```
const costsFetch = fetchCosts();
const { data: costsRaw, refresh: refreshCostsRaw } = costsFetch;

/* 컴포넌트 setup(또는 setup이 호출하는 파사드)에서 만듭니다 - 아래 "생성 위치" 참고 */
const { refreshFailed, refreshFailureDetail, attemptRefresh, retryRefresh } = useRefreshGuard(
  costsFetch /* 반환 객체를 그대로 넘긴다 - 필드를 재조립하지 않는다 */,
  { toast, summary: '목록 갱신 실패', logLabel: '[CostList] 목록 재조회 실패' },
);

/* 쓰기 성공 직후(Class B) - "쓰기는 반영됐고 목록만 못 불러왔다"를 문구가 분명히 한다 */
await attemptRefresh(
  `저장은 ${successCount}건 반영되었습니다. 목록을 불러오지 못했으니 잠시 후 다시 조회해 주세요.`
);

/* 쓰기가 선행하지 않는 재조회(Class C-3) - 쓰기 결과 문장을 넣지 않는다 */
await attemptRefresh('목록을 불러오지 못했습니다. 잠시 후 다시 시도해 주세요.');
```

계약:

항목	내용
target	{ refresh, error, status } — useApiFetch 반환 객체를 그대로 넘깁니다. 생성 시점에 배선을 검증하며 어긋나면 예외입니다
생성 위치	활성 effect scope 안(컴포넌트 setup / 파사드 / 테스트의 effectScope().run()). 밖에서 만들면 예외입니다
실패 판정	settleRefresh 위임 = status 기준. error.value truthy 판정을 직접 하지 않습니다
예외	attemptRefresh는 재조회 실패로는 던지지 않습니다. 배선 오류 등 그 밖의 예외는 삼키지 않고 그대로 올립니다
반환	'success' / 'aborted' / 'failed' — 기본은 무시합니다
실패 시	refreshFailed=true, refreshFailureDetail=인자 문구, 오류 Toast, console.error 진단 로그
성공 시	두 상태를 명시적으로 초기화합니다
abort 시	상태를 세우지도 지우지도 않습니다(실패도 성공도 아님)
자동 해제	status가 'success'가 되면(이 가드를 거치지 않은 경로의 성공 포함) 실패 상태를 해제합니다
retryRefreshh()	재조회만 다시 시도합니다. 쓰기를 재시도하면 중복 저장이 되므로 재시도 버튼은 반드시 이 함수에 연결합니다
Toast 억제	재조회가 도는 그 구간에만 그 인스턴스의 공통 오류 Toast를 끕니다(아래 "재조회 구간 한정 억제")
데이터 보존	재조회가 실패해도 data가 직전 정상값을 유지합니다(아래 "마지막 정상값 보존"). 401·403은 예외입니다

적용 규칙:

- 실패 문구는 호출 맥락이 정합니다. 헬퍼는 문구를 만들지 않습니다. 쓰기 성공 안내가 선행한 지점은 쓰기 결과 문장("저장은 N건 반영되었습니다")을 반드시 앞에 두어 "저장이 날아갔다"는 오해와 중복 쓰기 시도를 막습니다.
- 문구는 실제로 비는 대상과 일치해야 합니다. 한 화면이 서로 다른 조회 둘을 재조회한다면(예: admin/menus의 관리 트리 /api/admin/menus와 사이드바 /api/menus) 둘은 독립적으로 실패하고 비는 대상도 다르므로 가드도 문구도 대상별로 하나씩 둡니다. 반대로 같은 목록을 여러 모듈이 재조회하는 화면(전산업무비)은 가드를 하나만 만들어 공유합니다 — 가드가 갈라지면 배너가 서로 다른 사실을 말합니다.
- 기존 try/catch를 지우지 마세요. 쓰기 실패에 대해서는 여전히 도달하는 분기입니다. 죽어 있던 것은 "재조회 실패 시의 도달"뿐입니다.
- refreshFailed / refreshFailureDetail은 화면이 배너로 렌더링하고 그 배너에 retryRefresh를 건 버튼을 둡니다. Toast는 사라지므로 상태 없이 Toast만 두면 사용자가 놓칩니다.

생성 위치와 배선 검증 (오용 방지)

가드는 자동 해제 위치를 등록하므로 **활성 effect scope 안에서만** 만들 수 있습니다. Vue는 스코프 없이 `watch` 가 생성돼도 경고하지 않고 위치가 영원히 멈추지 않으므로, 이벤트 핸들러 안에서 만들면 클릭마다 위치가 쌓입니다(그리고 그 가드의 실패 상태를 렌더링할 주체도 없습니다). `useRefreshGuard` 는 `getCurrentScope()` 로 이를 확인해 **생성 시점에 예외**를 냅니다.

- 화면 코드: 컴포넌트 `setup`, 또는 `setup` 이 호출하는 파사드 composable에서 만듭니다.
- 테스트: 컴포넌트를 마운트하지 않고 만든다면 `effectScope().run(() => useRefreshGuard(...))` 로 감싸고 `afterEach` 에서 `scope.stop()` 합니다.

트레이드오프(의도한 것): 이 두 검증은 `import.meta.dev` 구분 없이 **프로덕션에서도 예외**입니다. 드물게 실행되는 핸들러 안에서 가드를 잘못 만들면 그 지점에서 화면이 죽습니다. 그래도 유지하는 이유는, 조용히 넘어갔을 때의 결과가 "위치가 영구히 쌓이고 실패 안내가 사라진다"라서 **관측되지 않는 오작동**이기 때문입니다(1차 리뷰에서 테스트 101건이 전부 통과하면서 모든 재조회가 실패로 관측된 사례가 있었습니다). 가드는 반드시 `setup` 경로에서 만들고, 이벤트 핸들러에서 만들지 마세요.

`target` 검증은 `refresh` 가 함수인지, `error.status` 가 `isRef` 인지까지만 봅니다. **서로 다른 요청의 ref를 섞어 재조립한 객체는 통과합니다**(TypeScript 구조적 타이핑과 같은 한계). 그래서 규칙은 여전히 "반환 객체를 그대로 넘긴다"입니다.

`target` 도 생성 시점에 검증합니다(`refresh` 가 함수인지, `error.status` 가 ref인지). 부분 mock이나 `as never` 캐스팅으로 `status` 를 빠뜨리면 `TypeError` 가 나고 과거의 가드 `catch` 가 그것을 **"재조회 실패"**로 둔갑시켜 왔습니다(테스트는 전부 통과하면서). 이제는 크게 실패합니다.

같은 이유로 가드에는 **`catch` 가 없습니다**. 재조회 판정은 `settleRefresh` 가 `await refresh()` 직후 **한 번의 동기 블록**에서 `status.error` 를 함께 읽어 **값으로** 확정합니다(`'success' / 'aborted' / 'failed'`). 판정을 예외로 실어 나르면 억제 구간의 `finally` (= `await` 경계)를 지나 `catch` 에 도달하는 사이에 같은 조회의 다른 요청이 `status` 를 `'pending'` 으로 바꿀 수 있고(pre-flush 위치가 트리거하는 `execute()` 는 동기적으로 `pending` 을 세웁니다), 그러면 `catch` 가 상태를 다시 읽어 **정상적인 재조회 실패를 그대로 다시 던집니다** — 쓰기는 성공했는데 호출부 `catch` 가 "저장 실패"를 띄우는 결과가 됩니다. 판정은 `await` 경계 뒤에서 다시 읽지 않습니다. `refresh()` 자체가 `reject`하는 배선(= `useAsyncData` 계약을 따르지 않는 대상)은 여전히 **삼키지 않고 그대로 호출부로 올라갑니다**.

예외 흐름이 필요한 호출자를 위해 `refreshOrThrow` 는 그대로 남아 있으며, 내부적으로 `settleRefresh` 의 판정을 예외로 바꿔 줍니다.

마지막 정상값 보존 (재조회 실패에도 화면이 비지 않게)

가드는 배너만 띄우는 장치가 아닙니다. 데이터가 비는 것 자체를 막습니다.

Nuxt는 재조회가 실패하면 데이터를 `default()` 가 돌려주는 값으로 덮습니다
(`node_modules/nuxt/dist/app/composables/asyncData.js` 447-449행):

```
asyncData.error.value = createError(error);
asyncData.data.value = unref(options.default());
asyncData.status.value = 'error';
```

default 가 없으면 undefined 이므로 data 가 비고, 거기서 파생된 모든 것이 함께 무너집니다.
 watch(detail) 에 null 가드를 넣어도 이것은 막지 못합니다 — 로컬 편집 필드는 지켜지지만
 isDraft / isInProgress / showXXX 같은 표시 조건이 뒤집혀 편집 UI가 읽기 전용으로 스왑되거나 섹션이 통째로 사라지고, statusData?.totalCount ?? 0 형태의 숫자는 0으로 표시됩니다.

그래서 useApiFetch 는 보존용 default 를 항상 넘기고, useRefreshGuard 가 생성 시점에 target.enableKeepPreviousData() 로 그것을 겁니다. 커진 조회는 성공할 때마다 그 값을 보존값으로 갱신하고, 실패하면 default() 가 그 값을 돌려줍니다. 데이터가 한 순간도 undefined 가 되지 않으므로 사후에 되돌리는 방식과 달리 파생 computed가 뒤집혔다 돌아오는 창(= 편집 UI 언마운트/재마운트)이 없습니다.

- 가드를 붙이면 자동입니다. 별도 옵션이 없습니다. 그러면 useRefreshGuard(target, { keepPreviousData: false }).
- 가드를 쓰지 않는 조회는 종전과 완전히 같습니다. 커지 않은 default 는 Nuxt 기본값과 같은 undefined 를 돌려줍니다. 직접 커려면 useApiFetch 반환 객체의 enableKeepPreviousData() 를 **활성 effect scope 안에서**(컴포넌트 setup·파사드·effectScope().run()) 호출합니다. scope 밖 호출은 보존 위치를 정리할 수 없으므로 useRefreshGuard 와 같은 규율로 예외입니다.
- 파사드가 개별 필드만 노출한다면 enableKeepPreviousData 도 함께 노출해야 가드가 켤 수 있습니다.
- clear() 와 언마운트 purge는 영향받지 않습니다 — 진행 중 요청과 겹쳐도 그렇습니다. clear() 는 abort를 동기로 부르고 곧바로 데이터를 비우는데(asyncData.js 214-224 → 317-336행) 거부 핸들러는 마이크로태스크이므로, 비우는 시점의 status 는 아직 'pending' 입니다. 즉 status === 'pending' 만으로는 실패 경로와 비우기 경로를 구분할 수 없습니다. 보존 판정은 status 와 함께 이번 요청이 시작된 뒤 error 에 새 오류가 대입됐는지를 봅니다(실패 경로는 448행 직전에 error.value = createError(...) 로 항상 새 참조를 넣습니다). 또한 status 가 'idle' 로 전이할 때 보존값을 그 시점의 data 로 재동기화하므로, **비운 데이터가 다음 재조회 실패에서 되살아나지 않습니다**. abort는 data 를 건드리지 않으므로 화면에 남은 직전 정상값이 그대로 보존값으로 유지됩니다.

보존하지 않는 경우 — 401-403 (권한 성격의 실패):

권한이 회수됐을 수 있는 상태에서 낡은 데이터를 계속 보여주지 않습니다. 이 두 상태에서는 종전대로 data 가 비고 배너가 이유를 설명합니다. 판정은 error.value 의 statusCode 로 하며 (h3 createError 가 statusCode / status 를 그대로 옮깁니다), 상태를 읽을 수 없으면 보존합니다. 404-409-5xx 등 나머지 실패는 보존 대상입니다 — 권한 신호가 아니기 때문입니다. **보존하면 안 되는 조회를 새로 발견하면 keepPreviousData: false 로 그 지점만 끄고 이유를 주석에 남기세요.**

401의 두 분기를 구분하세요 — 위 문장은 "토큰 갱신 실패" 분기에만 해당합니다. Access Token 만료 → 401 → 토큰 갱신은 이 앱에서 가장 흔한 실패 경로이고, 갱신 결과에 따라 화면 거동이 갈립니다. 실측 (tests/unit/composables/useApiFetch.keepPreviousData.test.ts 의 "401 투명 재조회 경로") 결과는 다음과 같습니다.

분기	data 전이	설명
갱신 성공 (refreshed === true)	직전값 → 새 값 (빈 적 없음)	조용한 재시도. 화면이 비지 않습니다
갱신 실패 (refreshed === false)	직전값 → undefined	로그아웃 흐름. 비우는 것이 의도된 동작입니다

갱신 성공 분기에서 화면이 비지 않는 이유는 보존이 아니라 **순서**입니다. `ofetch`는 `onResponseError`를 **await한 뒤에** 오류를 던지므로(`ofetch.CWycOUEr.mjs` 320-327행), `tokenRefreshSignal.value++`가 원 요청의 거부보다 먼저 일어납니다. 그 사이 `pre-flush` 위치가 새 `execute()`를 돌려 `_asyncDataPromises[key]`를 교체하므로, 뒤늦게 도착한 401 거부는 `_asyncDataPromises[key] !== promise` 조기 반환(`asyncData.js` 437행)에 걸려 `default()`를 **아예 호출하지 않습니다**. 측정에서 `default()` 호출 수는 생성 시점 1회뿐이었습니다. 따라서 401에서 보존을 켜지 않은 결정은 이 지배적 경로에 손해를 주지 않습니다. 다만 이것은 **마이크로태스크 순서에 기대는 성질**이므로 위 테스트가 그 순서를 계속 관찰합니다.

한계 — 같은 `asyncData` 키를 여러 인스턴스가 여는 조화:

Nuxt는 같은 키의 **최초 인스턴스**가 만든 `default`만 보관합니다(`asyncData.js` 88-91행의 `if (!existing?._init) buildAsyncData(...)`). 이 한계는 **양방향**입니다.

- **켜지지 않는 방향:** 최초 인스턴스가 보존을 켜지 않았다면 나중 인스턴스가 켜도 **효과가 없습니다**. 실제로 `/api/menus`가 여기에 해당합니다 — `AppHeader / AppSidebar`가 먼저 열고 `admin/menus`의 가드는 나중 인스턴스이므로, 그 화면의 사이드바 메뉴 조화는 보존되지 않습니다(배너는 정상 동작).
- **꺼지지 않는 방향:** 가드를 붙인 인스턴스가 그 키를 **먼저** 열면, 같은 키를 나중에 여는 **비가드 소비자까지 보존이 켜집니다**. 보관된 `default` 하나를 모두가 공유하기 때문입니다. 즉 "가드를 쓰지 않는 조화의 동작은 종전과 같다"는 **키를 공유하지 않는 조화에 한해** 참입니다. 아래 `/api/cost`가 실제로 이 쌍에 해당하므로, 가드를 새로 붙일 때 **그 키를 다른 곳에서도 여는지** 먼저 확인하세요.

화면 전용 조화(키가 그 화면에서만 만들어지는 조화)는 두 방향 모두 해당되지 않습니다.

공유 키 실측 — `/api/cost` (query 없음)

키 생성 규칙상 **명시 key가 없으면 `asyncData` 키는 `URL·method·baseUrl·query·body`만의 함수**입니다(`useApiFetch`의 `buildSuppressionScopeKey` JSDoc에 근거를 적어 두었습니다). `suppressNetworkError` 같은 확장 옵션은 `useFetch`로 넘어가기 전에 제거되므로 키에 관여하지 않습니다. 따라서 `useCost().fetchCosts()`를 **query 없이** 부르는 아래 다섯 곳이 모두 같은 키(`/api/cost`)를 엽니다.

소비자	가드
<code>app/composables/useCostListPage.ts</code>	있음 (생성 시점에 <code>enableKeepPreviousData()</code> 호출)
<code>app/pages/budget/list.vue</code>	있음 (동일)
<code>app/pages/project/contract/index.vue</code>	없음 (<code>await Promise.all([fetchProjects(), fetchCosts()])</code>)
<code>app/pages/project/deliberation/index.vue</code>	없음 (동일)
<code>app/pages/project/payment/index.vue</code>	없음 (동일)

`app/pages/budget/approval.vue` (`{ apfSts: 'none' }`)와 `app/pages/info/index.vue` (`{ bseYy }`)는 query가 있어 **다른 키**이므로 이 공유 범위 밖입니다.

`app/app.vue`가 `<NuxtPage keepalive :keepalive-props="{ max: 10 }" />`이므로 방문 순서가 뒤집히는 세션이 실제로 성립합니다.

- 비가드 페이지(입찰/계약·심의·지급)가 먼저 열면 → 가드 페이지의 `enableKeepPreviousData()` 가 **조용히 무효**가 되어 전산업무비 목록·예산 목록의 보존이 걸리지 않습니다(배너·Toast는 정상 동작).
- 가드 페이지가 먼저 열면 → 비가드 3개 화면까지 보존이 함께 커져, 그 화면들의 전산업무비 목록은 재조회가 실패해도 직전 정상값을 유지합니다.

`useCost` 는 `useApiFetch` 반환 객체를 그대로 돌려주는 **통과형 파사드**라 `enableKeepPreviousData` 스위치가 그대로 새어 나갑니다. 같은 이유로 `useMenu` 는 그 스위치를 **의도적으로 비노출**합니다(위 참고).

`useBoard` 도 같은 이유로 비노출이었으나 **2026-08-06 FE-24에서 아래 "향후 선택지"를 실제로 적용**했습니다 — `useBoard(options?: { key?: string })` 로 명시 `key` 를 받아 `admin/boards/index.vue` 만 `'admin-board-meta'` 를 열고, 공개 게시판 화면은 키를 넘기지 않아 종전 동작(URL 기반 자동 키)을 그대로 유지합니다. 키가 갈렸으므로 `enableKeepPreviousData` 를 노출해도 공개 화면으로 파급되지 않습니다. `key` 를 넘기지 않으면 Nuxt가 그 값을 `||` 로만 검사하므로(`node_modules/nuxt/dist/app/composables/fetch.js`) `{ key: undefined }` 와 키 미전달이 동일하게 동작합니다 — 공개 화면의 dedupe 그룹은 변하지 않았습니다.

현재 선택(2026-07-31): 동작은 바꾸지 않고 이 범위를 문서로만 고정합니다. 어느 순서든 사용자에게 해로운 동작은 관측되지 않았고(보존은 실패 시 낡은 값을 남길 뿐이며 401·403은 어차피 보존 제외), 세 화면의 전산업무비 목록은 읽기 전용 표시용이기 때문입니다.

향후 선택지(이번에 구현하지 않음): 이 조회에 명시 `key` 를 주어 **소비자별로 분리**하면 순서 의존이 사라집니다. 다만 명시 `key` 는 `asyncData` 키 자체를 바꾸므로 **dedupe 그룹이 함께 바뀝니다** — 지금은 다섯 소비자가 하나의 `asyncData` 엔트리(캐시·진행 중 요청·`_asyncDataPromises` 슬롯)를 공유하는데, 키를 나누면 그 공유가 사라져 요청 수와 취소(`abort`) 경로가 달라집니다. 억제 범위 키도 함께 갈라집니다. 별도 판단이 필요하므로 결정 전에는 적용하지 마세요.

자동 해제 판정

자동 해제 판정은 `status === 'success'` 입니다. 데이터의 truthiness가 아닙니다 — `useApiFetch(url, { default: () => [] })` 처럼 선언한 조회는 재조회가 실패해도 `data` 가 truthy한 `[]` 가 되므로, `data` 를 근거로 삼으면 배너가 뜬 직후 자동으로 지워집니다. 위 보존 장치가 켜진 조회도 마찬가지로 실패 후 `data` 가 truthy하게 남으므로, **판정 근거를 data로 바꾸지 마세요.** `target` 에 `data` 를 넘길 필요는 없습니다. 호출자가 `default` 를 넘기면 보존이 꺼져 있을 때만 그 값이 쓰이고, 켜져 있으면 직전 정상값이 우선합니다.

한계(의도한 것): 이 가드를 거치지 않은 경로가 실패했을 때 배너를 새로 띄우지는 않습니다. 그 실패에는 불일 맥락 문구가 없기 때문입니다(문구는 호출 맥락이 정합니다). 판정 없는 경로가 실패한 뒤 화면이 낡은 채 남을 수 있으므로, 그 경로에도 가드를 붙이는 것이 정답입니다.

재조회 구간 한정 억제 (Toast가 3중으로 겹치지 않게)

가드는 자체 실패 Toast + 배너로 안내합니다. 여기에 `useApiFetch` 의 공통 오류 Toast까지 겹치면 같은 실패에 Toast 2개 + 배너가 됩니다. 그렇다고 `suppressErrorToast` 를 켜면 **인스턴스 전체**가 조용해져 최초 조회 실패까지 무음이 됩니다(아래 "억제 판단 기준" 참고).

그래서 `useApiFetch` 는 호출마다 억제 실행기 `runWithErrorToastSuppressed(fn)` 를 만들어 반환 객체에 붙입니다. 가드는 `refresh()` 를 그 안에서 돌리므로 억제 범위가 **"이 조회의 이 구간"** 으로 한정됩니다. 모듈 전역 플러그였다면 `Promise.all` 로 동시에 도는 다른 조회(`budget/list.vue` 등)의 Toast까지 막혔을 자리입니다.

다.

억제 단위는 인스턴스가 아니라 asyncData 키입니다 (URL·method·baseUrl·query·body가 같으면 같은 키). Nuxt는 같은 키의 **최초 인스턴스**가 만든 오류 핸들러만 보관하고 두 번째 이후 인스턴스의 핸들러는 버립니다 (asyncData.js: if (!existing?._init) buildAsyncData(...)). /api/menus 처럼

AppHeader · AppBreadcrumb · AppSidebar · admin/menus 가 모두 useMenu() 로 여는 조회에서, 억제가 인스턴스 지역이면 실제로 실행되는 레이아웃 컴포넌트의 핸들러가 페이지 가드의 카운터를 읽지 못해 **억제가 조용히 사라집니다**. 같은 키는 같은 조회이므로 키 단위 억제가 무관한 요청의 Toast를 삼키지는 않으며, 서로 다른 키의 동시 조회는 그대로 안내됩니다. Nuxt의 키 생성 자체는 바꾸지 않고(= dedupe 그룹 불변) 같은 입력에서 같은 문자열이 나오는 자체 식별자를 억제 Map의 키로만 씁니다. FormData 처럼 직렬화할 수 없는 요청은 인스턴스 전용 키로 물려나 억제가 그 인스턴스 안에만 머뭅니다(과잉 억제보다 억제 누락이 안전합니다).

- 반환 객체를 그대로 넘기면 자동으로 배선됩니다. 별도 옵션이 없습니다.
- **await 한 반환값을 넘겨도 됩니다**. Nuxt useAsyncData 는 promise 객체를 돌려주면서 await 시에는 **내부의 다른 객체**(asyncReturn)를 resolve합니다
(node_modules/nuxt/dist/app/composables/asyncData.js 의 Object.assign(asyncDataPromise, asyncReturn) + .then(() => asyncReturn)). 그래서 promise에만 붙인 필드는 await 뒤에 사라집니다. useApiFetch 는 실행기를 **promise와 resolve되는 값 양쪽에** 붙이므로 const f = await fetchAdminTree() 와 const f = fetchAdminTree() 가 똑같이 동작합니다. 이 계약을 검증하는 테스트 mock은 tests/support/nuxtAsyncData.ts 를 쓰세요 — 리터럴을 돌려주는 평범한 async 함수 mock은 이 결함을 잡지 못합니다.
- 파사드가 개별 필드만 노출한다면 runWithErrorToastSuppressed 도 함께 노출하세요(useMenu 참고). 노출하지 않으면 공통 Toast가 함께 뜰 뿐 동작은 같습니다. 같은 이유로 enableKeepPreviousData 도 함께 노출합니다(노출하지 않으면 그 조회만 보존 없이 동작합니다).
- 단, useMenu 는 enableKeepPreviousData 를 일부러 노출하지 않습니다. 위 문장의 "useMenu 참고"는 runWithErrorToastSuppressed 에만 걸립니다. /api/menus 는 레이아웃 컴포넌트가 먼저 여는 **공유 키**라 거기서 보존을 켜면 아래 "한계" 절의 역방향 파급으로 **같은 키를 여는 비가드 소비자 전부의** 동작이 바뀝니다. 공유 키 파사드에서는 노출 여부를 그 파급까지 보고 결정하세요.
- 억제 대상은 네트워크·403·404·5xx이며 **401 인증 흐름은 억제 대상이 아닙니다**.
suppressNetworkError · suppressNotFound · suppressErrorToast 의 의미도 그대로입니다.

같은 조회를 여러 모듈이 재조회하면 가드는 하나만 (통합 리뷰 ②)

app/composables/costList/useCostEditingState.ts 의 viewMode 위치는 종전에 refreshOrThrow 3-인자 형태 + try/catch + 자체 Toast를 손으로 조립해 **배너 상태가 없었습니다**. 같은 costsFetch 를 재조회하는 나머지 네 경로(저장·취소는 useCostPersistence, 단말기 저장은 useCostTerminalDialogs, Excel 업로드는 useCostExcelTransfer)가 모두 useCostListPage 의 공유 가드를 경유하는데 이 경로만 벗어나 있어서

- 이 경로의 실패는 refreshFailed 배너에 나타나지 않았고 [다시 조회] 재시도 경로도 없었으며,
- 가드의 runWithErrorToastSuppressed 밖이라 5xx에서 **공통 Toast + 자체 Toast로 안내가 2개**였습니다 (suppressNetworkError 는 네트워크 오류만 끕니다).

지금은 파사드가 attemptRefresh 를 주입해 다섯 경로가 하나의 출처를 씁니다. **같은 조회를 여러 모듈이 재조회하면 가드를 하나만 만들어 공유하고, 모듈에는 가드 자체나 attemptRefresh 를 주입하세요**.

한편 같은 파일의 `onActivated` 위치는 판정 없이 맨 `refreshCostsRaw()` 를 호출합니다. 이것은 가드가 건 `status` 위치가 성공을 감지해 배너를 내리는 "가드를 거치지 않는 경로"로 설계상 허용됩니다(보존도 가드 생성 시점에 이미 켜져 있습니다).

호출부가 없는 재조회 (`watch` 옵션 자동 재조회)

`useApiFetch` 는 `watch` 옵션으로도 자동 재조회합니다(필터·페이징 화면이 그렇게 갱신됩니다). 이 경로에는 감쌀 `await refreshXxx()` 호출이 없으므로 가드의 `attemptRefresh` 를 적용할 수 없습니다. 가드는 이 경우를 위한 `status` 위치를 제공하지 않습니다 — 가드 범위 밖입니다.

그 화면의 조회 실패는 `useApiFetch` 의 공통 오류 Toast와 `error / status` 를 읽는 화면 자체의 상태 표현이 담당합니다(아래 "조회 상태 표현"). 가드를 억지로 끼워 넣지 말고, 안내가 부족하면 그 화면에 인라인 오류 상태를 만드세요. 다만 그 화면에 쓰기 직후의 명시적 재조회 호출이 따로 있다면 그것은 가드 대상입니다.

공통 오류 Toast 억제 (`useApiFetch` 옵션)

`useApiFetch` 는 오류 응답에 공통 Toast를 띄웁니다. 화면이 자체 오류 UI를 갖고 있으면 안내가 중복되므로 억제 옵션을 씁니다.

옵션	억제 범위	언제
<code>suppressNotFound: true</code>	404만	데이터 없음이 정상인 조회(코드 조회 등)
<code>suppressNetworkError: true</code>	네트워크 오류(서버 연결 불가)만	네트워크 실패만 자체 안내하는 화면
<code>suppressErrorToast: true</code>	네트워크 · 403 · 404 · 5xx 전부	실패를 배너·인라인 경고·전용 Toast로 직접 안내하는 화면

- `suppressErrorToast` 는 나머지 두 옵션을 포함하는 상위 옵션입니다. 기존 두 옵션의 의미는 그대로이며, 어느 하나라도 `true` 면 해당 Toast는 억제됩니다.
- 세 옵션 모두 기본값은 `false` (=공통 Toast 표시) 이며 옵션을 주지 않은 호출부의 동작은 바뀌지 않습니다.
- 401은 어떤 옵션으로도 억제되지 않습니다. 401 분기는 Toast가 아니라 토큰 갱신·로그아웃·리다이렉트를 수행하는 인증 흐름입니다.

쓰면 안 되는 경우: 자체 오류 UI가 없는 화면. `suppressErrorToast: true` 만 켜고 실패를 표시하지 않으면 사용자는 권한 없음·서버 오류 상황에서 아무 안내도 받지 못한 채 빈 화면만 보게 됩니다. 억제는 "안내를 없앤다"가 아니라 "안내를 화면이 책임진다"는 선언이므로, 켜는 커밋에서 대체 안내를 함께 만듭니다.

억제 판단 기준 (ERR-13). 이 옵션은 재조회 한 번이 아니라 이 `asyncData` 키의 최초 인스턴스에 걸립니다. `suppressNotFound · suppressNetworkError · suppressErrorToast` 는 `onResponseError / onRequestError` 클로저 변수이고, 그 핸들러는 `_handler` 에 실려 Nuxt가 같은 키의 최초 인스턴스 것만 보관합니다(`asyncData.js` 88-

91행의 `if (!existing?._init) buildAsyncData(...)`. 세 옵션은 `destructure`로 제거되어 Nuxt 키에 섞이지 않으므로, 같은 URL·query를 다른 억제 옵션으로 여는 두 인스턴스는 같은 키를 공유하고 나중 인스턴스가 준 옵션은 완전히 무시됩니다 — 범위는 "인스턴스 전체"가 아니라 "이 키의 최초 인스턴스"입니다.

같은 인스턴스(=같은 키의 최초 호출)의 최초 조회·`onActivated` 재조회·토큰 갱신 재요청 실패에도 그대로 적용되므로, `useRefreshGuard` 를 붙였다는 이유만으로 켜면 안 됩니다. 아래 두 조건을 모두 만족할 때만 켜집니다.

1. 그 인스턴스의 모든 실패 경로(최초 조회 포함)를 화면이 배너·인라인 경고로 안내한다.
2. 그 안내가 Toast처럼 사라지지 않고 상태로 남아 재시도 경로를 제공한다.

가드를 붙였지만 최초 조회 실패는 여전히 공통 Toast에 의존하는 화면이라면 **켜지 않습니다**. 가드가 붙은 지점의 Toast 중복은 이 옵션이 아니라 **재조회 구간 한정 억제**가 해결합니다 (위 `useRefreshGuard` 절 참고) — 그쪽은 최초 조회를 조용하게 만들지 않습니다. 네트워크 오류만 중복되는 경우에는 상위 옵션 대신 `suppressNetworkError` 로 좁혀 줍니다.

근접 사례(주의): `useCouncilCodes.ts` (144행 부근)는 `/api/ccodem/IT_PTL_ASCT_PRG_STS_TC` 를 명시 key 와 `suppressNotFound` 로 엽니다. `useCodeOptions.ts` (111행 부근 `useApiFetch` 호출)는 같은 URL을 key 없이(=자동 키) 열 수 있는 범용 조회입니다. 두 호출이 같은 키로 수렴하면 나중에 연 쪽의 억제 옵션은 조용히 무시되므로, 같은 조회를 다른 억제 옵션으로 여는 지점을 추가하기 전에 이 규칙을 먼저 확인하세요.

```
/* 실패를 인라인 배너로 직접 안내하므로 공통 Toast는 끈다 */
const { data, error, status } = useApiFetch<Cost[]>(url, { suppressErrorToast: true });
```

조회 상태 표현

조회 결과가 비어 있는 정상 상태와 네트워크·변환 실패를 같은 빈 배열이나 기본값으로 표현하지 않습니다.

- `composable`은 필요에 따라 `loading`, `loadError`, `retry` 를 함께 반환합니다.
- 화면은 실패 시 인라인 경고와 재시도 동작을 제공하고, 정상 빈 상태 문구는 성공 응답에서만 표시합니다.
- 이전에 성공한 본문이나 목록이 있으면 재조회 실패 시 가능한 범위에서 유지하고 경고를 덧붙입니다.
- 변환·출력 실패는 사용자가 알 수 있는 toast 또는 대체 다운로드를 제공하며 콘솔 로그만으로 끝내지 않습니다.

공통 컴포넌트 가이드

SOURCE: [it_frontend/docs/guides/components/common-components.md](#)

공통 컴포넌트 가이드

새 화면을 만들기 전에 `app/components/common` 을 검색합니다. 공통 컴포넌트는 명시적으로 import합니다.

컴포넌트	용도
<code>StyledDataTable</code>	표준 DataTable 스타일·동작
<code>ProjectListCard</code>	사업 목록형 카드
<code>ProjectListFilterBar</code>	목록 검색·필터 영역
<code>ProjectListContainer</code>	목록 화면 flex 레이아웃
<code>TocSidebar</code>	상세 화면 바로가기 목차(scrollspy 하이라이트)
<code>TableSearchInput</code>	테이블 통합검색
<code>AppDialogHeader</code>	다이얼로그 표준 제목·닫기 영역
<code>AppDialogFooter</code>	다이얼로그 표준 액션 버튼 정렬
<code>MigrationNoticeDialog</code>	수기 이관 데이터 보정 가능성 안내
<code>InlineEditCell</code>	인라인 편집 셀
<code>IoeCategorySelect</code>	비목(IOE) 계층 선택 CascadeSelect
<code>CodeTwoLineCell</code>	공통코드 조회 셀(코드값상세코드 main + 코드값명 sub)
<code>CodeMainSubLabel</code>	공통코드 선택 라벨(코드값명 main + 코드값상세코드 sub)
<code>CardSectionHeader</code>	카드 섹션 제목
<code>YearPickerTitle</code>	연도 선택과 제목
<code>TimelineNode</code>	세로 타임라인 항목
<code>DownloadButton</code>	표준 내려받기 버튼
<code>FileFormatIcon</code>	파일명 확장자에 맞는 형식 아이콘
<code>BoardAttachmentActions</code>	게시판 첨부 아이콘·목록·다운로드

컴포넌트	용도
AttachmentListSection	상세 화면의 첨부파일 목록 카드(조회·단건 내려받기)
AttachmentUploadField	품의 첨부파일 선택·삭제 예약(업로드는 호출 화면 저장)
FileDropzonePicker	파일·폴더 선택 및 드래그앤드롭
EmployeeSearchDialog	조직·직원 선택
EmployeeInfoDialog	직원 상세 정보 다이얼로그
EmployeeLink	직원명·사번 링크(직원 정보 다이얼로그 진입점)
VersionHistoryDialog	버전 이력
RevisionHistoryDialog	재상신 개정 순번 이력(관리번호+순번 업무 공통)

첨부파일

첨부파일 UI는 화면마다 새로 만들지 않고 아래 두 컴포넌트를 조합합니다. 파일 종류 (`apgF1KdNm`)와 부모 식별자(`apgF1LnkCtzNm`)는 화면(도메인 composable)이 정합니다.

상황	컴포넌트	책임
상세 화면의 목록 표시	AttachmentListSection	목록 렌더와 인증 쿠키를 포함한 단건 내려받기
등록·수정 품의 첨부 선택	AttachmentUploadField	선택·중복 제거·삭제 예약 (업로드는 호출 화면 저장)
목록 화면 행의 첨부 아이콘	BoardAttachmentActions	아이콘 배지 + 다이얼로그 + ZIP 내려받기(게시판)

- 한 화면이 출처가 다른 첨부를 함께 보여 줄 때는 `AttachmentListSection` 에 `files` 대신 `groups ( AttachmentGroup[] )`를 넘깁니다. 그룹이 둘 이상 보일 때만 그룹 이름이 표시되고, 비어 있는 그룹은 렌더링하지 않으며, 그룹별 재조회 실패 배너(ERR-13)는 `retry` 이벤트로 그룹 키를 올려 줍니다. 사업 상세가 사업 첨부와 편성요청서 반입 원본을 함께 싣는 예입니다.
- `AttachmentUploadField` 는 **업로드하지 않습니다**. 신규 등록은 저장 전에 부모 식별자가 없으므로, 선택 결과만 `v-model` 로 올려 주고 실제 업로드·삭제는 화면의 저장 흐름이 부모 식별자가 확정된 뒤 수행합니다(예: `features/project/useProjectAttachments`).
- 인증이 필요한 다운로드는 `<a href>` 가 아니라 `useAttachmentDownload()` 가 담당합니다.

다이얼로그 공통 스타일

신규·변경 다이얼로그는 먼저 기존 공통 다이얼로그를 재사용합니다. 업무 흐름상 새 다이얼로그가 필요하다면 아래 계약을 따르며 화면별 헤더·푸터·검색 입력 스타일을 새로 만들지 않습니다.

- 커스텀 제목 영역은 `AppDialogHeader`, 액션 영역은 `AppDialogFooter` 를 사용합니다.
- 목록 검색은 `TableSearchInput` 을 사용해 검색 아이콘과 입력창을 한 줄로 유지합니다.
- 직원 조회·선택은 `EmployeeSearchDialog`, 직원 상세는 `EmployeeLink` 를 사용합니다.
- 너비·색상·간격은 디자인 토큰을 사용하고 PrimeVue `Dialog` 의 `pt` 스타일은 기존 공통 다이얼로그 패턴을 따릅니다.
- `components/common` 컴포넌트는 명시적으로 import합니다.

직원 정보 다이얼로그

직원명·사번을 클릭해 상세 정보를 여는 화면은 `EmployeeLink` 만 사용합니다. 페이지마다 표시 상태·열기 함수·링크 버튼을 새로 만들거나 `EmployeeInfoDialog` 를 직접 배치하지 않습니다.

```
<EmployeeLink :eno="data.fstEnrUsid" :name="data.fstEnrUsNm" />
```

- `eno` 가 없으면 클릭할 수 없는 회색 텍스트(`name` 또는 `-`)로 렌더링합니다.
- `name` 이 없으면 `eno` 를 표시합니다.
- 셀 폭 제한·툴팁이 필요하면 `class="block truncate"`, `:title="..."` 을 그대로 넘깁니다. 링크 버튼에 전달됩니다.
- 클릭은 전파를 멈추므로 행 선택·행 클릭 동작과 겹치지 않습니다.

`EmployeeInfoDialog` 를 직접 배치하는 예외는 링크가 아닌 진입점을 쓰는 `AppHeader` (우측 상단 로그인 사용자 영역)뿐입니다.

공통코드 표기 (코드값상세코드 + 코드값명)

사업코드·비목코드처럼 코드값상세코드(계정과목코드 등)가 있는 공통코드는 화면 전체에서 같은 방식으로 표기합니다.

상황	컴포넌트	main	sub
표의 조회 셀 (코드를 먼저 읽음)	<code>CodeTwoLineCell</code>	코드값상세코드	코드값명 (아랫줄)
선택 UI (이름으로 고름)	<code>CodeMainSubLabel</code>	코드값명	코드값상세코드

- 비목(IOE) 선택은 화면마다 `CascadeSelect` 를 만들지 말고 `IoecategorySelect` 를 사용합니다. 항목은 2단, 선택값은 한 줄로 코드값상세코드를 자동 표기합니다.
- 일반 공통코드 `Select` 는 `useCodeOptions` 의 `toCodeSelectOption` 으로 옵션을 만들면 `sub` 가 채워집니다. `InlineEditCell (type="select")` 은 옵션에 `sub` 가 있으면 항목·선택값·조회 표시를 알아서 이 규칙으로 렌더링하므로 페이지에 슬롯을 작성하지 않습니다.
- PrimeVue `Select` 를 직접 쓰는 폼 필드만 `#option` · `#value` 슬롯에 `CodeMainSubLabel` 을 배치합니다(선택값은 `inline`).

전자결재 신청서 공통 컴포넌트

전자결재 상신 화면은 `components/approval` 의 아래 컴포넌트를 조합합니다.

컴포넌트	용도
<code>ApprovalFormWorkspace</code>	신청서 헤더·요약·PDF 미리보기·상태·액션
<code>ApprovalLineSelector</code>	팀장 → 부서장 표준 결재선 선택

양식 등록과 스키마 버전 규칙은 [전자결재 신청서 확장 가이드](#)를 따릅니다.

테이블의 상세 계약은 [StyledDataTable 가이드](#), 색상·간격은 [디자인 토큰](#)을 따릅니다.

디자인 토큰 가이드

SOURCE: `it_frontend/docs/guides/styling/design-tokens.md`

디자인 토큰 가이드

디자인 값의 실제 SoT는 `app/assets/css/tokens.css` 입니다. 이 문서는 토큰 선택과 변경 위치를 설명합니다.

CSS 책임

파일	책임
<code>main.css</code>	Tailwind와 폰트 진입점
<code>tokens.css</code>	<code>:root</code> , <code>html.dark</code> 디자인 변수
<code>base.css</code>	브라우저 기본 동작과 텍스트 선택 정책
<code>tags.css</code>	공통 태그 색상 클래스
<code>primevue.css</code>	PrimeVue preset 보정

사용 규칙

- 컴포넌트에서 원시 hex와 임의 픽셀 값을 새로 만들기 전에 기존 토큰을 검색합니다.
- 신규 색상·간격·반경·그림자는 `tokens.css` 에 의미 기반 이름으로 추가합니다.
- 밝은/어두운 모드에서 의미가 바뀌는 값은 두 스코프에 함께 정의합니다.
- 도메인 색상과 상태 색상을 혼용하지 않습니다.
- 페이지 컨테이너 폭과 테이블 컬럼 폭은 공통 레이아웃·컬럼 토큰을 우선합니다.
- `!important` 는 PrimeVue 내부 우선순위 보정처럼 이유가 명확한 경우에만 사용합니다.

```
.summary-card {
  color: var(--text-primary);
  background: var(--surface-card);
  border: 1px solid var(--border-subtle);
  border-radius: var(--radius-lg);
}
```

PrimeVue

- 버튼 색과 강조 단계는 Aura preset과 `primevue.css` 에서 관리합니다.
- 일반 버튼에 인라인 브랜드 색상을 추가하지 않습니다.
- 내려받기 동작은 `DownloadButton.vue` 를 사용합니다.
- PrimeVue 내부 클래스는 렌더링 결과를 확인한 후 래퍼 클래스 아래에서 제한적으로 보정합니다.

인라인 스타일

동적으로 계산되는 위치·크기처럼 CSS 변수나 클래스로 표현하기 어려운 값만 허용합니다. 고정 색상·여백·폰트 크기는 클래스 또는 토큰으로 옮깁니다.

변경 체크리스트

- 같은 의미의 토큰이 이미 있는지 확인했습니다.
- 라이트·다크 모드를 모두 확인했습니다.
- 컴포넌트 로컬 CSS보다 공통 책임 파일이 적절한지 검토했습니다.
- `npm run lint:css` 를 실행했습니다.

IT Portal 백엔드

SOURCE: `it_backend/README.md`

IT Portal 백엔드

정보화 예산, 사업, 문서 검토, 결재, 게시판, 알림과 관리자 기능을 제공하는 Spring Boot API 서버입니다.

기술 스택

- Java 25 / Spring Boot 4.1
- Gradle 9.2.1 / Spring Data JPA / QueryDSL 5.1.0 (Jakarta)
- Oracle Database
- Spring Security / JWT (JJWT 0.13.0)
- springdoc OpenAPI, Jsoup, Apache POI, Caffeine
- JUnit 5 / Mockito / AssertJ
- Spotless / JaCoCo 품질 게이트

정확한 버전은 `build.gradle` 과 Gradle lock·의존성 결과를 기준으로 확인합니다.

사전 준비

- JDK 25를 설치하고 `JAVA_HOME` 과 `java -version` 이 같은 JDK를 가리키는지 확인합니다. 별도 Gradle 설치의 필요하지 않으며 저장소의 Wrapper를 사용합니다. PATH에 다른 Gradle 설치본이 있어도 `it_backend` 에서 `gradle ...` 을 직접 실행하지 않습니다 — standalone Gradle의 `wrapper` 태스크가 `gradle-wrapper.properties` 를 자기 버전으로 덮어써 IDE 임포트가 `Can't use Java 25.0.2 and Gradle 8.9` 로 깨집니다(원복 절차는 루트 `README.md` 의 「IDE 설정(VS Code)」).
- 로컬 Oracle(127.0.0.1:11521/XEPDB1)과 접속 계정 `ITPAPP` 을 준비합니다. 객체는 `ITPOWN` 스키마에 있으며 커넥션 생성 시 `CURRENT_SCHEMA=ITPOWN` 이 적용됩니다.
- `it_backend` 와 `it_database` 를 `C:\it` 아래 형제 디렉터리로 둡니다. 로컬 프로파일의 Flyway는 `../it_database/migrations` 를 직접 읽고, Gradle `processResources` 도 같은 경로의 `v*.sql` 을 빌드 리소스에 포함합니다.
- 폐쇄망에서는 `C:\maven-repo` 에 Gradle 9.2.1 배포본과 필요한 Maven 아티팩트가 반입되어 있어야 합니다. `gradle/wrapper/gradle-wrapper.properties` 에서 온라인 `distributionUrl` 을 주석 처리하고 안내된 로컬 `file:///c:/maven-repo/gradle-9.2.1-bin.zip` 항목을 활성화합니다.

빠른 시작

```
cd C:\it\it_backend
$env:SPRING_PROFILES_ACTIVE = "local-ext"
./gradlew bootRun
```

- API: <http://localhost:28080>
- Swagger UI: <http://localhost:28080/swagger-ui/index.html>
- Health: <http://localhost:28080/actuator/health>
- Oracle: ITPAPP@127.0.0.1:11521/XEPDB1, CURRENT_SCHEMA= ITPOWN

Swagger UI와 API 명세는 `local-ext` · `local-int` · `dev` 에서만 활성화하며, `prod` 에서는 둘 다 비활성화합니다.

`local-ext` 는 외부망 개발용 프로파일로 모의 SSO, 로컬 HTTP 쿠키, Swagger의 Bearer 인증 폴백과 Flyway 자동 적용을 활성화합니다. 내부 ESSO에 연결할 수 있는 환경에서는 `local-int` 를 사용합니다. 두 로컬 프로파일은 개발 전용 DB-JWT 기본값을 제공하며, 환경변수를 지정하면 해당 값이 우선합니다.

프로파일	용도	SSO	Flyway	쿠키 / Bearer 헤더
<code>local-ext</code>	외부망 로컬 개발	모의 SSO	자동	Secure 해제 / 허용
<code>local-int</code>	내부망 로컬 개발	ESSO 실연동	자동	Secure 해제 / 허용
<code>dev</code>	개발 서버	ESSO 실연동	비활성	Secure 해제 / 허용
<code>prod</code>	운영 서버	ESSO 실연동	비활성	Secure 적용 / 허용 안 함

프로파일을 지정하지 않으면 공통 설정의 보수적 기본값이 적용됩니다. 이 경우 Flyway와 개발 사용자 전환 · Bearer 폴백은 꺼지고, `DB_PASSWORD` · `JWT_SECRET` 이 필요하며, 프론트 URL과 CORS 허용 Origin의 기본값은 비어 있습니다. 일반 로컬 개발은 프로파일 없는 기동보다 `local-ext` 또는 `local-int` 를 사용합니다.

주요 명령어

명령	용도
<code>./gradlew bootRun</code>	개발 서버 실행
<code>./gradlew test</code>	기본 단위·슬라이스 테스트
<code>./gradlew clean test</code>	전체 재검증
<code>./gradlew integrationTest</code>	로컬 Oracle 통합 테스트
<code>./gradlew spotlessCheck</code>	Java 포맷 검사
<code>./gradlew spotlessApply</code>	Java 포맷 자동 적용

명령	용도
<code>./gradlew check</code>	포맷·테스트·커버리지 게이트
<code>./gradlew build</code>	품질 게이트와 WAR 빌드
<code>./gradlew jacocoTestReport</code>	커버리지 보고서 생성
<code>./gradlew jacocoTestCoverageVerification</code>	설정된 커버리지 기준 검증

test 는 @Tag("it") 통합 테스트를 제외하고 종료 후 JaCoCo 보고서를 생성합니다. check 와 build 는 spotlessCheck 와 JaCoCo 검증까지 실행하므로 단순 테스트보다 강한 게이트이며, 현재 전체·클래스별 라인 과 클래스별 분기·복잡도 커버리지 기준은 build.gradle 의 70% 설정을 따릅니다.

Gradle Wrapper는 9.2.1을 사용합니다. 일반 의존성은 C:\maven-repo → 접속 가능한 내부 Nexus → Maven Central 순서로 탐색하며, 플러그인은 내부 Nexus → 로컬 저장소 → Gradle Plugin Portal/Maven Central 순서로 해석합니다. 폐쇄망에서는 Wrapper 배포본과 필요한 Maven 아티팩트를 C:\maven-repo 에 먼저 반입하고 gradle-wrapper.properties 의 로컬 distributionUrl 을 활성화합니다.

로컬 폴더 저장소는 **폴더 존재가 아니라 jar 보유 여부**로 등록합니다(build.gradle · settings.gradle 의 localMavenRepoUsable). 백신 정책 등으로 jar만 지워지면 POM과 .module 은 남기 때문에, Gradle이 이 저장소로 모듈을 확인한 뒤 아티팩트를 찾지 못해 실패하고 뒤 순위 저장소로 폴백하지 않습니다. jar이 하나도 없으면 경고를 남기고 저장소를 아예 등록하지 않아 Nexus-Maven Central로 자동 전환됩니다. 폐쇄망 빌드가 갑자기 외부망 저장소를 찾는다면 이 경고 로그부터 확인합니다.

프로젝트 구조

```

it_backend/
├─ src/main/java/com/kdb/it/
│  ├─ config/           Security, JPA, QueryDSL, Swagger 설정
│  ├─ common/           인증, SSO, IAM, MFA, 결재, 게시판, 코드, 다국어, 알림, 스피드다이얼, 공통 안내 팝업, 관리자 공통 기능
│  ├─ domain/           예산, 사업계획, 협의회, 사업 집행, 메뉴, 배너, 사용자가이드, 이관(편성요청서 반입·단말기 일괄 반입·공통 데이터), 감사 도메인
│  ├─ exception/        전역 예외 처리
│  └─ infra/            파일, AI, EAI 외부 연동
├─ src/main/resources/
│  └─ application*.properties  공통·프로파일별 설정
├─ src/test/java/        단위·슬라이스·Oracle 통합 테스트
├─ docs/guides/          아키텍처·보안·DB·연동 상세 가이드
├─ build.gradle           의존성·품질 게이트·Flyway 리소스 구성
└─ gradle/wrapper/        Gradle 9.2.1 Wrapper

```

기본 호출 방향은 Controller → Service → Repository → Oracle입니다. 자세한 패키지 책임은 [아키텍처 가이드](#)를 확인합니다.

아키텍처와 모듈 흐름

백엔드는 도메인별로 Controller·Service·Repository·Entity·DTO를 묶는 레이어드 구조입니다. `domain` 과 `infra` 는 인증·결재·알림 같은 `common` 기능을 사용하며, 결재 결과를 정보화사업 단계에 반영하는 검증된 흐름에서는 `common.approval` 이 `domain.budget.project.BprojasyncService` 를 호출합니다. 신규 의존은 이 예외를 일반화하지 말고 순환 참조 여부를 먼저 확인합니다.

HTTP 요청

- `SecurityFilterChain`(CORS·공개/관리자 URL 경계)
- `SimpleRequestCsrfFilter`(브라우저 단순 요청의 헤더 검증)
- `JwtAuthenticationFilter`(Access Token 쿠키 검증)
- `Controller`(DTO 변환·Bean Validation)
- `Service`(권한·업무 규칙·트랜잭션)
- `JpaRepository` 또는 `QueryDSL` 커스텀 `Repository`
- `Oracle`

Controller는 엔티티 대신 DTO로 HTTP 계약을 노출하고, 변경 요청은 `@Valid` 로 검증합니다. 응답 DTO의 `@Schema` 는 Swagger 문서용 장식이 아니라 프론트가 소비하는 계약입니다 — 프론트가 `/v3/api-docs` 에서 TypeScript 타입을 생성하므로 여기의 `requiredMode` · `nullable` · `allowableValues` 가 곧 프론트 타입이 되며, 그 계약은 `ApiResponseOpenApiContractTest` 와 도메인별 `*OpenApiContractTest` 가 고정합니다. 서비스는 JWT 인증 주체를 기준으로 역할·부서·소유권을 재검증하며, 조회와 쓰기 트랜잭션을 구분합니다. 처리 중 발생한 업무·검증 예외는 `GlobalExceptionHandler` 가 `timestamp` , `status` , `message` 를 가진 JSON 오류 응답으로 변환합니다.

목록·bulk 조회는 전체 엔티티를 메모리로 가져오거나 ID별 상세 조회를 반복하지 않습니다. 필요한 컬럼의 projection/read view, DB의 명시적 정렬과 페이지 상한, 식별자 IN 배치 조회를 사용하고, 누락 식별자는 응답 조립 중 조용히 성공 처리하지 않습니다. 이관·설정 파싱에서는 값이 없는 경우와 malformed인 경우를 구분해 진단·경고·차단으로 표면화합니다.

영역	주요 책임	연결되는 영역
<code>common.system</code> , <code>common.iam</code>	JWT 인증, Refresh Token, 로그인 이력, 사용자·조직·권한	전체 API의 인증 주체와 부서 범위를 제공
<code>common.sso</code>	ESSO 연동, SSO 상태 보관, 인증 완료 복귀	검증한 외부 인증 결과를 <code>common.system</code> 의 JWT 발급 흐름으로 전달
<code>common.approval</code>	신청서, 결재선, 승인·반려·회수	사업·협의회 상태 동기화와 알림 이벤트 발행

영역	주요 책임	연결되는 영역
<code>common.board</code> , <code>common.code</code> , <code>common.admin</code>	공통 게시판·코드와 관리자 운영 API. <code>common.admin.realtime</code> 은 감사·실시간 로그, <code>common.admin.waslog</code> 는 인메모리 링버퍼 기반 WAS 로그 조회·런타임 레벨 변경	파일·메뉴·사용자·감사로그 등 공통 관리 기능을 조합
<code>common.mfa</code>	추가 인증 거래 발급·검증·소비와 공유 저장소	수동 로그인과 전자결재 명령의 증표를 <code>common.system</code> · <code>common.approval</code> 에 제공
<code>common.i18n</code>	메뉴명·공통코드 표시명 번역과 변경 이력	메뉴·코드 조회 응답의 표시명을 언어별로 제공
<code>common.speeddiary1</code>	전역 스피드다이얼의 FAQ 조회·Q&A 등록·담당자 정보 조회	전용 테이블 없이 <code>common.board</code> 의 FAQ(004)·Q&A(005) 유형 게시판과 <code>BGDOCM</code> 단일 문서를 재사용
<code>common.popup</code>	공통 안내 팝업 게시·중지와 활성 팝업 조회	전용 테이블 없이 <code>BGDOCM</code> 의 <code>DOC_TTL_CONE='common.popup'</code> 단일 문서를 재사용
<code>common.notification</code>	인앱 알림 저장, 소유권 검증, 채널 라우팅	결재·게시판 이벤트와 <code>infra.eai</code> 연결
<code>domain.budget</code>	정보화사업·경상사업, 전산업무비, SNO 재상신 이력, 계획, 문서 검토, 예산 현황·작업. <code>budget.document.formguide</code> 는 사업 입력 길라잡이를 서버 고정 카탈로그 기준으로 등록·조회	협의회와 사업 집행의 기준 사업 데이터를 제공
<code>domain.bizplan</code>	정보기술부문 계획에 포함된 사업의 사업계획	<code>budget.plan</code> , <code>budget.project</code> 의 계획 관계·사업·품목·단계 상태 사용
<code>domain.council</code>	정보화실무협의회 일정·평가·질의·결과	결재 완료 이벤트를 같은 트랜잭션에서 상태에 반영
<code>domain.estimate</code> , <code>domain.deliberation</code> , <code>domain.contract</code> , <code>domain.payment</code>	사업 집행의 소요예산·심의·계약·지급 단계	정보화사업을 기준으로 단계별 문서와 상태를 관리
<code>domain.menu</code>	사용자 메뉴 조회와 관리자 메뉴·라우트 관리	인증 주체의 권한에 맞는 프론트 메뉴 구성을 제공

영역	주요 책임	연결되는 영역
domain.banner	/info 홈 배너 등록·노출·활성 전환	전용 테이블 없이 infra.file의 공통첨부파일을 규약 (APG_FL_KD_NM='배너')으로 재사용
domain.log	업무 엔티티 변경 스냅샷	@LogTarget 이 지정된 엔티티의 생성·수정·논리삭제를 기록
domain.migration	수기 엑셀(편성요청서) 반입 — 검증·진단, 원장 생성, 결재완료 표식, 원본 파일 보관. migration.terminal 은 금융정보단말기 행을 연도별 전산업무비 원장으로 일괄 반입하고, migration.commondata 는 메뉴·메뉴권한·경로·공통코드·다국어 5개 테이블을 개발→운영으로 내보내기·dry-run·확정 반영	budget 의 원장 (BPROJM·BCOSTM), common.approval 신청서, infra.file 첨부, domain.menu·common.code·common.i18n 에 연결
domain.userguide	전사 공개 사용자가이드 업로드·내려받기·이력 되돌리기	전용 테이블 없이 infra.file의 공통첨부파일을 규약 (APG_FL_KD_NM='사용자가이드')으로 재사용하고 활성 파일을 항상 0건 또는 1건으로 유지
infra.file, infra.eai, infra.ai	파일 저장, 표준전문 외부 전송, Gemini 연동. infra.file.authz 는 첨부파일 종류별 읽기·쓰기 판정기를 등록해 부모 자원 권한으로 접근을 판정	공통·도메인 서비스가 외부 자원을 사용할 때 호출

예산 재상신 버전 흐름

정보화사업과 경상사업은 같은 BPROJM 원장을 사용하며 (ABUS_MNG_NO, SNO) 로 개정본을 구분합니다.

ODN_YN='Y' 는 경상사업, NULL 또는 N 은 정보화사업입니다. 전산업무비는 BCOSTM 의 (BG_NO, BG_SNO) 를 사용합니다.

1. 대상 주관부서 사용자(전산업무비는 담당부서), 지정된 IT 조직 사용자 또는 시스템관리자가 결재완료 된 최종본에 POST /api/projects/{관리번호}/reapplications 또는 POST /api/cost/{관리번호}/reapplications 를 호출합니다.
2. 서버는 같은 관리번호의 최대 순번에 1을 더해 LST_YN='N' 초안을 만들고 하위 품목·단말기 관계도 새 부모 순번으로 복제합니다.
3. 이력은 .../{관리번호}/history 로 조회합니다. 정보화사업·경상사업의 특정 개정본은 /api/projects/{관리번호}/versions/{sno}, 전산업무비는 /api/cost/{관리번호}?sno={sno} 로 조회하며 수정·삭제·결재 신청에도 같은 순번을 전달합니다.
4. 결재 완료 이벤트는 신청서가 가리키는 정확한 순번만 LST_YN='Y' 로 전환합니다. 후속 업무 목록·집계·bulk 조회는 승인 전 초안이나 과거본을 제외하고 최종본만 반환하며, apfSts=none 미상신 작성 목록만 재상신 초안을 포함해 순번과 함께 반환합니다.

상세·이력·재상신은 대상 주관부서(전산업무비는 담당부서), 지정된 IT 조직 또는 시스템관리자 범위를 서비스 계층에서 검증합니다. BPLANM 은 별도 SNO 재상신 원장이 아니며 정보기술부문 계획 자체를 관리합니다.

사업계획(bizplan) 흐름

사업계획 API는 `/api/project/bizplans` 아래에 있으며, 정보기술부문 계획 관계인 BPLANA 에 포함된 최신 유효 사업만 대상으로 합니다. 관리자는 전체 목록을 보고 일반 사용자는 JWT의 부서 코드와 사업 주관부서가 일치하는 항목만 조회·저장·완료할 수 있습니다.

1. 최초 진입 `POST /{abusMngNo}` 은 BBIZPM 을 먹등하게 생성하고 BPROJA 의 BIZ-{사업관리번호} 단계 상태를 작성중 21 로 맞춥니다.
2. 최초 생성 시 사업명과 예산번호를 기존 사업 데이터에서 연결하고, 저장 이력이 없는 품목과 일정은 BITEMM, BPROJM 에서 초기화합니다.
3. 전체 저장 `PUT /{abusMngNo}` 은 보고서·전결권과 일정(BBIZSM)·품목(BBIZGM)·계약(BBIZCM)을 일련번호 기준으로 병합합니다. 요청에서 빠진 활성 자식 행은 물리 삭제하지 않고 논리 삭제합니다.
4. 완료 `POST /{abusMngNo}/status` 는 작성중 21 에서 작성완료 29 로만 전이합니다. 완료 후에도 내용 저장은 허용하며 완료 상태는 유지합니다.

사업계획의 상태를 마스터 BBIZPM 에 중복 저장하지 않고 공통 사업 단계 테이블 BPROJA 에서 관리하는 것이 핵심 설계입니다. 목록 조회는 BPLANA 를 기준으로 사업·계획·상태·조직을 QueryDSL로 조합하므로, 사업계획이 아직 생성되지 않은 대상도 함께 반환합니다.

인증과 비동기 부수효과

- 로그인과 토큰 갱신을 제외한 API는 기본적으로 인증이 필요합니다. `JwtAuthenticationFilter` 가 Access Token 쿠키를 검증하고 `CustomUserDetails` 를 보안 컨텍스트에 넣습니다.
- Access Token과 Refresh Token은 httpOnly 쿠키로 전달하며 서버 세션은 만들지 않습니다. 관리자 API는 URL 규칙과 `@PreAuthorize` 를 함께 사용하고, 일반 업무 API는 서비스에서 소유자·부서 범위를 추가로 검증합니다.
- 업무 API는 Stateless JWT를 사용하지만 SSO 진행 상태와 인증 완료 사변은 콜백 왕복을 위해 `HttpSession` 에 보관합니다. 이 세션은 업무 인증 토큰을 대신하지 않습니다.
- 결재 상태처럼 원 업무와 반드시 함께 반영되어야 하는 변경은 동기 `@EventListener` 로 같은 트랜잭션에서 처리합니다.
- 알림은 `@TransactionalEventListener(AFTER_COMMIT)` 에서 처리하고 저장이 필요하면 `REQUIRES_NEW` 트랜잭션을 사용합니다. 따라서 알림 실패가 이미 성공한 원 업무를 롤백하지 않습니다.
- `AFTER_COMMIT` 알림 리스너 자체는 별도 비동기 스레드가 아니라 커밋 후 콜백으로 실행되며, 실패한 알림은 60초 주기 재시도 작업이 건당 최대 5회까지 처리합니다.
- 알림 채널 라우터는 기본 인앱 채널과 EAI 그룹웨어 채널을 구분합니다. 외부 전송은 `EaiResult` 의 성공·스킵·실패로 표현하며 예외나 실패를 원 알림 저장 흐름으로 전파하지 않습니다.

데이터 설계 결정

- 접속 계정 `ITPAPP` 과 객체 소유 스키마 `ITPOWN` 을 분리하고, 커넥션 생성 시 `CURRENT_SCHEMA` 를 설정합니다. 엔티티와 쿼리에는 스키마 접두어를 하드코딩하지 않습니다.
- 데이터베이스 문자셋은 `AL32UTF8` 이고 `VARCHAR2` 는 `BYTE semantics`입니다. `@Size` 의 글자 수만으로는 저장 한도를 보장할 수 없으므로 신규·변경되는 DB 문자열 입력은 공통 `Utf8ByteLimit` 으로 UTF-8 실재 바이트 수를 검증하고, 물리 DDL은 `VARCHAR2(n BYTE)` 를 명시합니다.
- 업무 엔티티는 `BaseEntity` 의 논리삭제, GUID, 등록·변경 감사 필드를 공유합니다. 복합키 테이블은 `@IdClass` 로 기존 Oracle 물리 모델을 매핑하며, 물리 PK의 모든 컬럼을 `@Id` 로 매핑합니다. 일부만 매핑하면 서로 다른 행이 같은 JPA 식별자를 갖게 되므로, 이 정합은 `PhysicalCompositeIdMappingTest` 와 `PhysicalCompositeIdIsolationIt` 가 고정합니다.
- 응답 직렬화에만 쓰이는 조회는 엔티티 대신 필요한 컬럼만 담은 프로젝션(`*Row record` 또는 `*View 인터페이스`)으로 읽습니다. 쓰기 엔티티와 DDL은 그대로 두고 읽기 경로만 좁히는 방식이며, 엔티티 조회와의 결과·정렬·null 동등성은 `*ProjectionIt` Oracle 통합 테스트가 확인합니다.
- `@LogTarget` 엔티티는 대응하는 `BaseLogEntity` 하위 로그 엔티티에 생성·수정·논리삭제 스냅샷을 남깁니다.
- 감사 로그는 별도 `REQUIRES_NEW` 트랜잭션에서 `flush()` 해 기록하므로 감사 기록 실패가 원 업무 트랜잭션을 롤백시키지 않습니다.
- 단순 CRUD는 `JpaRepository` 를 사용하고 동적 검색·집계·다중 조인은 `*RepositoryCustom` 과 `*RepositoryImpl` 의 `QueryDSL` 구현으로 분리합니다.
- 공통코드, 메뉴 권한, 알림 미읽음 수, Tiptap 메타데이터는 Caffeine 캐시를 사용합니다. 캐시 쓰기는 트랜잭션 완료와 연동하고, 원본 변경 서비스가 `@CacheEvict` 로 즉시 무효화하며 TTL은 누락에 대한 안전망으로 사용합니다.
- 캐시 TTL은 공통코드·예산기간·메뉴권한 1시간, Tiptap 메타데이터 10분, 알림 미읽음 수 60초이며 모두 프로세스 로컬 Caffeine 캐시입니다.
- 정보화사업 금액은 화면·조회마다 다시 더하지 않고 활성 품목과 지급금액으로 한 번 계산해 사업 스냅샷에 기록합니다. 저장 단위 반올림과 `NUMBER(18,3)` 범위 검증을 한 곳에 모아, 외화 환산이 끼어드는 경로에서도 컬럼별 통화 의미가 갈리지 않게 합니다. 계약은 [사업 집행 가이드](#)와 [데이터 모델 인덱스](#)가 SoT입니다.
- 물리 스키마 변경의 기준은 `C:\it\it_database\migrations` 이며, 엔티티 매핑과 마이그레이션을 함께 검토합니다. 상세 매핑은 [데이터 모델 인덱스](#)를 확인합니다.

환경 설정

주요 운영 값은 환경변수로 주입합니다.

환경변수	용도
<code>DB_URL</code>	Oracle JDBC URL. 공통 로컬 기본값은 <code>127.0.0.1:11521/XEPDB1</code> . RAC 2노드 운영에서는 인스턴스마다 다르게 주입합니다 — 인스턴스별 환경변수 참조
<code>DB_USERNAME</code>	DB 접속 계정. 기본값은 <code>ITPAPP</code>

환경변수	용도
DB_PASSWORD	애플리케이션 DB 비밀번호
DB_SCHEMA	객체 소유 스키마. 기본값은 ITPOWN
JWT_SECRET	JWT 서명 키
TOKEN_FINGERPRINT_SECRET	Refresh Token·MFA 지문 HMAC 키. JWT_SECRET 과 분리하며 UTF-8 32바이트 이상이어야 함
APP_FRONTEND_URL	SSO 기본 복귀 URL과 기본 CORS Origin
CORS_ALLOWED_ORIGINS	다중 CORS Origin이 필요할 때 콤마 구분으로 별도 지정
APP_TRUSTED_PROXIES	X-Forwarded-For 를 신뢰할 프록시 IP 목록
SSO_BROWSER_BASE_URL	브라우저가 접근하는 ESSO 기준 URL
SSO_HOST_BASE_URL	백엔드가 접근하는 ESSO 기준 URL
SSO_AGENT_ID	ESSO가 발급한 업무 시스템 식별 번호
FILE_BASE_PATH	첨부파일 저장 경로
SERVER_INSTANCE_ID	인스턴스 ID. 멀티 서버 파일명 충돌 방지와 WAS 로그 인스턴스 식별에 함께 사용하므로 서버마다 서로 달라야 합니다
WAS_LOG_INTERNAL_SECRET	WAS 로그 피어 내부 API(/internal/was-logs/**) 공유 비밀번호. 비어 있으면 내부 컨트롤러 자체가 등록되지 않습니다
WAS_LOG_PEER_SVR1 · WAS_LOG_PEER_SVR2	인스턴스별 내부 호출 base URL. 지정하지 않으면 해당 인스턴스를 조회 대상에서 뺍니다
GEMINI_API_KEY	Gemini API 키. prod 에서는 기동 시 필수 검증
EAI_ENABLED	EAI 전송 활성화 여부. 공통 기본값 false , prod 기본값 true
EAI_URL	EAI 전송 URL. prod 에서 EAI가 활성화되면 기동 시 필수 검증
EAI_GWE_IF_ID	그룹웨어 EAI 인터페이스 ID
MFA_SITE_ID	OnePass 기관 식별자. 기본값 SIT01KDBBANK00000000
MFA_SVC_ID	OnePass 서비스 식별자. 기본값 SVC12SIT01KDBBANK000
MFA_FINGER_VEIN_FIXED_KEY	지정맥 해시 검증용 고정키. 모의 공급자를 끈 프로파일에서 필수
MFA_FIDO_REJECTED_STATUSES	OnePass 규격에서 사용자 거부로 확정된 FIDO 상태값. 쉼표로 구분하며 기본값은 비어 있음
JAVA_HOME	JDK25 설치 경로(C:\Program Files\Java\jdk-25.0.2)

운영에서는 개발·로컬 프로파일의 기본값을 사용하지 않습니다. `EnvironmentValidator` 는 모든 프로파일에서 DB 비밀번호와 JWT 시크릿의 빈값을 차단하고, `prod`에서는 Gemini 키, 활성 EAI URL, 프론트 URL, 명시적 CORS Origin과 운영 보안 토글을 추가로 검증합니다.

인스턴스별 환경변수 (RAC 2노드)

WAS를 2대로 운영할 때 인스턴스마다 값이 달라야 하는 환경변수는 두 개뿐입니다. 나머지는 두 대에 똑같이 주입합니다.

환경변수	SVR1	SVR2	다르게 주는 이유
<code>SERVER_INSTANCE_ID</code>	<code>SVR1</code>	<code>SVR2</code>	첨부파일명 충돌 방지와 WAS 로그 인스턴스 식별. 두 대가 같으면 피어 조회가 언제나 자기 링버퍼만 읽습니다
<code>DB_URL</code>	1번 RAC 노드 우선	2번 RAC 노드 우선	두 노드에 접속을 나눠 붙입니다. 값 형태는 아래 표 참조

`WAS_LOG_PEER_SVR1` · `WAS_LOG_PEER_SVR2` 는 이름과 달리 두 대에 같은 값을 넣습니다. 각 서버가 동일한 인스턴스 목록을 갖고, 자기 자신이 아닌 대상만 내부 HTTP로 위임하기 때문입니다.

`DB_URL` 은 두 DB 노드 주소를 `ADDRESS_LIST` 에 넣고 **AP마다 순서를 교차**해 주입합니다. 두 URL은 `ADDRESS` 순서만 다르고 나머지는 완전히 같습니다. 선택 기준과 주의사항은 [application-prod.properties](#) 의 `spring.datasource.url` 주석이 SoT입니다.

	첫 번째 ADDRESS (정상시)	두 번째 ADDRESS (장애 시)
AP1	1번 노드	2번 노드
AP2	2번 노드	1번 노드

`LOAD_BALANCE=OFF` 가 앞 주소 고정, `FAILOVER=ON` 이 뒤 주소로의 자동 전환을 담당하므로 **둘 다 명시해야 합니다**. 빠뜨리면 Oracle이 두 주소 중 하나를 무작위로 골라 교차 배분 자체가 무너집니다. 전체 URL은 줄바꿈·공백 없이 한 줄로 주입합니다.

```
AP1: jdbc:oracle:thin:@(DESCRIPTION=(CONNECT_TIMEOUT=5)(RETRY_COUNT=3)(RETRY_DELAY=1)(ADDRESS_LIST=(LOAD_BALANCE=OFF)(FAILOVER=ON)(ADDRESS=(PROTOCOL=TCP)(HOST=db1-ip)(PORT=11521))(ADDRESS=(PROTOCOL=TCP)(HOST=db2-ip)(PORT=11521)))(CONNECT_DATA=(SERVICE_NAME=PPRMDB)))
AP2: jdbc:oracle:thin:@(DESCRIPTION=(CONNECT_TIMEOUT=5)(RETRY_COUNT=3)(RETRY_DELAY=1)(ADDRESS_LIST=(LOAD_BALANCE=OFF)(FAILOVER=ON)(ADDRESS=(PROTOCOL=TCP)(HOST=db2-ip)(PORT=11521))(ADDRESS=(PROTOCOL=TCP)(HOST=db1-ip)(PORT=11521)))(CONNECT_DATA=(SERVICE_NAME=PPRMDB)))
```

받은 주소가 노드 VIP인지 물리 IP인지 확인합니다. VIP는 노드가 죽으면 살아있는 노드가 넘겨받아 즉시 거절 응답을 주므로 두 번째 주소로 곧바로 넘어갑니다. 물리 IP는 아무 응답이 없어 `CONNECT_TIMEOUT` (위 예시 5초)을 다 기다린 뒤에야 전환하므로, 그 시간만큼 로그인·조회가 멈춥니다.

주소를 교차했다고 노드가 반드시 고정되지는 않습니다. RAC의 각 노드 리스너는 그 서비스가 떠 있는 모든 인스턴스를 알고 있어서, AP1이 1번 노드 주소로 접속해도 리스너가 서버 사이드 로드밸런싱으로 2번 인스턴스에 넘길 수 있습니다(`LOAD_BALANCE=OFF`는 클라이언트 쪽 주소 선택만 끕니다). 배포 후 아래로 실제 분포를 확인하고, 의도대로 갈리지 않으면 DBA와 서비스의 `CLB_GOAL` 조정 또는 노드 선호 서비스 전환을 협의합니다.

```
SELECT inst_id, COUNT(*) FROM gv$session WHERE username = 'ITPAPP' GROUP BY inst_id;
```

AP 2대 · RAC 2노드의 최종 주입 예시입니다. 위쪽 두 블록만 서버마다 다르고, 아래 공통 블록은 **한 글자라도 다르면 안 됩니다**.

```
# — AP1 전용 —————
SERVER_INSTANCE_ID=SVR1
DB_URL=jdbc:oracle:thin:@(DESCRIPTION=(CONNECT_TIMEOUT=5)(RETRY_COUNT=3)(RETRY_DELAY=1)
(ADDRESS_LIST=(LOAD_BALANCE=OFF)(FAILOVER=ON)(ADDRESS=(PROTOCOL=TCP)(HOST=db1-ip)(PORT=11521))
(ADDRESS=(PROTOCOL=TCP)(HOST=db2-ip)(PORT=11521)))(CONNECT_DATA=(SERVICE_NAME=PPRMDDB)))

# — AP2 전용 —————
SERVER_INSTANCE_ID=SVR2
DB_URL=jdbc:oracle:thin:@(DESCRIPTION=(CONNECT_TIMEOUT=5)(RETRY_COUNT=3)(RETRY_DELAY=1)
(ADDRESS_LIST=(LOAD_BALANCE=OFF)(FAILOVER=ON)(ADDRESS=(PROTOCOL=TCP)(HOST=db2-ip)(PORT=11521))
(ADDRESS=(PROTOCOL=TCP)(HOST=db1-ip)(PORT=11521)))(CONNECT_DATA=(SERVICE_NAME=PPRMDDB)))

# — 두 대 공통 —————
DB_USERNAME=ITPAPP
DB_SCHEMA=ITPOWN
DB_PASSWORD=...
JWT_SECRET=... # 두 대가 다른 AP1이 발급한 토큰을 AP2가 거부합니다
TOKEN_FINGERPRINT_SECRET=... # 위와 같은 이유로 반드시 동일
FILE_BASE_PATH=/dat/springitp # 두 대가 같은 공유 스토리지를 봐야 합니다
WAS_LOG_INTERNAL_SECRET=...
WAS_LOG_PEER_SVR1=https://ap1-host:28080
WAS_LOG_PEER_SVR2=https://ap2-host:28080
```

알아 둘 점:

- **데이터는 한 벌입니다.** RAC 두 노드는 같은 DB를 보므로 어느 노드에 붙든 MFA·로그인대기 공유 테이블(`app.mfa.store=jpa`)과 감사 로그가 그대로 동작합니다. 노드를 나눠 붙이는 것은 다중 인스턴스 정합성과 무관합니다.
- **한 노드가 죽으면 AP 2대가 모두 살아남은 노드로 몰립니다.** 정상 시 노드당 커넥션은 AP 1대분이지만 장애 시에는 2대분이 됩니다. Hikari `maximum-pool-size`는 현재 지정하지 않아 기본값 10이므로 최대 20 세션이 한 노드에 몰리며, 노드의 `sessions·processes` 여유가 이 값을 감당해야 합니다. 풀 크

기를 올릴 때는 정상치가 아니라 이 장애 시 합계로 계산합니다.

- **Hikari는 FAN(ONS)을 구독하지 않습니다.** 노드 장애를 통보로 알지 못하므로 죽은 커넥션은 획득.검증 시점에야 걸러지고, 복구된 노드로의 원복도 `spring.datasource.hikari.max-lifetime` (기본 30분)에 맞춰 커넥션이 재생성되면서 서서히 일어납니다. 전환을 빠르게 하려면 이 값을 줄입니다.
- **쓰기를 두 노드에 흠뻑리지 않습니다.** 같은 테이블을 두 노드에서 동시에 갱신하면 cache fusion(`gc buffer busy`)으로 오히려 느려집니다. 인스턴스 단위 고정은 안전하지만, 요청·트랜잭션마다 노드를 바꾸는 `AbstractRoutingDataSource` 라운드로빈은 이 프로젝트에 넣지 않습니다.
- **이름 세 가지를 혼동하지 않습니다.** 접속 계정은 `ITPAPP`, 객체 소유 스키마는 `ITPOWN`, DB 서비스명은 `PPRMDB` 입니다. URL 끝(또는 `SERVICE_NAME`)에 오는 것은 **서비스명**이고, 스키마는 `DB_SCHEMA` 가 커넥션 초기화 SQL(`CURRENT_SCHEMA`)로 전환합니다.
- **다른 선택지도 있었습니다.** DBA가 노드 선호 서비스(`srvctl add service -preferred/-available`)를 등록해 주면 AP는 서비스명만 다르게 쓰고 노드 배치를 DB 쪽에서 옮길 수 있고, 노드 고정이 필요 없으면 SCAN 주소 하나로 끝낼 수도 있습니다. 이 환경은 AP에 DB 주소를 교차 주입하는 방식으로 정해졌으므로 위 구성을 따릅니다.

내부망 MFA (지정맥·FIDO·mOTP)

수동 로그인과 사용자 전자결재 명령은 MFA를 통과해야 진행됩니다. 설정은 `app.mfa.*` 구성 속성으로 묶여 있습니다.

속성	기본 값	설명
<code>app.mfa.endpoint</code>	프로파일 별	OnePass 연동 URL. 공통 기본값은 비어 있고 프로파일에서 지정합니다
<code>app.mfa.site-id</code>	<code>SIT01KDBBANK0000000</code>	<code>MFA_SITE_ID</code> 로 재정의
<code>app.mfa.svc-id</code>	<code>SVC12SIT01KDBBANK000</code>	<code>MFA_SVC_ID</code> 로 재정의
<code>app.mfa.connect-timeout</code>	<code>5s</code>	OnePass 연결 타임아웃

속성	기본 값	설명
<code>app.mfa.read-timeout</code>	5s	OnePass 응답 타임아웃
<code>app.mfa.challenge-ttl</code>	90s	MFA 거래 유효 시간
<code>app.mfa.max-failures</code>	5	한 거래의 허용 실패 횟수. 초과 시 <code>MFA_LOCKED</code>
<code>app.mfa.mock-enabled</code>	false	모의 공급자 사용 여부. <code>local-ext</code> 에서만 <code>true</code> 를 허용
<code>app.mfa.fido-rejected-statuses</code>	빈 집합	사용자 거부로 확정된 <code>trStatus</code> 허용 목록. <code>MFA_FIDO_REJECTED_STATUSES</code> 로 재정의
<code>app.mfa.store</code>	jpa	<code>jpa</code> (기본)는 MFA·로그인대기 거래를 Oracle(<code>TPRMPP_CMFATM</code> / <code>TPRMPP_CMFADM</code>)에 저장해 다중 인스턴스를 지원합니다. <code>memory</code> 는 인스턴스 내부에만 보관하며 단일 인스턴스 전용이고 재시작 시 유실됩니다. <code>spring.jpa.hibernate.ddl-auto=none</code> 이라 기동 시 스키마를 검증하지 않으므로, <code>app.mfa.store=jpa</code> (기본값)로 기동하기 전에 반드시 마이그레이션 <code>V20260820_002__CreateMfaTransactionTables.sql</code> 을 적용해야 합니다. 적용하지 않으면 기동은 정상적으로 끝나고 최초 로그인 시도에서야 Oracle "table or view does not exist" 오류로 실패합니다
<code>app.mfa.cleanup-fixed-delay-ms</code>	300000	만료 후 10분 유예가 지난 MFA·로그인대기 행을 Oracle에서 물리 삭제하는 배치 주기(ms). 순수한 용량 관리 설정이며 정합성에는 영향이 없습니다(모든 조화쓰기 경로가 만료 행을 이미 자체적으로 제외합니다)

프로파일별 동작:

프로파일	MFA 동작	OnePass URL
local-ext	대화상자 확인 시 모의 성공	호출하지 않음
local-int	실제 연동	https://dopsap.kdb.co.kr:20443/interfBiz/processRequest.do
dev	실제 연동	https://dopsap.kdb.co.kr:20443/interfBiz/processRequest.do
prod	실제 연동	https://opsap.kdb.co.kr:20443/interfBiz/processRequest.do

MfaConfig 는 local-ext 외의 프로파일에서 app.mfa.mock-enabled=true 이면 기동을 실패시키고, 모의 공급자를 끈 상태에서 endpoint가 비어 있어도 기동을 실패시킵니다. 운영에서 MFA를 우회하는 설정 경로는 없습니다.

운영 전제와 한계:

- **BioAgent 전제** — 지정맥은 사용자 PC에서 BioAgent가 실행 중이어야 합니다. 브라우저가 ws://127.0.0.1:8089/bio 로 연결하며, 미실행 시 화면이 실행 안내를 표시합니다. 서버는 에이전트 설치 여부를 알 수 없습니다.
- **지정맥 해시 검증** — mfa.md 의 「지정맥인증 연계 보안방안」대로 서버가 결과를 독립 검증합니다. FingerVeinMfaProvider 가 거래마다 6자리 랜덤키를 발급해 challenge 응답에 신고, 화면이 그 키로 BioAgent를 호출하면 에이전트가 년월일 + 사번 + 랜덤키 + 검증값(SUCCESS|FAIL) + 고정키를 SHA-256으로 3회 해시한 값을 돌려줍니다. 서버는 성공 검증값으로 같은 해시를 만들어 상수 시간 비교하므로 클라이언트가 보낸 결과 코드(FE00)를 신뢰하지 않습니다. 랜덤키는 검증 성공/실패와 무관하게 1회 사용 후 폐기해 재전송을 막습니다.
- **지정맥 고정키** — MFA_FINGER_VEIN_FIXED_KEY 환경변수로 주입합니다. 비밀값이라 프로파일 파일에 기본값을 두지 않으며, 모의 공급자를 끈 프로파일에서 값이 비어 있으면 MfaConfig 가 기동을 실패시킵니다.
- **년월일 기준** — 해시의 년월일은 서버 시계(clock bin)를 씁니다. 자정 경계에 에이전트와 서버의 날짜가 갈리면 검증이 실패할 수 있습니다.
- **FIDO 재조회** — 사용자가 휴대폰에서 승인할 때까지 화면이 결과를 반복 조회합니다. trStatus=1 은 성공이고, app.mfa.fido-rejected-statuses 에 등록된 값은 즉시 실패로 분리합니다. 그 밖의 값은 미결정(UNDECIDED)으로 보아 실패 횟수에 집계하지 않습니다. 현재 반입 규격에는 사용자 거부 상태 값이 없으므로 기본 목록은 비어 있으며, 공급자가 값을 확정된 뒤에만 운영 환경변수로 등록합니다.
- **다중 인스턴스와 재시작** — 기본값 app.mfa.store=jpa 에서 MFA 거래와 로그인대기 거래는 Oracle 공유 테이블에 있으므로 인스턴스가 여러 대여도, 인스턴스 하나가 재시작해도 진행 중 거래가 유지됩니다. 상태 전이는 전부 조건부 UPDATE의 영향 행 수로 판정하므로 두 인스턴스가 같은 거래를 동시에 다뤄도 증표가 두 번 소비되지 않습니다. app.mfa.store=memory 로 바꾼 경우에만 거래가 인스턴스 메모리에 남아 재시작 시 폐기되고 사용자가 MFA를 다시 수행합니다.

실시간 WAS 로그 (/admin/was-logs)

관리자가 서버 재기동이나 파일 접근 없이 애플리케이션 로그를 보고 로그 레벨을 임시로 올릴 수 있는 기능입니다. 설정은 app.was-log.* 로 묶여 있습니다.

속성	기본값	설명
<code>app.was-log.buffer-capacity</code>	2000	인메모리 링버퍼 용량(줄). <code>logback-spring.xml</code> 이 <code><springProperty></code> 로 같은 값을 읽어 appender에 주입하므로 이 프로퍼티가 단일 출처입니다
<code>app.was-log.peers.{인스턴스ID}</code>	비어 있음	인스턴스ID → 내부 호출 base URL. <code>WAS_LOG_PEER_SVR1</code> · <code>WAS_LOG_PEER_SVR2</code> 로 주입합니다
<code>app.was-log.internal-secret</code>	비어 있음	피어 내부 API 공유 비밀값
<code>app.was-log.connect-timeout-ms</code> · <code>read-timeout-ms</code>	1000 · 3000	피어 호출 타임아웃(ms)
<code>app.was-log.restore-scan-ms</code>	30000	만료된 로그레벨 오버라이드를 되돌리는 스캔 주기(ms)

설계상 알아 둘 점:

- **저장은 프로세스 메모리뿐입니다.** `RingBufferAppender` 가 링버퍼에 적재하므로 재기동 이전 로그는 조회할 수 없고, 용량을 넘으면 오래된 줄부터 버립니다. 버려진 줄이 있으면 응답이 `dropped` 로 알려 화면이 배너로 표면화합니다. 장기 보관이 필요한 로그는 기존 파일 appender가 계속 담당합니다.
- **다중 인스턴스는 피어 팜아웃으로 처리합니다.** 자기 인스턴스가 아닌 대상을 조회하면 `/internal/was-logs/**` 로 위임합니다. 이 경로는 `SecurityConfig` 에서 `permitAll` 이고 공유 비밀 헤더 `X-Internal-Token` 이 유일한 관문이므로, 비밀값이 비면 컨트롤러 빈 자체를 등록하지 않아 인증 없는 로그 엔드포인트가 열리는 경로를 구조적으로 없앱니다. 운영 배포 시 방화벽에서 이 경로를 사내 서버 대역으로 제한하고 피어 URL은 HTTPS를 사용합니다.
- **로그레벨 변경은 항상 TTL을 가집니다.** Actuator 엔드포인트를 열지 않고 `LoggingSystem` 빈만 사용하며, 최대 120분·동시 50건 상한과 `com.kdb.it.org.springframework.org.hibernate` 접두사 화이트리스트를 적용합니다. 루트 로거 전체 변경은 허용하지 않습니다. 만료되면 스케줄러가 직전 레벨로 되돌리고 재기동 시에는 설정 파일 레벨로 자연 복원됩니다.
- **조회·레벨변경·다운로드**는 모두 관리자 감사 로그를 남깁니다. 로그 본문을 마스킹하지 않는 대신 누가 언제 무엇을 봤는지 남기는 것이 보상 통제이며, 내부 API는 공유 비밀 불일치로 거부한 시도도 원격 주소와 함께 기록합니다.

내부망 IP로 접속할 때 (CORS)

`cors.allowed-origins` 는 명시한 **Origin**만 허용합니다(와일드카드 기본값 없음). `local-int` 기본값은 `http://localhost` , `http://localhost:3000` , `http://localhost:3002` 뿐이므로, 프론트를 `http://<내부IP>:3000` 처럼 IP로 접속하면 모든 `/api/**` 요청이 CORS 단계에서 차단됩니다.

```
# 백엔드 기동 전 (IP 접속 + localhost 병행 허용)
setx APP_FRONTEND_URL "http://10.9.16.109:3000"
setx CORS_ALLOWED_ORIGINS "http://10.9.16.109:3000,http://localhost:3000"
```

- `APP_FRONTEND_URL` 은 SSO 복귀 대상과 기본 CORS Origin을 함께 정합니다. 여러 Origin이 필요할 때만 `CORS_ALLOWED_ORIGINS` 로 CORS만 따로 지정합니다.
- 프론트도 같은 호스트를 보도록 `NEXT_PUBLIC_API_BASE=http://<내부IP>:28080` 을 맞춰야 합니다. 쿠키는 포트가 아니라 호스트 기준이라 페이지와 API 호스트가 다르면 인증 쿠키가 실리지 않습니다.
- `setx` 는 새 프로세스부터 적용되므로 IDE-터미널을 재시작해야 합니다.

확인 방법 — 허용 Origin이면 401 과 함께 `Access-Control-Allow-Origin` 이 오고, 비허용이면 헤더 없는 403 이 옵니다.

```
curl -i -H "Origin: http://10.9.16.109:3000" http://127.0.0.1:28080/api/menus
```

리버스 프록시(same-origin) 배포 시

프론트 정적 빌드(`npm run generate`)를 nginx-WebTobe로 서빙하고 `/api/` · `/sso/` 를 백엔드로 프록시하는 구조에서는 API 호출이 same-origin이라 CORS가 발생하지 않습니다. 다만 두 가지는 여전히 백엔드 설정에 의존합니다.

- **SSO 복귀 origin 검증:** SSO 완료 후 복귀 주소는 `cors.allowed-origins` 목록으로 검증합니다 (`SsoController.resolveFrontendBaseUrl`). 프론트 접속 origin(로컬 nginx `http://localhost`, 운영 프론트 URL)이 목록에 없으면 SSO 성공 후 `app.frontend-url` 기본값으로 되돌아갑니다.
- **프로파일·환경변수 우선순위:** `local-ext` / `local-int` 기본값에는 `http://localhost` 가 포함되어 있지만, 프로파일 없이 기동하면 허용 목록이 비어 모든 교차 출처가 차단되고, `APP_FRONTEND_URL` · `CORS_ALLOWED_ORIGINS` 환경변수가 등록되어 있으면 기본 목록을 통째로 덮어씁니다. 로컬 nginx 검증 시에는 프로파일을 지정해 기동하거나 `CORS_ALLOWED_ORIGINS` 에 `http://localhost` 를 직접 포함시킵니다.
- **Origin 문자열 형식:** 허용 여부는 문자열을 정확히 비교하므로 `scheme://host[:port]` 만 등록합니다. 경로나 끝 슬래시가 붙은 URL은 브라우저 Origin과 일치하지 않습니다.

```
# 로컬 nginx(http://localhost) 검증용 기동 예시
$env:SPRING_PROFILES_ACTIVE = "local-ext"
./gradlew bootRun
```

데이터베이스 변경

DDL과 데이터 마이그레이션은 `C:\it\it_database\migrations` 에 새 Flyway 스크립트로 추가합니다.

```
V20260715_001__DescribeChange.sql
```

로컬 프로파일은 자동 적용할 수 있지만 dev/prod는 DBA 검토 후 수동 적용합니다. 상세는 [Flyway 가이드](#) 를 따릅니다.

테스트

일반 변경:

```
./gradlew test
```

인증·결재·파일·QueryDSL·감사로그처럼 공통 영향이 큰 변경:

```
./gradlew clean test
```

로컬 Oracle을 사용하는 테스트는 기본 테스트에서 제외되며 `integrationTest` 로 실행합니다.

Controller 계약은 MockMvc 슬라이스 테스트, 서비스 규칙은 Mockito 기반 단위 테스트, 실제 Oracle 매핑과 QueryDSL은 `@Tag("it")` 통합 테스트로 분리합니다. 기본 `test` 는 통합 태그를 제외하고 종료 후 JaCoCo HTML/XML 보고서를 생성합니다. 커버리지 기준 자체를 게이트로 확인할 때는 `jacocoTestCoverageVerification` 을 별도로 실행합니다.

빌드와 배포

```
./gradlew clean build
```

`war` 플러그인과 `SpringBootServletInitializer` 를 함께 사용하므로 생성된 실행 가능 WAR은 `java -jar` 로 구동하거나 외장 서블릿 컨테이너에 배포할 수 있습니다. 운영 배포 전 DB·JWT·프론트 URL·CORS·파일 경로·쿠키 Secure 설정을 확인합니다. `EnvironmentValidator` 는 운영 프로파일에서 필수 비밀값, CORS 와일드카드와 개발용 인증 토글을 검사하고 안전하지 않으면 기동을 중단합니다.

보안 참고

- 인증 토큰은 httpOnly 쿠키로 전달합니다.
- 쿠키 기반 JWT는 Stateless여도 CSRF 검토 대상입니다.
- CORS는 허용 Origin을 제한하고 `allowCredentials=true` 와 와일드카드를 함께 사용하지 않습니다.
- 관리자 화면 숨김은 보안 경계가 아니며 API 권한 검증이 필요합니다.
- 첨부파일 접근은 경로나 파일명이 아니라 첨부파일 종류에 등록된 판정기가 부모 자원 권한으로 판정합니다. 알려진 종류 목록은 손으로 관리하지 않고 판정기가 선언한 종류의 합집합이라, 판정기 없는 종류를 클라이언트가 지어내 업로드하는 경로가 구조적으로 막힙니다.

세부 정책의 SoT는 [CLAUDE.md](#)와 [인증·인가 가이드](#)입니다.

일반적인 개발 흐름

1. 대상 도메인의 Controller·Service·Repository와 기존 테스트를 확인합니다.
2. 실패 테스트 또는 검증 시나리오를 먼저 추가합니다.
3. 엔티티 변경이면 새 Flyway 스크립트를 작성합니다.
4. 권한·부서·소유권 범위를 서비스 계층에서 확인합니다.
5. 관련 테스트와 정적 빌드를 실행합니다.

문서

- [필수 백엔드 규칙](#)
- [상세 개발 가이드](#)
- [데이터 모델 인덱스](#)
- [DB 마이그레이션 작성 가이드](#)
- `docs/operations/` : 백엔드 배포·복구 기록
- [루트 개발 안내](#)
- [기술부채와 후속 과제](#)

IT Portal 백엔드 작업 계약

SOURCE: it_backend/CLAUDE.md

IT Portal 백엔드 작업 계약

이 파일은 백엔드 변경 시 반드시 지킬 규칙만 정의합니다. 설치·실행·배포·문제 해결은 [README](#), 배경과 예제는 [가이드 인덱스](#)를 따릅니다. 상위 공통 규칙은 [루트 CLAUDE](#)를 함께 적용합니다.

1. 기술과 구조

- Java 25, Spring Boot 4, Gradle Wrapper, Oracle, Spring Data JPA·QueryDSL을 사용합니다.
- standalone `gradle` 을 실행하지 말고 항상 `./gradlew` 를 사용합니다.
- 호출 방향은 `controller → service → repository → entity` 입니다. Controller는 검증·응답 변환, Service는 트랜잭션·권한·업무 흐름, Repository는 영속성만 담당합니다.
- 도메인 코드는 `com.kdb.it.domain.{domain}`, 공통 기능은 `common`, 외부 연동은 `infra` 에 둡니다.
- 엔티티를 API로 직접 반환하지 않고 DTO로 변환합니다.
- 상세 레이어 규칙은 [레이어와 패키지](#)를 따릅니다.

2. Oracle과 영속성

- 접속 계정은 `ITPAPP`, 객체 소유자는 `ITPOWN` 입니다. 세션의 `CURRENT_SCHEMA=ITPOWN` 을 전제로 코드에 스키마 접두어를 쓰지 않습니다.
- 물리 모델은 `../it_database/migrations/`, ORM은 엔티티, 조회용 색인은 [데이터 모델 인덱스](#)가 SoT 입니다.
- 엔티티·컬럼명은 `../meta/meta.txt` 를 우선하고, 충돌 처리만 [컬럼 명명 가이드](#)를 따릅니다.
- Oracle 빈 문자열은 NULL이므로 빈 문자열과 NULL을 다른 업무 상태로 설계하지 않습니다.
- ITPOWN의 문자셋은 `AL32UTF8` 이며 `VARCHAR2(n BYTE)` 에는 UTF-8 실제 바이트 길이로 저장 한도를 적용합니다. `@Size(max=n)` 이나 `String.length()` 만으로 BYTE 한도를 검증하지 말고 공통 `Utf8ByteLimit` 을 재사용하며, 글자 수 업무 제한이 따로 있으면 두 조건을 각각 검증합니다.
- 목록·대시보드 조회는 전체 엔티티보다 필요한 컬럼만 반환하는 `projection/read view/Row` DTO를 우선 사용하고 Service에서 API DTO로 변환합니다.
- 네이티브 `Object[]` 결과는 중앙 `NativeRowMapper` 와 전용 `fromRow` 팩토리로 변환하며, SELECT 컬럼 수·순서 불일치를 경계에서 검출합니다.
- 정보화사업 금액은 화면·서비스마다 다시 계산하지 않고 `ProjectAmountCalculator` 가 산출한 스냅샷을 사용합니다. 저장 단위 반올림과 `NUMBER(18,3)` 범위 검증은 한 곳에 모아 적용하고, 통화별 환산 규칙과 컬럼별 원금 의미는 [데이터 모델의 정보화사업 금액 계약](#)과 [사업 집행 가이드](#)를 SoT로 따릅니다.
- 복합키는 기존 `@IdClass` 패턴을 유지하고 `equals/hashCode` 계약을 함께 검증합니다.

- 감사 필드와 NOT NULL 기본값은 [엔티티와 감사 로그](#)를 따릅니다.
- QueryDSL-네이티브 타입-Jackson 경계는 [QueryDSL과 Oracle](#)을 따릅니다.

3. 마이그레이션

- DDL-시드-데이터 보정은 `../it_database/migrations/V{YYYYMMDD_NNN}__{CamelCaseDescription}.sql` 로 추가합니다.
- 적용된 Flyway 스크립트는 수정하지 않습니다. 변경은 새 버전으로 추가합니다.
- 로컬 프로파일만 애플리케이션 Flyway를 사용하고 dev/prod는 DBA 적용을 전제로 합니다.
- 엔티티 변경에는 해당 마이그레이션과 스키마 검증을 함께 추가합니다.
- 상세 작성·복구 규칙은 [DB 마이그레이션 가이드](#)와 [Flyway 운영](#)을 따릅니다.

4. API와 서비스

- 요청은 Bean Validation으로 검증하고 공통 예외 응답 계약을 사용합니다.
- `@ModelAttribute` 로 바인딩하는 DTO의 서버 전용 필드는 setter를 만들지 않고 (`@Setter(AccessLevel.NONE)`) 의미를 드러내는 전용 메서드로만 겁니다. `@Schema(hidden = true)` 는 OpenAPI 노출만 막고 쿼리 파라미터 바인딩은 막지 못합니다. setter 제거는 Lombok 어노테이션 정리로 조용히 되살아나므로, 해당 컨트롤러 전용 `@ControllerAdvice` 의 `@InitBinder` 에서 `setDisallowedFields` 로 그 필드를 함께 차단합니다.
- 사용자·외부에서 온 HTML을 저장할 때는 서버가 `HtmlSanitizer` 로 정화한 뒤 저장합니다. 프론트의 렌더 직전 정화는 이중 방어이지 서버 정화를 대신하지 않습니다.
- HTML 본문의 "비어 있음" 판정은 정화 **결과**로 다시 수행합니다. `@NotBlank` 와 길이 검증은 정화 전 원문에 걸리므로 정화가 전부 제거한 본문을 통과시킵니다.
- 서버가 문자열 연결로 HTML(메일·알림 본문·게시글 머리말 등)을 조립할 때 삽입하는 사용자·외부 값은 `HtmlUtils.htmlEscape` 로 이스케이프합니다. 이 경로는 `HtmlSanitizer` 를 타지 않으므로 정화 규칙이 적용되지 않으며, 도메인마다 이스케이프 헬퍼를 새로 만들지 않습니다.
- 목록 API의 페이지 파라미터는 검색 조건 DTO에 섞지 않고 공통 `ListPageParams` 를 별도 `@ModelAttribute` 로 바인딩합니다. 미지정이면 상한까지 조회하고, 지정하면 그 구간만 조회하면서 조건에 맞는 전체 건수를 `x-Total-Count` 헤더로 함께 돌려줍니다. 본문 배열 계약은 그대로 두며, 잘못된 페이지 입력은 기본값으로 되돌리지 말고 예외로 구분합니다. 새 헤더를 응답에 실으면 `SecurityConfig` 의 `setExposedHeaders` 에 함께 등록합니다.
- 신청서 상태 전이·결재선 변경처럼 동시 실행이 서로를 덮어쓰는 명령은 원장 행을 `findByIdForUpdate(PESSIMISTIC_WRITE)` 로 잠근 뒤 수행합니다.
- 조회는 읽기 전용 트랜잭션, 명령은 Service의 명시적 트랜잭션 경계에서 처리합니다.
- 조회 중심 Service는 클래스 수준 `@Transactional(readOnly = true)` 를 기본으로 두고, 쓰기 메서드만 `@Transactional` 로 명시적으로 오버라이드합니다.
- 목록 API는 필요한 필터와 정렬을 DB에 적용하고 메모리 전량 필터링을 만들지 않습니다.
- 목록 API는 전체 엔티티 fetch 대신 필요한 컬럼의 projection/read view/Row DTO를 우선하고, 명시적 안정 정렬과 페이지 크기 또는 상한을 DB 쿼리에 함께 둡니다.

- 여러 식별자를 조립하는 bulk 조회는 `IN` 배치 읽기와 `bounded request size`를 사용하며, 찾지 못한 식별자는 성공 데이터에 섞지 말고 호출자가 구분할 수 있게 보존합니다.
- DTO나 Controller 계약이 바뀌면 OpenAPI 계약 테스트와 프론트 `npm run codegen` 영향을 확인합니다.
- `@Schema(requiredProperties)`, nullable, enum 계약을 바꾸면 OpenAPI 계약 테스트와 프론트 `app/types/api.d.ts` 재생성을 같은 변경 묶음으로 완료합니다.
- 설정·이관 입력은 값이 없을 때만 업무상 기본값을 적용합니다. malformed 값은 absent/default로 풀백하지 말고 셀 진단, 경고 또는 차단으로 구분합니다.
- 업무 코드·상태·메뉴 경로 같은 식별자는 화면 표시명이 아니라 안정된 코드값을 사용합니다.

5. 인증·인가·데이터 범위

- Access/Refresh Token은 httpOnly 쿠키로만 전달합니다. Bearer 풀백은 명시적으로 허용된 개발·API 테스트 환경에서만 사용합니다.
- 서버 권한은 `SecurityConfig`, `@PreAuthorize`, Service 데이터 범위 검사로 강제합니다. 프론트 가드를 보안 경계로 간주하지 않습니다.
- `ROLE_INFOSEC_ADMIN`을 일반 `ROLE_ADMIN`과 합치지 않습니다. 협의회 심의유형 범위는 Service에서 검증합니다.
- `athIds`에서 역할로 변환하는 매핑은 명시적 allowlist로 유지하며, 미지원 자격등급을 관리자·특수 권한으로 매핑하지 않습니다. 새 역할은 매핑과 회귀 테스트를 함께 추가합니다.
- 부서·작성자·소유권 필터는 [데이터 접근 범위](#)를 따릅니다.
- 파일 업로드·다운로드·연결 대상 검증은 [파일 보안](#)을 따르며 경로 문자열만으로 권한을 판단하지 않습니다.
- 첨부파일 종류(`APG_FL_KD_NM`)를 새로 쓰려면 읽기 또는 쓰기 판정기를 먼저 등록합니다. 알려진 종류 목록은 손으로 관리하지 않고 판정기가 선언한 종류의 합집합이므로, 판정기 없는 종류가 업로드 경로로 조용히 생기지 않습니다. 쓰기 판정기를 추가할 때는 범용 파일 API의 수정·삭제를 열어 둘지 (`allowsGenericMutation`)를 기본값에 맡기지 말고 종류마다 명시적으로 판단합니다.
- 클라이언트 입력이 파일시스템 경로 세그먼트가 될 때는 허용문자 필터와 정규화 후 기준 디렉터리 포함 검증을 함께 겁니다. 읽기와 쓰기 양쪽에 같은 2단 검증을 적용합니다.
- 첨부 ZIP의 엔트리명은 저장 파일명·상대경로 원문을 그대로 쓰지 않고 경로 구분자·제어문자·상위 이동(..)을 제거하거나 거부한 뒤 `AttachmentArchiveSupport.uniqueEntryName`에 넘깁니다. 이 유틸은 중복만 해소하므로 정화 책임은 호출자에게 있으며, 여기서 지키는 대상은 서버가 아니라 ZIP을 푸는 수신자의 PC입니다.
- 관리자 전용 API는 `/api/admin/**` 경로에 두어 `SecurityConfig` URL 규칙을 받고 클래스 수준 `@PreAuthorize("hasRole('ADMIN')")`를 함께 답니다. 사용자 API와 경로를 공유해야 하면 관리자 메서드마다 `@PreAuthorize`를 명시하고 클래스 JavaDoc에 보호 범위를 적습니다.
- 클라이언트가 보낸 화면 경로·복귀 URL은 서버도 단일 `/`로 시작하는 내부 경로만 허용하고 `//`와 스킴(`://`) 포함 값을 거부합니다. 프론트의 같은 검증을 신뢰하지 않습니다.
- 목록·bulk·건수 조회는 상세와 같은 데이터 범위를 공유합니다. 부서 범위는 클라이언트가 보낸 조건이 아니라 인증 주체에서 유도해 덮어쓰며, 조건이 비어 있다는 이유로 전체 조회를 열지 않습니다. 상세만 보호하고 목록·bulk를 열어 두면 같은 응답 DTO가 우회 경로로 나옵니다.

- 인증 주체 판정은 fail-closed를 기본으로 합니다. principal이 기대한 타입이 아니면 권한 없음으로 처리하고, 권한 문자열만으로 관리자 여부를 단정하지 않습니다.
- 보안 목적이 아닌 식별자라도 난수는 `SecureRandom` 으로만 만듭니다. 미지원 환경을 위한 약한 난수 폴백을 두지 말고 명시적으로 실패시킵니다.
- 수동 로그인과 사용자 전자결재 상태 변경은 사용자·용도에 귀속된 1회용 MFA 증표를 요구합니다. 조회·임시저장·SSO·개발 사용자 전환·외부 결재 콜백에는 적용하지 않습니다.
- MFA 거래 상태를 서비스나 공급자의 인스턴스 로컬 필드에 두지 않습니다. 저장과 상태 전이는 `MfaTransactionStore` · `LoginPendingTransactionStore` 구현에만 맡기고, 전이는 조건부 UPDATE의 영향 행 수로 판정해 다중 인스턴스에서도 증표가 한 번만 소비되게 합니다.
- MFA pending/proof는 httpOnly·운영 Secure·SameSite=Lax 쿠키로만 전달하고 응답 본문이나 JavaScript 상태에 넣지 않습니다. 쿠키 수명은 서버 거래의 잔여 TTL을 넘기지 않습니다.
- CSRF 토큰을 사용하지 않는 동안 unsafe 단순 Content-Type 요청은 X-Requested-With 헤더를 요구하며, 프론트 인증 fetch는 `credentials: 'include'` 와 해당 헤더를 함께 보냅니다. 이 헤더 값 자체는 토큰으로 간주하지 않습니다.
- 운영 프로파일은 비밀값·Origin·프론트 URL을 fail-fast로 검사하고 모의 SSO, 직접 사번, 개발 사용자 전환, Bearer 폴백, 비보안 쿠키를 허용하지 않습니다.
- 전체 계약은 [인증과 인가](#)를 SoT로 사용합니다.

6. 이벤트와 외부 연동

- 트랜잭션 성공 뒤 실행해야 하는 알림·메일은 커밋 이후 이벤트로 처리하고 본 업무 트랜잭션과 실패 경계를 분리합니다.
- 알림 상태·재시도는 [알림 가이드](#), SSO는 [SSO](#), EAI는 [EAI](#)를 따릅니다.
- 외부 응답 문자열·HTML·파일명을 신뢰하지 말고 허용 목록, 정규화, 크기 제한과 안전한 오류 매핑을 적용합니다.
- EAI 전문 로그에는 개인정보·비밀값을 남기지 않고 승인된 필드만 마스킹 후 기록합니다.

7. 도메인 공통 규칙

- 게시판, 댓글, 첨부파일의 권한과 순서는 서버가 최종 검증합니다. 클라이언트가 보낸 작성자·부서·순서를 신뢰하지 않습니다.
- 정보화사업 집행 계약은 [사업 집행 가이드](#)를 따릅니다.
- 정보화사업과 경상사업은 같은 `BPROJM` 원장에서 (`ABUS_MNG_NO`, `SNO`)로 재상신 개정본을 식별하며 `ODN_YN='Y'` 가 경상사업, NULL 또는 `N` 이 정보화사업입니다. 전산업무비는 `BCOSTM` 의 (`BG_NO`, `BG_SNO`)를 사용합니다. 결재완료 최종본만 다음 순번의 `LST_YN='N'` 초안으로 복제하고, 승인 완료 이벤트가 가리키는 정확한 순번만 원자적으로 `Y` 로 전환합니다. 후속 업무 목록·집계·bulk 조회는 `LST_YN='Y'` 만 소비하되 `apfSts=none` 미상신 작성 목록은 재상신 초안을 포함하고 순번을 반환합니다.
- Tiptap 변수 카탈로그와 해석 계약은 [Tiptap 변수](#)를 따릅니다.

- 사업 입력 길라잡이의 대상 필드는 서버 고정 카탈로그가 SoT입니다. 길라잡이 ID는 사업 유형 접두사로 범위를 구분하고, 카탈로그에 없는 ID는 저장하지 않습니다. 사용자 조회 API는 본문이 등록된 항목만 돌려주고 카탈로그 전체 조회와 등록·삭제는 관리자 전용입니다.
- 메뉴명과 공통코드 표시명은 DB 번역 데이터가 SoT입니다. 분기와 저장에는 번역명이 아니라 코드값을 사용합니다.
- 전용 테이블 없이 BGD0CM 단일 문서로 운영하는 기능(공통 안내 팝업, 스피드다이얼 담당자 정보 등)은 DOC_TTL_CONE 고정 식별자와 그 식별자로 범위를 좁힌 조건부 유일 인덱스로 "활성 1건" 불변식을 지킵니다. 최초 생성은 동시 저장의 유일 제약 실패가 호출자 트랜잭션을 오염시키지 않도록 REQUIRES_NEW 로 분리합니다. DOC_MNG_NO 접두사는 기능마다 새로 정합니다 — 기존 접두사를 재사용하면 접두사만으로 필터링하는 다른 기능의 목록·상세에 그 문서가 섞여 노출됩니다.
- 전용 화면 없이 공통 게시판을 재사용하는 기능은 게시판을 이름이나 관리번호로 찾지 않고 BoardTypeResolver.requireUniqueActiveBoard(유형코드) 로 해석합니다. 활성 게시판이 없거나 둘 이상이면 조용히 첫 건을 고르지 말고 실패시킵니다.
- 준비중 메뉴는 화면 경로를 사람이 입력받지 않고 /preparing/{menuId 소문자} 로 채번해 라우트 카탈로그에 함께 등록합니다. 준비중을 해제하거나 메뉴를 삭제할 때 회수하는 대상은 이 규칙으로 자동 생성한 경로뿐이며, 사람이 직접 등록한 준비중 경로는 다른 메뉴가 참조할 수 있으므로 회수하지 않습니다. 준비중 화면의 안내 문구는 그 카탈로그 행의 비고(RMK)를 사용자에게 그대로 보여주므로 비고에 내부 메모를 적지 않습니다.

8. 주석·로그·테스트

- 신규 주석은 한글로 작성하고 [주석 스타일](#)을 따릅니다.
- 애플리케이션 로그와 실시간 로그의 책임은 [로그](#), [실시간 로그](#)를 따릅니다.
- WAS 로그 링버퍼는 관리자 화면과 다운로드로 원문이 나가는 경로이므로 적재 전에 마스킹하고, 조 회·레벨변경·다운로드는 예외 없이 관리자 감사 로그를 남깁니다. 클라이언트가 보낸 문자열을 로그에 남길 때는 개행·제어문자를 제거해 로그 줄 위조를 막습니다.
- 런타임 로그레벨 변경은 Actuator 엔드포인트를 열지 않고 `LoggingSystem` 빈으로만 처리하며, 항상 TTL·동시 건수 상한·로거 접두사 화이트리스트를 함께 적용합니다. 루트 로거 전체 변경은 허용하지 않습니다.
- 인스턴스 간 내부 전용 엔드포인트는 공유 비밀 헤더로만 인증하고, 비밀값이 비어 있으면 컨트롤러 빈 자체를 등록하지 않아 인증 없는 경로가 열리지 않게 합니다.
- 순수 Service-유틸은 JUnit 단위 테스트, MVC·보안 계약은 MockMvc/통합 테스트, 연속성 변경은 Oracle 호환 검증을 추가합니다.
- 의존성·보안·DB 변경은 관련 회귀 테스트와 문서 계약을 함께 갱신합니다.

변경 범위에 맞게 다음 명령을 실행합니다.

```
./gradlew test
./gradlew check
./gradlew bootJar
```

특정 테스트는 `./gradlew test --tests '패키지.클래스명'` 으로 먼저 확인하되 완료 전에는 영향 범위 전체를 실행합니다.

9. 문서 위치

- 설치·실행·환경·배포·문제 해결: [README](#)
- 상세 개발 가이드: [docs/guides/README.md](#)
- DBA 배포·복구 기록: `docs/operations/`
- 물리 마이그레이션: `../it_database/migrations/`
- 미구현·기술부채: `../TASK.md`

백엔드 개발 가이드

SOURCE: [it_backend/docs/guides/README.md](#)

백엔드 개발 가이드

필수 작업 계약은 [백엔드 CLAUDE](#), 설치·실행·배포는 [백엔드 README](#)가 담당합니다. 이 디렉터리는 반복해서 참고하는 구현 방법과 설계 배경만 보관합니다.

아키텍처와 규약

- [레이어와 패키지](#)
- [한글 주석 스타일](#)

영속성과 데이터 모델

- [데이터 모델 인덱스](#)
- [컬럼 명명과 충돌 규칙](#)
- [엔티티와 감사 로그](#)
- [QueryDSL과 Oracle](#)

물리 DDL·DML과 마이그레이션 작성 규칙은 [데이터베이스 가이드](#)를 따릅니다.

인증·인가·파일 보안

- [인증과 인가](#)
- [데이터 접근 범위](#)
- [파일 보안](#)

특정 마이그레이션의 적용·복구 기록은 `docs/operations/`에 둡니다.

도메인

- [정보화사업 집행](#)
- [편성요청서 반입](#)
- [신청서 상태와 작성완료 스탬프](#)
- [Tiptap 변수](#)

외부 연동

- [EAI](#)
- [알림](#)
- [SSO](#)

운영

- [Flyway](#)
- [로깅](#)
- [실시간 로그](#)
- [감사로그 저장 실패 모니터링](#)

작성 원칙

- 가이드에는 재사용 가능한 규칙·예제·체크리스트만 기록합니다.
- 날짜, 적용 환경, 배포 상태, 복구 결과는 `../operations/`에 기록합니다.
- 파일·테스트 개수처럼 자주 변하는 수치를 넣지 않습니다.
- CLAUDE와 README의 본문을 복제하지 말고 정식 문서로 연결합니다.

데이터 모델 인덱스

SOURCE: [it_backend/docs/guides/persistence/data-model.md](#)

데이터 모델 인덱스

물리 구조의 SoT는 `C:\it\it_database\migrations`, ORM 매핑의 SoT는 엔티티입니다. 이 문서는 도메인과 테이블을 빠르게 찾기 위한 인덱스입니다.

기본 명명

- 테이블 접두사: `TPRMPP_`
- 공통 영역: `C*`, 업무 영역: `B*`
- 마스터: `*M`, 로그: `*L`, 이력: `*H`, 관계: `*A`
- 엔티티와 대응 `*L` 로그 엔티티는 같은 업무 필드명을 사용합니다.

주요 매핑

도메인	주요 엔티티·테이블
정보화사업	<code>Bprojm/TPRMPP_BPROJM</code> , <code>Bproja/TPRMPP_BPROJA</code> , <code>Bitemm/TPRMPP_BITEMM</code>
전산업무비	<code>Bcostm/TPRMPP_BCOSTM</code> , <code>Btermm/TPRMPP_BTERMM</code>
계획·예산	<code>Bplanm</code> , <code>Bplana</code> , <code>Bbugtm</code>
문서·검토	<code>Bgdocm</code> , <code>Brdocm</code> , <code>Brivgm</code>
사업 집행	<code>Bestim</code> , <code>Besttm</code> , <code>Bdelim</code> , <code>Bcontm</code> , <code>Bpaymm</code> , <code>Bpaymt</code>
협의회	<code>Basctm</code> , <code>Bcmmtm</code> , <code>Bevalm</code> , <code>Bperfm</code> , <code>Bpovwm</code> , <code>Bpqnam</code> , <code>Bmqnam</code> , <code>Brsltm</code> , <code>Bschdm</code>
결재	<code>Capplm</code> , <code>Cappla</code> , <code>Cdecim</code>
IAM	<code>CuserI</code> , <code>CorgnI</code> , <code>CauthI</code> , <code>CroleI</code> , <code>Clognh</code> , <code>Crtokm</code>
게시판	<code>Cblbmm</code> , <code>Cblbcm</code> , <code>Ccmmtm</code>
공통·인프라	<code>Ccodem</code> , <code>Cfilem</code> , <code>Cinfmm</code> , <code>Cmenum</code> , <code>Cmenua</code> , <code>Cmenud</code>
추가인증(MFA)	<code>MfaTransactionEntity/TPRMPP_CMFATM</code> , <code>LoginPendingTransactionEntity/TPRMPP_CMFADM</code>

정확한 물리 테이블명·컬럼·제약은 엔티티와 최신 마이그레이션을 함께 확인합니다.

BPROJA 관계

BPROJA 는 사업과 단계별 원본문서를 연결하고 단계 상태를 보관합니다. 사업 대표 상태는 유효 관계 행의 상태 집계로 계산합니다.

단계	원본	관계 키
사전협의	BRDOCM	문서관리번호
예산편성	BBUGTM	예산번호
정보기술부문계획	BPLANM	요청문서번호
협의회	BASCTM	협의회 ID
소요예산	BESTIM	요청문서번호
과업심의	BDELIM	문서관리번호
입찰·계약	BCONTM	문서관리번호
대금지급	BPAYMM	문서관리번호

정보화사업 금액 계약

물리 구조는 `it_database/migrations/` , 계산·저장 동작은 정보화사업 서비스가 SoT이며, 아래 표는 BITEMM 과 BPROJM 의 금액 의미를 고정한다.

테이블	컬럼	통화	의미
TPRMPP_BITEMM	AMT	KRW	품목의 당해 원화 금액. 외화 품목도 환산된 당해 원화 금액을 저장한다.
TPRMPP_BITEMM	FC_AMT	외화	외화 품목의 당해 원금. CUR_C 가 KRW 이거나 null인 품목에서는 null이어야 한다.
TPRMPP_BITEMM	MPL_AMT	원금 통화	품목의 예정 원금. KRW/null 통화 품목은 원화 원금, 외화 품목은 외화 원금이다.
TPRMPP_BPROJM	MPL_AMT	KRW	활성 품목 예정금액의 원화 환산 합계 스냅샷.
TPRMPP_BPROJM	DFR_AMT	KRW	사업에 기록하는 지급 원화 금액.

테이블	컬럼	통화	의미
TPRMPP_BPROJ M	TOT_RQM_AM T	KRW	당해 원화 금액, 예정 원화 금액, 지급 원화 금액을 합한 총소요금액 스냅샷.

활성 품목은 `DEL_YN = 'N' AND LST_YN = 'Y'` 로 한정한다. 예정 원화 금액은 KRW/null 통화에서 `NVL(MPL_AMT, 0)`, 외화에서 `ROUND(NVL(MPL_AMT, 0) * XCR, 3)` 으로 계산한다. 외화 품목은 `FC_AMT` 와 양수 `XCR` 가 필요하다.

기존 업무 데이터는 이 계약으로 마이그레이션하거나 보정하지 않는다. 별도 운영 절차로 기존 데이터를 삭제한 뒤에 새로 저장하는 데이터부터 이 계약을 적용한다. 적용·공유된 과거 Flyway 스크립트는 불변이며, 물리 변경이 필요하면 새 마이그레이션으로만 추가한다.

공통 컬럼

`BaseEntity` 는 삭제 여부, GUID, 최초 등록과 최종 변경 일시·사용자를 제공합니다. 물리 삭제 대신 Soft Delete를 사용합니다.

금액·상태처럼 계산으로 파생되는 API 필드는 테이블 컬럼으로 가정하지 않습니다. 제거·추가 이력은 마이그레이션과 Git 이력에서 확인합니다.

인증과 인가 가이드

SOURCE: [it_backend/docs/guides/security/authentication-authorization.md](#)

인증과 인가 가이드

JWT와 쿠키

- Access Token은 15분, Refresh Token은 7일입니다.
- 두 토큰은 httpOnly 쿠키로 전달합니다.
- Access Token에는 사번, 자격등급 목록과 부서코드를 포함합니다.
- Refresh Token 갱신 시 사용자 권한과 부서를 DB에서 다시 읽습니다.
- Refresh Token은 회전하며 유효 기간 이후 재사용이 탐지되면 사용자 토큰 패밀리를 폐기합니다.
- Refresh Token 원문은 DB에 저장하지 않습니다. `TPRMPP_CRTOKM.ECY_RNW_PUB_TOK_CONE`에는 조회용 소문자 SHA-256 HEX 값만 저장합니다.
- Access Token이 만료되어도 Refresh 쿠키로 로그아웃하면 해당 토큰 소유자의 패밀리를 폐기합니다.
- 운영에서는 Bearer 헤더 폴백을 비활성화하고 쿠키 인증을 기본으로 합니다.
- SSO 검증 사번과 결과는 JWT와 분리된 서버 세션에 보관합니다. 운영 `JSESSIONID`는 `Path=/`, `Secure`, `HttpOnly`, `SameSite=Lax`를 모두 유지하며 `EnvironmentValidator`가 누락·override를 기동 시 차단합니다.

운영 인증 안전장치

`prod` 프로파일에서는 다음 위험 설정을 허용하지 않으며, 하나라도 감지되면 애플리케이션 기동이 실패합니다.

- `sso.mock-enabled=true`
- `app.auth.allow-bearer-header=true`
- `app.cookie.secure=false`
- `server.servlet.session.cookie.secure` 또는 `http-only`가 명시적 `true`가 아닌 경우
- `server.servlet.session.cookie.same-site`가 명시적 `Lax`가 아닌 경우

SSO 인증 성공 경계에서는 기존 HTTP 세션 ID를 교체하여 세션 고정을 방지합니다.

RBAC

자격등급	역할	의미
ITPAD001	ROLE_ADMIN	시스템 관리자
ITPAD002	ROLE_INFOSEC_ADMIN	정보보호 관리자
ITPZZ002	ROLE_DEPT_MANAGER	부서 관리자
ITPZZ001	ROLE_USER	일반 사용자

관리자 전용 도메인은 컨트롤러 클래스 수준 `@PreAuthorize("hasRole('ADMIN')")` 를 사용합니다. 협의회와 사업 집행처럼 세부 업무 범위가 필요한 API는 서비스 계층에서 역할·소유권·상태를 검증합니다.

CSRF와 CORS

쿠키 기반 JWT는 Stateless여도 브라우저가 자격증명을 자동 전송하므로 CSRF 검토 대상입니다.

- 현재 경계는 `SameSite=Lax`, 명시적 CORS Origin과 `allowCredentials=true` 조합입니다.
- CORS는 응답 읽기 정책이지 단독 CSRF 방어가 아닙니다.
- `SameSite=None`, 와일드카드 Origin, 임의 Origin 추가는 별도 CSRF 보강 없이 허용하지 않습니다.
- `/sso/**` 전체 페이지 콜백 예외를 일반 SPA API로 확대하지 않습니다.

현재 위험 모델

현재 전역 `csrf.disable()` 은 아래 경계가 모두 유지되는 동안에만 적용합니다. 각 경로의 자동 전송 자격증명, 상태 변경 여부와 방어 수단을 함께 검토합니다.

경로	자동 전송 자격증명	상태 변경	현재 방어	잔여 위험
POST <code>/api/auth/login/start</code>	기존 JWT 불필요	MFA 로그인 대기 쿠키 발급	명시 CORS, 로그인 검증 잠금	JWT는 발급하지 않으며 MFA 완료 후에만 로그인 완료 API를 호출
POST <code>/api/auth/login/complete</code>	pending-MFA proof 쿠키	Access/Refresh 발급	증표 소유자·용도·만료·재사용 검증	로그인 CSRF는 공격자 계정 세션 주입 관점에서 별도 관찰
POST <code>/api/auth/refresh</code>	Refresh 쿠키 (<code>/api/auth</code>)	토큰 회전	<code>SameSite=Lax</code> , 명시 Origin, POST	<code>SameSite</code> 완화 시 최우선 CSRF 토큰 대상
POST <code>/api/auth/logout</code> 및 인증 변경 API	Access/Refresh/User 쿠키와 서버 세션	서버 세션 무효화/DB 변경	<code>SameSite=Lax</code> , 명시 Origin, unsafe method, 서버 토큰 폐기 실패 시에도 로컬 세션과 세 쿠키 삭제	CORS만 단독 방어로 간주하지 않음

경로	자동 전송 자격증명	상태 변경	현재 방어	잔여 위험
POST/PUT/PATCH/DELETE <code>/api/**</code>	Access 쿠키(/)	업무 데이터 변경	SameSite=Lax, 명시 Origin, 인증·인가	교차 사이트 SPA/iframe 도입 시 보강 필요
GET <code>/api/boards/{b1bMngNo}/posts/{nacMngNo}</code>	Access 쿠키(/)	없음(순수 상세 조회)	read-only 서비스, 조회수 변경 미호출 회귀 테스트	GET에 DB 변경을 다시 결합하지 않음
POST <code>/api/boards/{b1bMngNo}/posts/{nacMngNo}/views</code>	Access 쿠키(/)	조회수 1 증가	SameSite=Lax, 명시 Origin, 게시판-게시물 소속·읽기 권한 검증, 비관적 쓰기 잠금과 managed entity 변경으로 감사 로그 유지	조회수 실패는 상세 조회와 분리
GET/POST <code>/sso/{business,checkauth,loginProc,agentProc}</code>	SSO 상태 JSESSIONID ; 같은 사이트에서는 Path=/ Access 쿠키도 전송 가능	외부 인증 콜백	SSO 상태를 JWT와 분리된 서버 검증 세션에 보관, Agent 결과 원자적 1회 소비, JSESSIONID Secure/HttpOnly/SameSite=Lax, CORS allowCredentials=false	Access 쿠키를 SSO 검증 상태로 사용하지 않으며 예외를 <code>/api/**</code> 로 확대 금지
POST <code>/sso/logout</code>	SSO 상태 JSESSIONID	서버 세션 무효화	POST 전용, SameSite=Lax, SSO CORS allowCredentials=false	GET <code>/sso/logout</code> 은 Allow: POST와 함께 405를 반환하며 세션을 변경하지 않음
GET <code>/api/auth/sso/complete</code>	검증된 JSESSIONID/SSO 상태 쿠키	JWT 쿠키 발급 후 SSO 세션 무효화	검증 사번 원자적 1회 소비, origin allowlist, safe next, 성공·실패 세션 종료	SSO 완료 전용 예외 이며 일반 상태 변경 GET의 선례로 확대 금지

CSRF 보강 트리거와 목표 구현

다음 중 하나라도 발생하면 현재 `csrf.disable()` 유지는 금지됩니다. 해당 변경과 같은 배포 단위에서 CSRF 토큰 또는 동등한 서버 검증 Origin/nonce 방어를 활성화해야 합니다.

1. Access/Refresh/User/SSO 상태 쿠키 또는 `JSESSIONID` 중 하나를 `SameSite=None`으로 변경한다.
2. `credentialed /api/**`에 새 교차 사이트 Origin을 추가하거나 wildcard/pattern으로 완화한다.
3. 프론트를 cross-site iframe에 임베드하거나 별도 사이트의 SPA가 쿠키 API를 호출한다.
4. 모든 GET은 순수 조회로 유지합니다. GET 상태 변경은 금지하며, 발견하면 unsafe method로 분리하기 전까지 배포하지 않습니다.
5. `/sso/**`의 `allowCredentials`를 `true`로 변경한다.

목표 구현은 Spring Security `CookieCsrfTokenRepository` 또는 동등한 synchronizer/double-submit token을 사용합니다. 프론트의 `$apiFetch / useApiFetch`는 unsafe method에 토큰 헤더를 전송하고, 로그인·refresh-logout-SSO complete의 토큰 발급·회전 예외를 별도 테스트로 검증합니다. Origin/Referer 검증만으

로 대체하려면 누락 헤더 정책과 신뢰 프록시 경계를 문서화하고 보안 리뷰 승인을 받아야 합니다.

운영 비밀값

DB 비밀번호, JWT secret, 외부 API 키는 환경변수로 주입합니다. `EnvironmentValidator`의 운영 기동 차단 규칙을 우회하지 않습니다.

로그인 보호

사번 기준 로그인 실패 횟수와 잠금은 DB 로그인 이력으로 판단합니다. IP·기기 기반 보호와 사용자 열거 방지처럼 미구현된 개선 사항은 `TASK.md`에서 관리합니다.

Flyway 운영 가이드

SOURCE: `it_backend/docs/guides/operations/flyway.md`

Flyway 운영 가이드

위치와 이름

마이그레이션은 `C:\it\it_database\migrations` 에 둡니다.

```
V{YYYYMMDD_NNN}__{CamelCase설명}.sql
```

DDL과 필요한 데이터 백업을 한 변경 단위로 구성하고 재실행 가능성을 고려합니다. 성공 적용된 파일은 체크섬 대상이므로 수정하지 않습니다.

적용 정책

- Gradle `processResources` 가 마이그레이션을 classpath에 포함합니다.
- `local-ext`, `local-int` 만 자동 적용합니다.
- dev/prod는 DBA 검토 후 수동 적용합니다.
- 애플리케이션 계정과 DDL 계정이 다르면 `FLYWAY_USER`, `FLYWAY_PASSWORD` 를 사용합니다.
- 빈 스키마와 기존 baseline 스키마의 적용 경로를 각각 검증합니다.

컬럼 변경

- rename은 명시적 `ALTER TABLE ... RENAME COLUMN` 을 사용합니다.
- 타입 변경은 임시 컬럼, 데이터 변환, 기존 컬럼 제거, rename 순서로 안전하게 수행합니다.
- NOT NULL 추가는 기존 데이터 백업 후 제약을 적용합니다.
- 엔티티 변경, 로그 미러, 인덱스·시퀀스와 테스트 픽스처를 함께 확인합니다.

IT Portal 데이터베이스

SOURCE: it_database/README.MD

IT Portal 데이터베이스

IT Portal의 Oracle 물리 스키마, 시드와 Flyway 마이그레이션을 관리하는 독립 저장소입니다.

저장소 구조

```
it_database/
├─ migrations/      # 버전형 DDL·DML, 물리 모델 SoT
├─ tools/           # DB 검증 도구
├─ docs/guides/     # 반복 개발·운영 절차
├─ docs/operations/ # 버전별 DBA 인계·배포·복구 기록
├─ docs/verification/ # 실행 가능한 버전별 검증 SQL
├─ apply-ddl-live.ps1 # 스키마 부트스트랩·수동 재생성
├─ export-ddl-live.ps1 # 현재 스키마 DDL 추출
└─ *.par            # Data Pump 옵션(자격증명 제외)
```

필수 변경 규칙은 [CLAUDE.md](#), 반복 절차는 [가이드 인덱스](#)를 먼저 확인합니다.

사전 준비

- Oracle Database 21c XE
- 개발 서비스 XEPDB1
- 접속 주소 127.0.0.1:11521/XEPDB1
- 접속 계정 ITPAPP
- 객체 소유 스키마 ITPOWN
- 형제 디렉터리 C:\it\it_backend

상세한 PDB·리스너·확장 문자열 설정은 [로컬 Oracle XE 설정](#)을 따릅니다.

빠른 시작

접속과 버전을 확인합니다. 비밀번호는 SQL*Plus 콘솔 프롬프트에서만 입력합니다.

```
sqlplus ITPAPP@127.0.0.1:11521/XEPDB1
```

```
SHOW CON_NAME;  
SELECT banner FROM v$version;  
ALTER SESSION SET CURRENT_SCHEMA = ITPOWN;
```

로컬 증분 마이그레이션은 백엔드의 `local-ext` 또는 `local-int` 프로파일 기동 시 적용됩니다. 단, `V20260831_003` 미적용 환경은 백엔드를 먼저 기동하지 말고 [byte-semantics-revert-precheck.sql](#) 을 SQL*Plus 또는 SQLcl에서 실행해야 합니다. 차단 0건·조회실패 0건과 정상 종료를 배포 증적으로 남긴 경우에만 Flyway를 진행합니다.

```
cd C:\it\it_backend  
./gradlew bootRun --args='--spring.profiles.active=local-int'
```

적용 뒤 `flyway_schema_history` 에서 버전, 성공 여부와 checksum을 확인합니다. dev/prod는 애플리케이션 자동 적용 대상이 아니며 DBA가 검토 후 수동 적용합니다.

마이그레이션 추가

```
migrations/V{YYYYMMDD_NNN}__{CamelCaseDescription}.sql
```

- 기존 버전과 충돌하지 않는 다음 일련번호를 사용합니다.
- 적용되었거나 공유된 스크립트는 수정하지 않고 새 버전을 추가합니다.
- DDL, 데이터 보정, 검증과 복구 경계는 [마이그레이션 작성 가이드](#)를 따릅니다.
- 사전 점검·유지보수 창·복구 절차가 필요한 변경은 `docs/operations/` 에 DBA 인계 노트를 작성합니다.

현재 문자열 길이 기준

ITPOWN의 데이터베이스 문자셋은 `AL32UTF8` 이고 `VARCHAR2` 는 BYTE semantics입니다. `VARCHAR2(300 BYTE)` 는 한글 300자가 아니라 UTF-8 인코딩 결과 300바이트를 저장하므로, 화면·서버·이관 검증도 같은 바이트 기준을 사용해야 합니다. 신규 DDL은 세션의 `NLS_LENGTH_SEMANTICS` 에 의존하지 않고 `VARCHAR2(n BYTE)` 를 명시합니다.

스키마 전체 전환 기준은 [V20260831_003_RevertCharSemanticsToByteSemantics.sql](#)입니다. BYTE 길이를 줄이거나 기존 CHAR 컬럼을 전환하는 후속 작업은 [byte-semantics-revert-precheck.sql](#)과 같은 `LENGTHB` 사전 진단으로 초과 데이터를 먼저 확인합니다. 이미 전환한 환경은 [postverify](#)와 [절단 영향 감사 런북](#)에 따라 구조와 원본 보존을 서로 분리해 검증합니다.

예산 재상신 물리 키

업무	원장과 버전 키	최종본
정보화사업-경 상사업	TPRMPP_BPROJM (ABUS_MNG_NO, SNO)	ODN_YN='Y' 는 경상사업, NULL/ N 은 정보화사업이며 같은 관리번호에 서 LST_YN='Y' 인 1개 개정본
전산업무비	TPRMPP_Bcostm (BG_NO, BG_SNO)	같은 관리번호에서 LST_YN='Y' 인 1개 개정본

재상신 초안은 같은 관리번호의 다음 순번과 LST_YN='N' 으로 저장되고, 결재 완료 시 신청서가 가리키는 정확한 순번이 최종본으로 전환됩니다. 정보기술부문 계획 TPRMPP_BPLANM 은 별도 SNO 재상신 원장이 아니며, 로컬 전용으로 추가됐던 버전 컬럼은 V20260831_005__RemoveLocalPlanVersionColumns.sql 에서 제거되었습니다.

가져오기

apply-ddl-live.ps1 은 스키마 최초 구성이나 승인된 수동 재생성에만 사용합니다.

```
it_database\apply-ddl-live.ps1 -DropExisting -StripCollation  
impdp 'ITPAPP@"127.0.0.1:11521/XEPDB1"' parfile=it_database\import_data_only.par
```

평상시 증분 변경에는 이 스크립트 대신 Flyway를 사용합니다. -DropExisting 은 기존 객체를 제거하므로 대상 스키마와 백업을 확인한 경우에만 실행합니다.

내보내기

.par 파일에는 접속 정보나 비밀번호를 넣지 않습니다.

```
expdp 'ITPAPP@"127.0.0.1:11521/XEPDB1"' parfile=it_database\export.par  
it_database\export-ddl-live.ps1
```

데이터 전용 반입과 Wallet 기반 무인 실행은 [가져오기·내보내기 가이드](#)를 따릅니다.

보안과 운영 주의

- 수동 실행은 콘솔 비밀번호 프롬프트, CI·무인 실행은 Oracle Wallet 별칭을 사용합니다.
- 비밀번호를 명령행, 환경변수, .par, SQL, 로그에 기록하지 않습니다.
- 로컬 XE 편의 권한을 dev/prod 권한 모델로 복제하지 않습니다.

- `EXPDAT.DMP` 의 Git 히스토리 재작성은 일반 유지보수가 아닙니다. 필요 시 [별도 운영 런북](#)에 따라 승인·공지·신선한 클론에서 수행합니다.

문서

- [필수 DB 작업 규칙](#)
- [DB 개발 가이드](#)
- [백엔드 Flyway 운영 가이드](#)
- [백엔드 데이터 모델 인덱스](#)
- `docs/operations/` : DBA 인계와 배포·복구 기록
- `docs/verification/` : 버전별 검증 SQL

IT Portal 데이터베이스 작업 계약

SOURCE: it_database/CLAUDE.md

IT Portal 데이터베이스 작업 계약

이 파일은 `it_database` 변경 시 반드시 지킬 규칙을 정의합니다. 설치·실행은 [README](#), 반복 절차는 [가이드 인덱스](#), 날짜가 있는 배포·복구 기록은 `docs/operations/`를 따릅니다. 상위 공통 규칙은 [루트 CLAUDE](#)를 함께 적용합니다.

1. 저장소 책임

- `migrations/`: 애플리케이션 데이터베이스의 물리 DDL·시드·데이터 보정 SoT
- `tools/`: 재사용 가능한 DB 검증 도구
- `docs/guides/`: 마이그레이션 작성, 로컬 Oracle, 가져오기·내보내기 절차
- `docs/operations/`: 특정 버전의 DBA 인계·배포·복구 기록
- `docs/verification/`: 특정 마이그레이션의 실행 가능한 검증 SQL

백엔드 엔티티는 ORM SoT이며 물리 구조와 함께 변경합니다. 조회용 관계 설명은 `../it_backend/docs/guides/persistence/data-model.md`를 따릅니다.

2. 마이그레이션

- 파일명은 `V{YYYYMMDD_NNN}__{CamelCaseDescription}.sql` 형식을 사용합니다.
- 적용되었거나 공유된 마이그레이션은 수정·교체·재번호화하지 않고 새 버전을 추가합니다. 병행 작업으로 같은 날짜 번호가 겹치는 것을 막으려면 내용을 채우기 전에 빈 파일로 번호를 선점합니다. 개변이 이미 일어났다면 [개변 복구 절차](#)를 따릅니다.
- Oracle DDL은 implicit commit이 있으므로 데이터 보정과 DDL을 같은 실패 단위로 가정하지 않습니다.
- destructive DDL, NOT NULL 전환, PK·인덱스 재생성, 대량 backfill은 사전 진단·적용 후 검증·롤백 또는 재시도 절차를 `docs/operations/`에 기록합니다.
- SQL은 재실행 가능성과 실패 시 상태를 명확히 하며, 업무 데이터의 제목·본문·개인정보를 진단 출력에 포함하지 않습니다.
- 테이블·컬럼·시퀀스 이름은 `../meta/meta.txt`와 현재 마이그레이션을 대조합니다.
- ITPOWN의 데이터베이스 문자셋은 `AL32UTF8`, `VARCHAR2` 길이 기준은 BYTE semantics입니다. 신규·변경 컬럼은 `VARCHAR2(n BYTE)`를 명시하고, `CHAR` semantics로 되돌리는 변경은 별도 승인과 전수 영향 분석 없이 추가하지 않습니다.

- BYTE 길이를 줄이거나 CHAR에서 BYTE로 전환할 때는 적용 전에 `LENGTHB` 로 초과 행을 전수 진단합니다. 업무 문자열을 자동 절단하는 보정은 일반 규칙이 아니며, 명시적 데이터 보정 승인·백업·검증·복구 절차가 있을 때만 수행합니다.
- 재상신 물리 키는 정보화사업·경상사업의 `BPROJM(ABUS_MNG_NO, SNO)` 와 전산업무비의 `BCOSTM(BG_NO, BG_SNO)` 입니다. `BPROJM.ODN_YN='Y'` 는 경상사업, NULL 또는 N 은 정보화사업입니다. `LST_YN` 최종본 불변식과 순번을 참조하는 품목·단말기·결재 매핑을 함께 검증하며, `BPLANM` 에 로컬 전용 버전 컬럼을 다시 추가하지 않습니다.
- 상세 규칙은 [마이그레이션 작성 가이드](#)를 따릅니다.

3. 계정과 비밀값

- 접속 계정은 `ITPAPP`, 객체 소유 스키마는 `ITPOWN` 입니다.
- 수동 작업은 Oracle 도구의 콘솔 비밀번호 프롬프트를 사용합니다.
- 비밀번호를 명령행 인자, `.par` 파일, SQL, 로그, Git 추적 파일에 기록하지 않습니다.
- CI·무인 실행은 Oracle Wallet 별칭만 사용합니다.
- 운영 권한은 최소 권한으로 부여하며 README의 로컬 XE 부트스트랩 예시를 운영 권한 모델로 사용하지 않습니다.

4. 적용 경계

- 백엔드 `local-ext`·`local-int` 만 Flyway 자동 적용 대상입니다.
- dev/prod는 DBA가 검토하고 수동 적용합니다.
- 기존 non-empty 로컬 스키마의 기준선은 백엔드 설정과 일치해야 하며 문서 값을 임의로 바꾸지 않습니다.
- `apply-ddl-live.ps1` 은 부트스트랩·수동 재생성 도구이며 일상적인 증분 변경은 Flyway를 사용합니다.
- Data Pump와 DDL export 절차는 [가져오기·내보내기 가이드](#)를 따릅니다.

5. 검증

변경마다 다음을 확인합니다.

1. 파일명 버전이 기존 `migrations/V*.sql` 과 충돌하지 않는지 확인합니다.
2. SQL 객체명과 컬럼 타입을 현재 마이그레이션·백엔드 엔티티와 대조합니다.
3. 대응하는 `docs/verification/*.sql` 또는 마이그레이션 자체 검증 블록을 실행합니다.
4. 로컬 Flyway 적용 뒤 `flyway_schema_history` 의 성공·checksum을 확인합니다.
5. 엔티티나 API 계약이 바뀌면 백엔드 테스트를 함께 실행합니다.

검증 SQL은 결과 건수와 식별 가능한 키만 출력하고 비밀값·개인정보·업무 본문을 출력하지 않습니다.

6. Git 안전

- `EXPDAT.DMP`, live DDL, 로그는 명시된 저장소 정책을 확인한 뒤 변경합니다.
- Git 히스토리 재작성, force-push, 스키마 drop은 일반 문서 정리나 마이그레이션 작성 범위에 포함되지 않습니다.
- 히스토리 정리가 필요하면 [덤프 히스토리 재작성 런북](#)에 따라 별도 승인·공지·신선한 클론에서 수행합니다.

데이터베이스 개발 가이드

SOURCE: [it_database/docs/guides/README.md](#)

데이터베이스 개발 가이드

필수 작업 계약은 [데이터베이스 CLAUDE](#), 로컬 설치와 빠른 시작은 [README](#)가 담당합니다.

반복 가이드

- [로컬 Oracle XE 설정](#)
- [Flyway 마이그레이션 작성과 검증](#)
- [Data Pump와 DDL 가져오기·내보내기](#)

운영 기록

특정 버전의 사전 진단, 유지보수 창 비용, 적용 결과와 복구 절차는 `../operations/`에 둡니다. 실행 가능한 버전별 검증 SQL은 `../verification/`에 둡니다.

작성 원칙

- 가이드에는 여러 변경에서 재사용할 절차만 기록합니다.
- 날짜·배포 상태·실측 건수는 운영 기록으로 분리합니다.
- 비밀번호나 운영 비밀번호를 예시에 넣지 않습니다.
- 적용된 마이그레이션의 내용을 문서가 대신 정의하지 않으며 `migrations/`를 물리 구조 SoT로 사용합니다.

Flyway 마이그레이션 작성과 검증

SOURCE: [it_database/docs/guides/migrations.md](https://it-database/docs/guides/migrations.md)

Flyway 마이그레이션 작성과 검증

파일과 버전

```
migrations/V{YYYYMMDD_NNN}__{CamelCaseDescription}.sql
```

- 같은 날짜의 일련번호는 기존 파일의 최댓값 다음 번호를 사용합니다.
- 설명은 변경 대상과 의도가 드러나는 CamelCase로 작성합니다.
- 적용되었거나 공유된 파일은 수정하지 않고 후속 버전을 추가합니다.
- 병행 작업으로 같은 날짜 번호가 겹칠 수 있으므로, 내용을 채우기 전에 빈 파일을 먼저 커밋해 번호를 선점합니다.
- push된 번호는 개변하지 않습니다. 다른 개발자가 이미 적용했을 수 있으므로 그날의 다음 번호로 새 스크립트를 추가합니다. 개변이 불가피했다면 복구 절차를 [docs/operations/](#)에 남깁니다([2026-08-23 개변 복구 절차](#)).

변경 단위

- Oracle DDL은 implicit commit되므로 원자적 롤백이 필요한 데이터 보정과 분리합니다.
- 데이터 보정은 대상 집합을 안정된 키로 한정하고 변경 전·후 불변식을 검사합니다.
- NOT NULL 전환은 NULL 진단과 backfill을 먼저 수행합니다.
- PK·UNIQUE·대형 인덱스 재생성은 잠금과 유지보수 창 비용을 운영 인계 노트에 기록합니다.
- 멱등 PL/SQL이 필요하다면 "이미 적용됨"과 "실패함"을 구분하고 예상하지 못한 예외를 삼키지 않습니다.

파괴적 DDL 사전·사후 체크리스트

`DROP TABLE`, `DROP COLUMN`, PK·인덱스 재생성, BYTE 길이 축소, 대량 backfill처럼 복구 입력이 필요한 변경은 migration 본문만으로 끝내지 않습니다.

1. 적용 전에 영향 객체·행 수·참조 제약·권한·가용성 창을 기술 키와 건수로 진단하고, 복원 시험을 마친 백업 식별자와 승인자를 운영 인계에 기록합니다.
2. 차단 가능한 위험은 DDL 전에 명시적으로 실패시킵니다. 예외를 삼키거나 `IF EXISTS` 만으로 부분 적용과 정상 적용을 같은 상태로 만들지 않습니다.

3. DDL의 implicit commit 지점과 적용 후 데이터 사전·행 수·제약·인덱스·Flyway 이력 검증을 `docs/operations/` 와 필요 시 `docs/verification/` 에 남깁니다.
4. 재실행 가능 범위를 구분합니다. 일부 단계가 commit된 뒤에도 재실행해도 되는지, 복원은 백업/Flashback/원본 시스템 중 무엇을 입력으로 하는지, 자동 복구할 수 없는 데이터가 무엇인지 명시합니다.
5. 적용된 migration에는 소급 가드를 추가하지 않습니다. checksum을 보존한 채 실제 스키마를 진단하고, 승인된 repair 또는 백업 기반 복구 절차로 분기합니다.

2026-08-29-2026-08-31의 기존 위험 DDL에 대한 환경별 분기는 [파괴적 DDL 소급 운영 인계](#)를 따릅니다.

CREATE TABLE의 BYTE 컬럼 계약

신규 `CREATE TABLE` 또는 `_NEW` 교체 테이블의 `VARCHAR2` 컬럼은 `VARCHAR2(n BYTE)` 로 명시한다. 세션의 `NLS_LENGTH_SEMANTICS` 에 맡긴 bare `VARCHAR2(n)` 은 허용하지 않는다. 같은 migration의 DDL 뒤에는 데이터 사전으로 선언과 실제 적용 결과를 함께 검증하는 블록을 둔다.

아래 블록의 상수만 대상 테이블·컬럼·바이트 길이로 바꿔 복사한다. 컬럼이 없거나 하나보다 많아도 계약 불일치로 실패하도록 행 존재 여부와 semantics·길이를 모두 확인한다.

```

DECLARE
    c_table          CONSTANT VARCHAR2(128) := 'TPRMPP_EXAMPLE';
    c_column         CONSTANT VARCHAR2(128) := 'EXAMPLE_NM';
    c_expected_bytes CONSTANT PLS_INTEGER := 100;
    v_bad            NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_bad
    FROM all_tab_columns
    WHERE owner = 'ITPOWN'
    AND table_name = c_table
    AND column_name = c_column
    AND (char_used <> 'B' OR data_length <> c_expected_bytes);

    IF v_bad > 0 THEN
        RAISE_APPLICATION_ERROR(-20099, 'BYTE 컬럼 계약 불일치');
    END IF;

    SELECT COUNT(*) INTO v_bad
    FROM all_tab_columns
    WHERE owner = 'ITPOWN'
    AND table_name = c_table
    AND column_name = c_column;

    IF v_bad <> 1 THEN
        RAISE_APPLICATION_ERROR(-20099, 'BYTE 컬럼 계약 대상 누락 또는 중복');
    END IF;
END;
/

```

검증 SQL은 `ALL_TAB_COLUMNS` 결과와 Flyway 성공 이력을 확인하되, 업무 문자열이나 개인정보를 출력하지 않는다. 여러 컬럼이면 대상마다 검증하거나 대상 목록을 가진 반복 블록으로 같은 조건을 검사한다.

감사와 개인정보

- 애플리케이션 listener 밖에서 수행하는 보정은 Flyway 이력과 승인 기록을 감사 근거로 남깁니다.
- 보정 주체가 필요하면 승인된 고정 식별자를 사용하고 변경 시각을 함께 기록합니다.
- 진단 출력에는 건수·상태·기술 키만 포함하고 제목·본문·개인정보를 출력하지 않습니다.

적용과 실패 복구

- 로컬은 백엔드 `local-ext` 또는 `local-int` 기동으로 적용합니다.
- dev/prod는 DBA가 검토 후 수동 적용합니다.
- 실패 이력을 `repair` 하기 전 스키마의 실제 상태와 checksum 불일치 원인을 확인합니다.
- destructive 변경이나 대량 보정은 `docs/operations/` 에 사전 점검, 적용, 검증, 실패 시 재시도 또는 롤백을 기록합니다.

검증 체크리스트

1. 버전과 파일명 충돌이 없는지 확인합니다.
2. 객체명·타입·기본값을 현재 마이그레이션과 백엔드 엔티티에 대조합니다.
3. 사전 진단에서 변경 대상 규모와 위반 불변식을 확인합니다.
4. 적용 뒤 객체 상태, 제약, 인덱스, 대상 건수와 `flyway_schema_history` 를 확인합니다.
5. 관련 `docs/verification/*.sql` 과 백엔드 테스트를 실행합니다.

1. 프로젝트와 저장소

SOURCE: CLAUDE.md

[프로젝트 메인 가이드] 이 파일은 IT Project Portal 4개 저장소의 공통 작업 계약과 문서 단일 출처를 정의합니다. 백엔드 작업은 `it_backend/CLAUDE.md`, 프론트엔드 작업은 `it_frontend/CLAUDE.md`, DB 작업은 `it_database/CLAUDE.md`를 추가로 읽습니다.

1. 프로젝트와 저장소

IT Project Portal은 정보화 예산·사업·인력과 관리자 실시간 로그를 관리합니다.

저장소	책임	상세 규칙
<code>C:\it</code>	공통 문서·도구·제품 요구사항	이 파일
<code>it_backend</code>	Spring Boot API	<code>it_backend/CLAUDE.md</code>
<code>it_frontend</code>	Nuxt 4 CSR UI	<code>it_frontend/CLAUDE.md</code>
<code>it_database</code>	Oracle DDL·시드·Flyway 마이그레이션	<code>it_database/CLAUDE.md</code>

세 하위 디렉터리는 각각 독립 원격 저장소이며 루트 Git에는 포함되지 않습니다. 호환 커밋 조합은 `versions.lock`에 기록하고 `scripts/update-versions-lock.ps1`로 갱신합니다. 교차 저장소 변경은 백엔드 API 계약을 먼저 확정하고 프론트 타입과 소비 코드를 뒤이어 갱신합니다.

주요 루트 경로:

- `docs/`: 리포트·운영 기록·Superpowers 산출물
- `.agents/skills/`: 반복 사용하는 프로젝트 공용 워크플로우 스킬
- `meta/`: 엔티티·컬럼 명명 SoT와 코드·조직·테이블 목록
- `prds/`: 제품 요구사항
- `FP/`: 기능점수 규칙과 산출물
- `tools/`, `scripts/`: 분석·운영 도구
- `TASK.md`, `TASK_DONE.md`: 활성 과제와 완료 기록

2. 단일 진실 공급원

주제	SoT
API-엔티티-서버 인증·운영 비밀값	<code>it_backend/CLAUDE.md</code> 와 백엔드 코드
UI-라우팅-클라이언트 인증 책임	<code>it_frontend/CLAUDE.md</code> 와 프론트엔드 코드
물리 데이터 모델	<code>it_database/migrations/</code>
ORM 매핑	백엔드 엔티티
조회용 데이터 모델 인덱스	<code>it_backend/docs/guides/persistence/data-model.md</code>
엔티티-컬럼 명명	<code>meta/meta.txt</code>
컴포넌트 사용법	<code>it_frontend/docs/guides/components/</code>
미구현-기술부채	<code>TASK.md</code>

문서 역할은 다음처럼 구분합니다.

- `README.md`: 사람을 위한 설치·실행·구조 개요·문제 해결
- `CLAUDE.md`: 에이전트가 반드시 지킬 규칙·금지 사항·검증 명령
- `docs/guides/`: 반복해서 참고하는 구현법·계약·예제
- `docs/operations/`: 날짜가 있는 배포·복구·인계 기록
- `docs/superpowers/`: 설계와 구현 계획

같은 설명을 복제하지 말고 SoT를 링크합니다. 파일·테스트 개수나 완료 날짜 같은 시점성 정보는 CLAUDE에 넣지 않습니다.

3. 공통 작업 규칙

공유 워킹트리

여러 작업이 동시에 네 워킹트리를 공유하므로 무관한 변경이 존재하는 것이 정상입니다.

- `git add -A`, `git add .`, `git commit -a`를 사용하지 않고 경로를 명시합니다.
- 한 파일에 변경이 섞이면 `git add -p` 또는 `cached patch`로 자기 hunk만 스테이징합니다.
- 커밋 전 `git diff --cached --stat` 과 `git rev-parse --abbrev-ref HEAD` 를 확인합니다.
- 사용자나 다른 작업의 변경을 되돌리거나 정리하지 않습니다.
- 이미 푸시된 혼합 커밋은 이력을 재작성하지 말고 `TASK_DONE.md` 에 대응 관계를 기록합니다.

코드와 주석

- 신규 JavaDoc·TSDoc·인라인 주석은 한글로 작성합니다.
- 단순 대입이나 자명한 메서드에는 주석을 추가하지 않습니다.
- 공개 API, 서비스 메서드, composable 반환 함수에는 입력과 실패 조건을 기록합니다.
- TODO/FIXME는 후속 조치가 가능한 문장으로 쓰고 장기 과제는 `TASK.md`에 등록합니다.
- 문서·주석 현행화 작업에서 별도 요청 없이 비즈니스 로직을 바꾸지 않습니다.

인증과 비밀값

- 브라우저 토큰은 백엔드 httpOnly 쿠키로 전달하며 프론트엔드 저장소에서 읽거나 저장하지 않습니다.
- 프론트 라우트 가드는 UX 보호일 뿐이며 서버가 JWT와 서비스 계층에서 최종 권한을 검증합니다.
- 운영 비밀값은 환경변수 또는 비공개 프로파일로 주입합니다.
- Oracle 비밀번호를 명령행 인자, 셸 히스토리, 문서, 로그에 기록하지 않습니다. 수동 작업은 콘솔 프롬프트, CI·무인 실행은 Wallet 별칭을 사용합니다.
- 상세 인증 정책은 `it_backend/CLAUDE.md`와 `it_backend/docs/guides/security/`를 따릅니다.

데이터베이스 변경

- 물리 변경은 `it_database/migrations/V{YYYYMMDD_NNN}_{CamelCaseDescription}.sql`로 추가합니다.
- 적용된 Flyway 스크립트는 수정하지 않고 새 버전으로 변경합니다.
- ITPOWN의 데이터베이스 문자셋은 `AL32UTF8`, `VARCHAR2` 길이 기준은 `BYTE semantics`입니다. 신규·변경 DDL은 세션 기본값에 맡기지 않고 `VARCHAR2(n BYTE)`를 명시하며, 저장 가능 여부는 글자 수가 아니라 UTF-8 바이트 길이와 `LENGTHB`로 확인합니다.
- 엔티티·검증·운영 적용 규칙은 `it_database/CLAUDE.md`와 `it_database/docs/guides/migrations.md`를 따릅니다.

4. 작업 흐름

1. 요청 범위의 CLAUDE와 가이드를 읽습니다.
2. 코드·설정·테스트에서 문서의 사실관계를 확인합니다.
3. 기존 공통 구현과 SoT를 검색한 뒤 최소 범위로 변경합니다.
4. 변경 유형에 맞는 테스트와 Health Stack을 실행합니다.
5. 문서·OpenAPI 타입·마이그레이션 등 함께 바뀌는 계약을 확인합니다.

성능·계약·실패 상태를 바꾸는 구현은 다음 공통 기준을 지킵니다.

- 목록 API는 DB 필터·명시적 안정 정렬·유한한 페이지 또는 상한을 함께 적용하고, bulk API는 IN 배치 조회로 N+1을 만들지 않습니다.
- 백엔드 DTO/OpenAPI 계약을 먼저 확정된 뒤 프론트에서 `npm run codegen`과 `npm run codegen:check`를 실행합니다. 생성 타입은 수기로 편집하지 않습니다.
- 빈 결과·누락 값·잘못된 입력·조회 실패를 같은 상태로 취급하지 않습니다. malformed 값은 정상 기본 값이나 미존재로 숨기지 말고 진단·경고·차단 중 계약에 맞게 표면화합니다.

- 초기 조회 실패는 정상 빈 화면으로 대체하지 않고 재시도 가능한 오류 상태로 유지하며, 재조회 실패 때 이전 성공 데이터를 새 성공처럼 표시하지 않습니다.
- 정보화사업·경상사업은 BPROJM(ABUS_MNG_NO, SNO) 의 순번으로 재상신 이력을 식별하며 ODN_YN='Y' 가 경상사업, NULL 또는 N 이 정보화사업입니다. 전산업무비는 BCOSTM(BG_NO, BG_SNO) 를 사용합니다. 후속 업무 목록·집계·bulk 조회는 LST_YN='Y' 최종본만 사용하고, 미상신 작성 목록은 재상신 초안을 포함해 응답 순번을 보존합니다. 이력 조회·수정·결재 매핑도 관리번호와 순번을 함께 사용합니다.

Superpowers 산출물은 docs/superpowers/{specs,plans,done}/ 에 둡니다. 프로젝트 규칙과 일반 스킴이 충돌하면 이 저장소의 CLAUDE와 docs/guides/ 가 우선합니다.

5. Health Stack

루트에서 필요한 저장소만 선택해 실행합니다.

```
# 백엔드
cd C:\it\it_backend
./gradlew test

# 프론트엔드
cd C:\it\it_frontend
npm run format:check
npm run check
npm test

# 호환 버전 기록
cd C:\it
./scripts/update-versions-lock.ps1
```

전체 설치·기동 URL·IDE 설정은 README.md, 세부 검증 명령은 각 하위 저장소 CLAUDE와 테스트 가이드를 따릅니다.